INSTITUTO DE EDUCACION SUPERIOR TECNOLÓGICO PRIVADO “CIBERTEC”

FACULTAD DE TECNOLOGÍAS DE LA INFORMACIÓN

CARRERA DE COMPUTACIÓN E INFORMÁTICA

PROYECTO DE INVESTIGACIÓN APLICADA

“Desarrollo de una aplicación web para la gestión musical de la plataforma Melodic Vault, Lima, 2026”

Autores:

Vidal Merino Carlos

Román Quispe André Fabrizzio

Garay Alzamora Leonardo Joaquin

Burgos Villon Cesar Jesus

Zelada Ruiz Ryk Anthony

Docente:

Mtro. (c) Bocanegra Pinchi Yan Carlos

Curso:

Lenguaje de Programación II

Lima - Perú

2026

Índice

Índice …… ii
1. Resumen 3
2. Introducción 4
3. Diagnóstico 5
4. Objetivos 6
4.1. Objetivo general 6
4.2. Objetivos específicos 7
5. Justificación del proyecto 7
6. Definición y alcance 8
7. Productos y entregables 11
7.1. Metodología de desarrollo del proyecto 12
7.2. Análisis funcional del sistema 13
7.3. Diseño de base de datos y persistencia JPA 14
7.4. Configuración del proyecto con Maven 15
7.5. Control de versiones con Git y GitHub 16
7.6. Implementación web con Spring Boot, MVC, JPA e Hibernate 17
7.6.1. Arquitectura del sistema 17
7.6.2. Gestión de sesiones y roles 18
7.6.3. Implementación de mantenimientos 19
7.6.4. Implementación de consultas 20
7.6.5. Implementación del caso transaccional 21
7.7. Implementación de reportes con JasperReports 22
7.8. Despliegue e integración continua en Azure con GitHub Actions 23
7.9. Video Demo Reel estructurado 24
7.10. Checklist de cumplimiento técnico 25
8. Conclusiones 26
9. Recomendaciones 27
10. Glosario 28
11. Bibliografía 29
12. Anexos 30

1. Resumen

Melodic Vault fue un Sistema Web de Gestión de Catálogo Musical desarrollado con el propósito de centralizar, organizar y facilitar la consulta de información relacionada con bandas, álbumes y canciones, principalmente orientadas a géneros como Hard Rock, Melodic Hard Rock y AOR.

El objetivo general del proyecto fue desarrollar una aplicación web para la gestión musical de la plataforma Melodic Vault, que permitiera administrar de manera eficiente el catálogo mediante mantenimientos de bandas, álbumes y canciones, con control de acceso por roles y generación de reportes.

Para organizar el desarrollo se adoptó la metodología ágil Scrum, con tres sprints entre el 14 de septiembre y el 10 de octubre, lo que permitió implementar, probar y mejorar progresivamente los diferentes módulos del sistema.

La solución se desarrolló con Java 17 y Spring Boot, aplicando el patrón MVC. Para la capa de vista se utilizó Thymeleaf, mientras que la persistencia se implementó mediante Spring Data JPA e Hibernate, con MySQL como gestor de base de datos. La autenticación y los roles se resolvieron con Spring Security, con contraseñas cifradas con BCrypt; los reportes se generaron con JasperReports; Maven administró las dependencias y el ciclo de construcción, y Git y GitHub gestionaron el control de versiones.

Entre las funcionalidades implementadas se encontraron los tres mantenimientos (bandas, álbumes y canciones) con registro, consulta, actualización y eliminación; la asociación de cada álbum con su banda y de cada canción con su álbum; las búsquedas de bandas por nombre, país o género y de álbumes por título, tipo o banda; el registro público de usuarios con rol Lector; el inicio de sesión con roles Administrador y Lector; el registro de un álbum con sus canciones en una sola transacción; dos reportes en PDF (álbumes de una banda y canciones de un álbum); y un panel principal con indicadores generales del catálogo.

Como resultado, se obtuvo una aplicación web funcional, estructurada por capas y con capacidad de crecimiento.

Se concluyó que Melodic Vault demostró la aplicación práctica de tecnologías Java Web para transformar información musical dispersa en un catálogo digital organizado, seguro y consultable.

2. Introducción

La disponibilidad de información musical en Internet ha permitido que los usuarios puedan consultar discografías, datos de artistas, fechas de lanzamiento y otros contenidos relacionados con sus géneros favoritos. Sin embargo, en determinados géneros y escenas musicales, especialmente Hard Rock, Melodic Hard Rock y AOR, esta información puede encontrarse distribuida entre diferentes portales, bases de datos y sitios especializados, dificultando disponer de una consulta organizada y centralizada de bandas y sus respectivos álbumes.

A partir de esta situación surgió Melodic Vault – Sistema Web de Gestión de Catálogo Musical, una propuesta orientada a organizar y administrar información de bandas y producciones discográficas mediante una aplicación web. El proyecto tomó como referencia conceptual la utilidad de los catálogos musicales especializados, pero planteó una solución propia desarrollada con tecnologías Java Web que permitió gestionar la información almacenada y presentar al usuario una experiencia de consulta clara, visual y estructurada.

La digitalización de este proceso permitió reemplazar registros dispersos por información relacionada dentro de una base de datos. De esta manera, cada banda pudo mantener sus datos principales y asociarse con los álbumes que formaban parte de su discografía, lo que facilitó las operaciones de registro, actualización, eliminación, búsqueda y consulta detallada.

Para su implementación se empleó Java y Spring Boot como tecnologías principales del backend, aplicando el patrón Model-View-Controller (MVC) para mantener una adecuada separación de responsabilidades. La presentación de información se realizó mediante Thymeleaf, mientras que el acceso y la persistencia de los datos se gestionaron mediante Spring Data JPA, Hibernate y MySQL. Asimismo, Maven facilitó la administración de dependencias y la construcción del proyecto.

El impacto esperado de Melodic Vault fue proporcionar exploración de información musical, ofreciendo a los usuarios una experiencia de catálogo moderno y especializado, con contenidos más detallados, ordenados y visualmente accesibles. De este modo, los aficionados y usuarios que buscaban conocimiento de rock podrían localizar con mayor facilidad información relevante sobre sus bandas, álbumes, géneros y trayectorias musicales.

3. Diagnóstico

El entorno musical ha experimentado una transformación significativa como consecuencia de la digitalización y del crecimiento de las plataformas en línea. Actualmente, los usuarios disponen de una gran cantidad de información relacionada con artistas, álbumes, géneros y producciones musicales; sin embargo, el incremento del contenido digital también genera dificultades relacionadas con su organización, clasificación y descubrimiento, especialmente cuando se trata de géneros musicales especializados.

En este contexto, Melodic Vault buscó aprovechar las posibilidades que ofrecía el entorno digital mediante una aplicación web orientada a organizar información de bandas y álbumes dentro de un catálogo estructurado. Para identificar los factores externos relacionados con esta necesidad se realizó un análisis SEPTE/PESTEL considerando las dimensiones sociales, económica, político/legal y tecnológica.

Tabla *1*

Análisis SEPTE / PEST del proyecto

Dimensión	**Situación identificada**	**Evidencia o fuente consultada**	**Oportunidad de mejora para el sistema**
Social	Los entornos digitales se convirtieron en nuevos hábitats, espacios vitales para la comunicación, el intercambio de pensamientos y la puesta en común de ideas, y en un mediador clave para las prácticas artísticas, religiosas y tradicionales.	UNESCO (2022)	Melodic Vault puede contribuir mediante un catálogo especializado que permita consultar y descubrir bandas y álbumes organizados por información como género, país y discografía, facilitando el acceso a contenidos musicales específicos.
Económica	Menor presencia comercial de musica de decadas anteriores, frente a lanzamientos de mayor demanda, lo que reduce el incentivo economico destinados a su publicidad, promoción, difusión	IPEI (2019)	Oportunidad para organizar y dar mayor visibilidad a bandas, álbumes y géneros que actualmente poseen menor exposición comercial, mediante un catálogo web especializado

	digital.		que facilite su consulta y descubrimiento sin depender exclusivamente de campañas publicitarias o de su presencia en tendencias musicales actuales.
Política	Las políticas públicas orientadas a la transformación digital y al fortalecimiento del uso de tecnologías digitales generan un entorno favorable para el desarrollo y adopción de soluciones y servicios mediante plataformas web.	Gob.pe (2024)	Este entorno permite que pueda desarrollarse bajo un enfoque digital, aprovechando tecnologías actuales para organizar, administrar y facilitar el acceso a información musical especializada.
Legal	La utilización de nombres, imágenes, portadas, descripciones y otros contenidos relacionados con obras musicales dentro de una plataforma digital requiere considerar los derechos de autor y de propiedad intelectual, especialmente respecto al uso de material perteneciente a terceros.	Decreto Legislativo N°822 (2021)	Melodic Vault gestiona responsablemente los recursos incorporados al catálogo, priorizando información descriptiva, referencias y recursos cuyo uso sea permitido, evitando almacenar, reproducir o distribuir material protegido sin la autorización correspondiente.
Tecnológica	El crecimiento del acceso a Internet y del uso de tecnologías digitales facilita que los usuarios consulten información y accedan a contenidos mediante plataformas web.	INEI (2025)	Se aprovecha el nivel de acceso mediante una aplicación web que permita centralizar, administrar, buscar y consultar información sobre bandas, álbumes y géneros.

Nota. Elaboración a partir de información de UNESCO (2022) , IPEI (2019), Gob.pe (2024), Decreto Legislativo N°822 (2021), INEI (2025), World Wide Web Consortium (2026)

INTERPRETACIÓN

A partir del análisis PESTEL realizado, se identificó como oportunidad de mejora la necesidad de contar con un espacio digital especializado que facilitara la organización, administración, consulta y descubrimiento de información sobre bandas, álbumes y géneros musicales, especialmente de aquellos pertenecientes a escenas, épocas o estilos que podían tener menor visibilidad comercial.

El sistema consideró aspectos relacionados con los hábitos de consumo digital, la protección de la propiedad intelectual, el aprovechamiento de las tecnologías web y la aplicación de buenas prácticas de sostenibilidad digital. De esta manera, Melodic Vault buscó proporcionar una solución organizada y accesible para la gestión y el descubrimiento de información musical especializada.

4. Objetivos

4.1. Objetivo general

Desarrollar una aplicación web para la gestión musical de la plataforma Melodic Vault, Lima, 2026.

4.2. Objetivos específicos

Analizar los requerimientos funcionales y no funcionales necesarios para la gestión y consulta de información de bandas y álbumes dentro del catálogo musical Melodic Vault.

Diseñar el modelo de base de datos, los casos de uso y la arquitectura de la aplicación, estableciendo las relaciones entre bandas, álbumes y las nuevas entidades requeridas por el sistema.

Implementar los módulos de mantenimiento, consultas y operaciones principales utilizando Java, Spring Boot, Spring MVC, Spring Data JPA, Hibernate y MySQL.

Implementar reportes de información relevante del catálogo mediante JasperReports, permitiendo consultar y presentar los datos almacenados en el sistema.

Gestionar el código fuente y sus diferentes versiones mediante GitHub e integrar un proceso de construcción y despliegue mediante GitHub Actions.

5. Justificación del proyecto

Melodic Vault surgió como respuesta a la necesidad de organizar y facilitar la consulta de información sobre bandas, álbumes y canciones dentro de un catálogo musical especializado. Aunque Internet ofrece gran cantidad de información musical, esta se encuentra distribuida entre diferentes portales, plataformas y fuentes, lo que dificulta su consulta estructurada, especialmente cuando se busca información de determinados géneros o escenas musicales, como el hard rock melódico y el AOR. El proyecto utilizó tecnologías web para centralizar esta información y gestionarla, relacionarla y consultarla de manera organizada.

5.1. Justificación práctica

La implementación de Melodic Vault centralizó la información de bandas, álbumes y canciones en una base de datos estructurada, evitó registros aislados y facilitó su administración mediante una interfaz web accesible desde cualquier navegador.

El sistema ofreció registro, actualización, consulta y eliminación de información, además de búsquedas por nombre, país o género en bandas, y por título, tipo o banda en álbumes. Las relaciones entre bandas, álbumes y canciones permitieron navegar desde un artista hacia su discografía y desde un álbum hacia sus canciones. El registro de un álbum con sus canciones se resolvió en una sola transacción: se guardó completo o no se guardó.

La autenticación y los roles diferenciaron las operaciones administrativas de las de consulta, protegiendo la integridad de la información. Los reportes en PDF permitieron obtener las canciones de un álbum y los álbumes de una banda.

5.2. Justificación tecnológica

Se utilizaron Java y Spring Boot por permitir aplicaciones web con arquitectura organizada, modular y escalable. El patrón MVC separó presentación, procesamiento de solicitudes y lógica de datos. Spring Data JPA e Hibernate facilitaron la persistencia mediante

un modelo orientado a objetos, y MySQL almacenó los registros del catálogo. Spring Security gestionó autenticación y roles, con contraseñas cifradas con BCrypt. Maven administró dependencias y construcción, y JasperReports generó los reportes.

Estas tecnologías permitieron una solución web ordenada, escalable y alineada con el curso.

5.3. Beneficiarios directos

Administrador: gestionó el catálogo, realizó el mantenimiento y descargó reportes.

Lector (usuario registrado): consultó el catálogo y descargó reportes en PDF.

Visitante: consultó el catálogo sin crear una cuenta.

5.4. Beneficiarios indirectos

Aficionados, investigadores, estudiantes y personas interesadas en información musical organizada, quienes se beneficiaron del catálogo sin participar en su administración.

6. Definición y alcance

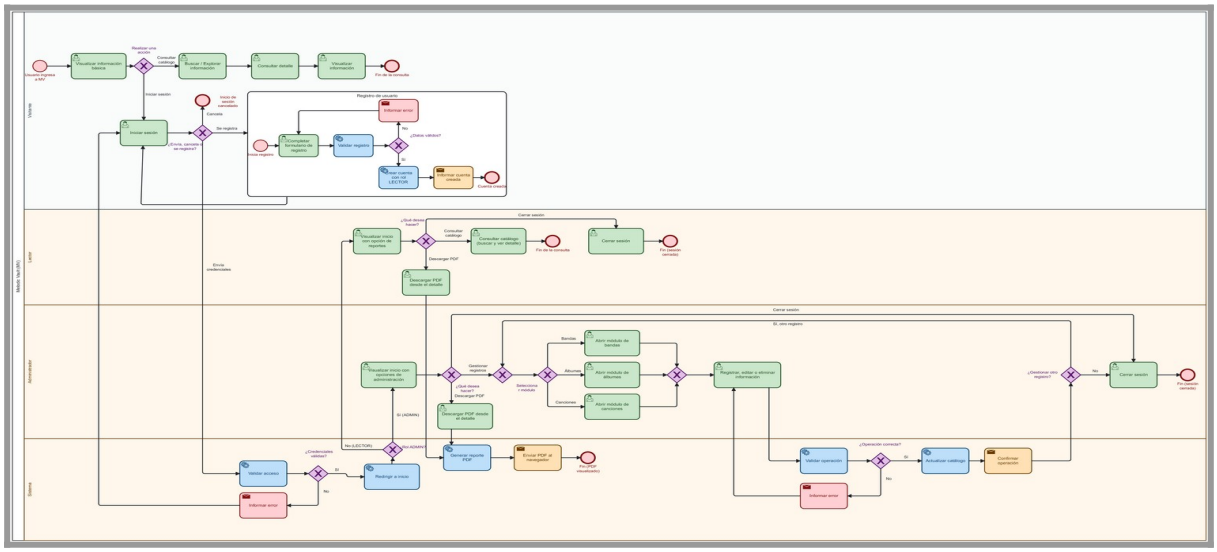
6.1. Descripción general del sistema

Melodic Vault fue una aplicación web de gestión de un catálogo musical, desarrollada para atender la dispersión de la información sobre bandas, álbumes y canciones. Permitió almacenar esa información de forma estructurada, relacionar cada álbum con su banda y cada canción con su álbum, y ofreció búsqueda, consulta y visualización detallada.

La consulta del catálogo fue pública. Cualquier visitante pudo registrarse como Lector, rol que además permitió descargar reportes. El registro, la edición y la eliminación quedaron restringidos al Administrador mediante autenticación y control de acceso por roles. El sistema también permitió registrar un álbum con sus canciones en una sola operación transaccional y generó dos reportes con JasperReports: los álbumes de una banda y las canciones de un álbum.

Se construyó con Spring Boot y el patrón MVC, con MySQL como base de datos relacional.

Diagrama 1: BPMN del proceso AS-IS



INTERPRETACION

El diagrama modela el recorrido completo de Melodic Vault en cuatro carriles: Visitante, Lector, Administrador y Sistema. El Visitante pudo consultar el catálogo sin sesión (buscar, ver el detalle y terminar) o ir a iniciar sesión. Desde ahí tuvo tres salidas: enviar credenciales, cancelar o registrarse. El registro se representó como un subproceso en el que el Sistema validó los datos, informó los errores y devolvió al formulario, o creó la cuenta siempre con rol LECTOR y volvió al inicio de sesión.

El Sistema validó las credenciales. Si eran incorrectas, informó el error y regresó al login. Si eran correctas, redirigió al inicio y una compuerta decidió el carril según el rol: el rol lo determinó el Sistema y no el usuario.

El Lector consultó el catálogo, descargó PDF desde el detalle o cerró sesión. El Administrador, además de descargar PDF, eligió un módulo (bandas, álbumes o canciones) y registró, editó o eliminó información. El Sistema validó cada operación y, si era correcta, actualizó el catálogo y confirmó; si no, informó el error y devolvió al administrador al registro. El ciclo se repitió hasta cerrar sesión.

Ambos roles convergieron en la generación del PDF, que el Sistema envió al navegador.

6.2. Alcance funcional

El alcance funcional de Melodic Vault comprendió:

- Conexión y persistencia de información en una base de datos MySQL.

- Gestión de autenticación y sesiones.

- Registro público de usuarios, siempre con rol Lector.

- Gestión de roles (Administrador y Lector) y permisos de acceso.

- Tres mantenimientos (bandas, álbumes y canciones) con registro, consulta, actualización y eliminación.

- Asociación de cada álbum con su banda y de cada canción con su álbum.

- Consultas especializadas: búsqueda de bandas por nombre, país o género; búsqueda de álbumes por título, tipo o banda; y estadísticas por banda (cantidad de álbumes y rating promedio).

- Dos reportes en PDF con JasperReports: álbumes de una banda y canciones de un álbum.

- Caso de uso transaccional: registro de un álbum con sus canciones en una sola operación, que se guardó completa o no se guardó.

- Panel principal con indicadores generales del catálogo.

- Control de versiones con GitHub.

Quedaron fuera del alcance la reproducción directa de contenido musical, la integración con servicios de streaming, el comercio electrónico, la aplicación móvil nativa, las recomendaciones mediante inteligencia artificial y las funcionalidades sociales avanzadas, consideradas como posibles ampliaciones.

6.3. Alcance técnico

Tabla *2*

Alcance técnico

Capa	Tecnología	Función en Melodic Vault	Valor para el proyecto
Lenguaje	Java 17	Codificó las clases Banda, Album, Cancion y Usuario, los servicios y los controllers.	Lenguaje tipado, estable y robusto, muy usado en entornos empresariales.
Framework	Spring Boot 3.3.4	@SpringBootApplication en MelodicVaultProject activó la configuración, el escaneo de componentes (desde el paquete raíz pe.cibertec.melodicvault) y la autoconfiguración. Esta detectó starter-data-jpa y mysql-connector-j y levantó el DataSource, el EntityManagerFactory y el gestor de transacciones. El Tomcat embebido sirvió la aplicación en el puerto 8090.	Sin infraestructura manual: ni fábrica de EntityManager, ni conexión en código, ni servidor externo. spring-boot-starter-parent fijó versiones compatibles entre las librerías.
Arquitectura web	Spring MVC	El DispatcherServlet recibió cada petición y la derivó al controller según la ruta. Ocho controllers (Home, Login, Registro, Banda, Album, AlbumCompleto, Cancion, Reporte) siguieron el flujo controller → service → repository → modelo.	Cada capa tuvo una sola responsabilidad: los controllers no tocaron datos y los repositorios no conocieron la web.
Acceso a datos	Spring Data JPA	Por cada interfaz (IBanda, IAlbum, ICancion, IUsuario) Spring generó un proxy en el arranque. El CRUD básico fue a SimpleJpaRepository, los métodos como findByNombreContainingIgnoreCase… se interpretaron por nombre (PartTree) y las consultas @Query resolvieron las estadísticas por banda y las canciones de un álbum.	Evitó SQL y CRUD repetitivo: el proxy (JDK Proxy) eligió la implementación adecuada y solo se declararon las interfaces.
Mapeo		Mapeó Banda, Album, Cancion y	Se trabajó con objetos y no

objeto-relacional	Hibernate	Usuario a sus tablas con dos relaciones uno a muchos bidireccionales: Banda → Album (@OneToMany con cascada y LAZY; @ManyToOne dueño de la FK) y Album → Cancion (@OneToMany ; @ManyToOne). Tradujo la navegación entre entidades (banda.getAlbumes(), a.banda.nombre) a JOIN, propagó la eliminación en cascada y declaró la pista única por álbum con @UniqueConstraint.	con SQL. Las reglas del modelo quedaron en las entidades: un álbum no existe sin banda, una canción no existe sin álbum y la eliminación se propaga en cascada.
Vistas	Thymeleaf	Generó las páginas dinámicas, con una plantilla HTML por pantalla. Los formularios de banda, álbum y canción sirvieron para registrar y editar: según el id oculto, save() insertó o actualizó; th:field enlazó con la entidad y th:errors mostró las validaciones. sec:authorize ocultó botones según la sesión.	Integración nativa con Spring y con la seguridad por rol. Un mismo formulario atendió dos operaciones. Limitación: sin fragmentos compartidos, cada plantilla repitió cabecera, menú y pie.
Interfaz	Bootstrap 5 y Bootstrap Icons	Diseño adaptable e iconografía.	Interfaz consistente sin desarrollo gráfico desde cero.
Seguridad	Spring Security y BCrypt	Un SecurityFilterChain interceptó cada petición antes del controller: consulta pública, reportes con sesión y mantenimiento solo para ADMIN. UsuarioDetailsService validó contra la tabla usuario, las claves se guardaron como hash BCrypt y el registro público siempre asignó el rol LECTOR.	Control de acceso aplicado en el servidor, sin depender de lo que mostrara la vista.
Base de datos	MySQL	Cuatro tablas (banda, album, cancion, usuario). La restricción	Integridad garantizada en la base, no solo en la

		UNIQUE (id_album, numero_pista) impidió repetir una pista dentro de un álbum.	aplicación.
Construcción	Maven	Resolvió las dependencias del pom.xml y ejecutó el ciclo de construcción (validar, compilar, probar, empaquetar). Cada starter (web, data-jpa, security, thymeleaf, validation) trajo sus librerías de forma transitiva. Con esas librerías en el classpath, @SpringBootApplication activó la autoconfiguración.	Compilación reproducible en cualquier equipo.
Reportes	JasperReports 6.20.6	Generó en PDF el reporte de álbumes de una banda y el de canciones de un álbum, a partir de plantillas .jrxml compiladas al iniciar la aplicación.	Información del catálogo lista para presentar y compartir.
Control de versiones	Git y GitHub	Repositorio del código fuente.	Historial de cambios, trazabilidad y respaldo.

INTERPRETACIÓN

El pom.xml fue el punto de partida: declaró los starters, que trajeron de forma transitiva todas las librerías, y spring-boot-starter-parent fijó versiones compatibles. Con esas librerías en el classpath, @SpringBootApplication activó la autoconfiguración, que levantó el DataSource, el EntityManagerFactory, el gestor de transacciones y el Tomcat embebido. Sobre esa base, Spring Data JPA generó los proxies de los repositorios, que trabajaron con las entidades mapeadas por Hibernate a las tablas de MySQL, cuyas restricciones (como la pista única por álbum) respaldaron lo declarado en el modelo. Encima de los datos, los services aplicaron las reglas de negocio y las transacciones, Spring MVC enrutó cada petición a su controller, Thymeleaf armó la respuesta, JasperReports la convirtió en PDF cuando correspondía y Spring Security vigiló todo el recorrido desde la entrada. Cada capa resolvió

lo que la anterior dejaba abierto: Maven, las librerías; la autoconfiguración, su conexión; JPA, el acceso a datos; Hibernate, el SQL; y las vistas y la seguridad convirtieron ese acceso en un sistema usable y con permisos.

La misma cadena mostró dónde el sistema fue débil y por dónde pudo crecer. Los detalles recorrieron relaciones LAZY que dependieron de open-in-view; cargarlas con @EntityGraph, como en el reporte de banda, las independizaría de esa configuración. Las vistas repitieron cabecera, menú y pie, y los fragmentos de Thymeleaf lo corregirían sin tocar las capas inferiores. Como el controller no tocó datos y el service no conoció la web, una API REST reutilizaría services y repositorios cambiando solo la capa de entrada y la seguridad. Y como ddl-auto=update no resultó confiable con restricciones, el paso natural sería versionar el esquema con Flyway.

6.4. Limitaciones del proyecto

El proyecto tuvo como principal limitación el tiempo disponible, pues debió completarse dentro del calendario académico, lo que restringió la implementación y validación de funcionalidades adicionales a las requeridas por el curso.

El mantenimiento de canciones obligó al administrador a elegir el álbum de una lista y a escribir manualmente el número de pista.

Con un catálogo de cientos o miles de álbumes, esa lista sería poco manejable y aumentaría el riesgo de repetir una pista. Además, el rol Lector se limitó a consultar y descargar reportes: no pudo registrar álbumes ni modificar los que él mismo hubiera aportado, por lo que todo el mantenimiento recayó en el Administrador.

Por tratarse de una propuesta académica y no de un sistema para una empresa musical específica, la información almacenada se compuso de datos públicos y registros de prueba y demostración, por lo que no hubo validación operativa en una organización real ni con usuarios finales.

El sistema se ejecutó en entorno local y requirió conexión a internet para cargar Bootstrap desde CDN. Las imágenes y portadas, protegidas por derechos de autor, se utilizaron solo con fines académicos y enlazadas desde sus fuentes originales.

6.5. Documentación y evidencias a entregar

Como evidencia del análisis, diseño, implementación y funcionamiento de Melodic Vault se entregaron el diagrama de casos de uso y sus especificaciones, el modelo de base de datos, el script SQL (bd_melodicvault.sql), los diagramas de arquitectura, las capturas de las funcionalidades, el código fuente, el repositorio GitHub y los reportes generados con JasperReports.

7. Productos y entregables

7.1. Metodología de desarrollo del proyecto

Melodic Vault se desarrolló con la metodología ágil Scrum, organizada en tres sprints entre el 14 de septiembre y el 10 de octubre. El equipo estuvo conformado por cuatro integrantes: Leonardo Garay (Product Owner), Fabrizzio Román (Scrum Master), y Carlos Vidal y César Burgos (desarrolladores).

El Product Backlog se construyó a partir de la tabla de requerimientos funcionales: 21 requerimientos, de los cuales 11 fueron de prioridad alta, 9 media y 1 baja. El Product Owner los ordenó según su prioridad. Al inicio de cada sprint, el equipo seleccionó los requerimientos que entraban, definió el objetivo del sprint y lo desglosó en historias de usuario. Durante el sprint se hizo seguimiento del avance, y al cierre se elaboró un acta de Sprint Review con la duración, el equipo, el objetivo y el resultado.

Tabla *3*

Sprints realizados en Melodic Vault

Sprint	Fechas	Duración	Objetivo	Requerimientos	Resultado
1	14 al 27 de septiembre	2 semanas	El administrador gestionó bandas, álbumes y canciones, y el visitante consultó el catálogo sin registrarse.	RF-04 a RF-17	Cumplido
2	29 de septiembre al 5 de	1 semana	Los usuarios con cuenta iniciaron sesión y accedieron a las funciones de su	RF-01 a RF-03 y RF-18	Cumplido

	octubre		rol.		
3	6 al 10 de octubre	5 días	Un visitante se registró como Lector y los usuarios registrados descargaron en PDF los reportes del catálogo.	RF-19 a RF-21	Cumplido

Interpretación

Un incremento se consideró terminado cuando cumplía su requerimiento, funcionaba sin errores en el flujo principal, respetaba los permisos por rol, se integraba al repositorio compartido y podía demostrarse. Cada sprint entregó una parte funcional: la primera versión del catálogo, el inicio de sesión con control por roles, y el registro de usuarios con los reportes. La experiencia del primer sprint se utilizó para planificar los dos siguientes.

El equipo trabajó de forma asíncrona y sin reuniones presenciales. Por esa razón no se aplicó planning poker como técnica de estimación: el Product Owner ordenó el backlog por prioridad, y el cierre de cada sprint se documentó en el acta de Sprint Review, sin reunión. El código se gestionó con Git y GitHub .

7.1.1. Punto de partida del proyecto

El proyecto partió de una versión funcional inicial de Melodic Vault, construida por Carlos Vidal. Esa base incluyó el modelo de datos de bandas y álbumes, los mantenimientos de ambas entidades con búsqueda, el detalle de cada registro, la página de inicio con indicadores, la página “Acerca de”, la hoja de estilos y el script de la base de datos. Carlos también elaboró el diagrama BPMN que modela el proceso del sistema para los actores Visitante, Lector, Administrador y Sistema.

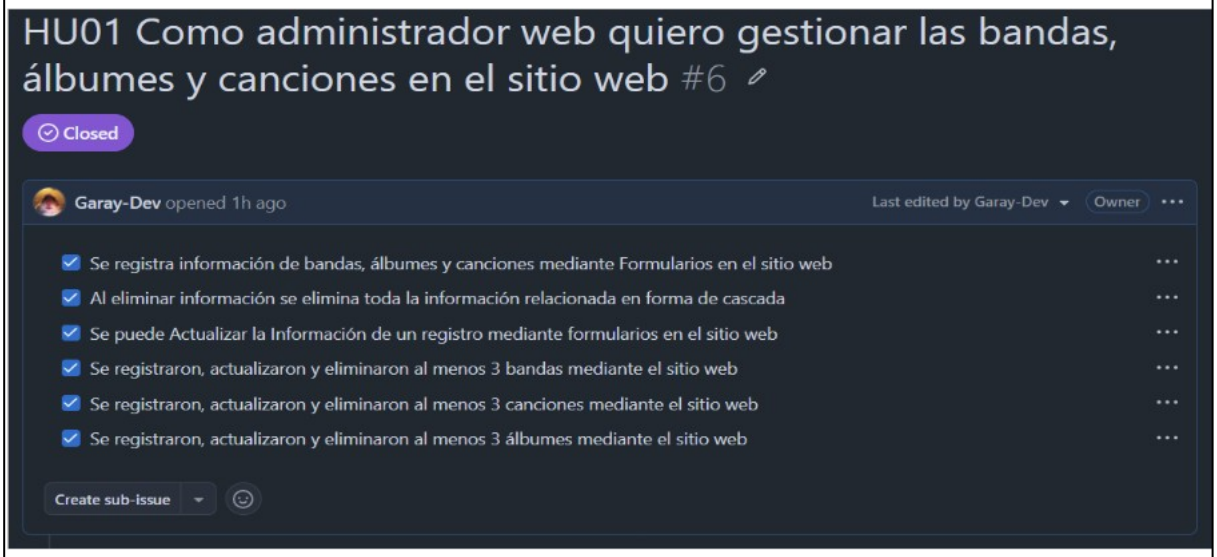
Sobre esa base, el equipo realizó una nueva etapa de análisis, mejora e implementación, en la que incorporó la gestión de canciones, el caso transaccional, el registro de usuarios, las sesiones y los roles, los reportes en PDF y el control de versiones.

7.1.2. Desarrollo de los sprints

7.1.2.1. Primer sprint

Las siguientes imágenes muestran las historias de usuario del primer sprint, desarrollado del 14 al 27 de septiembre.

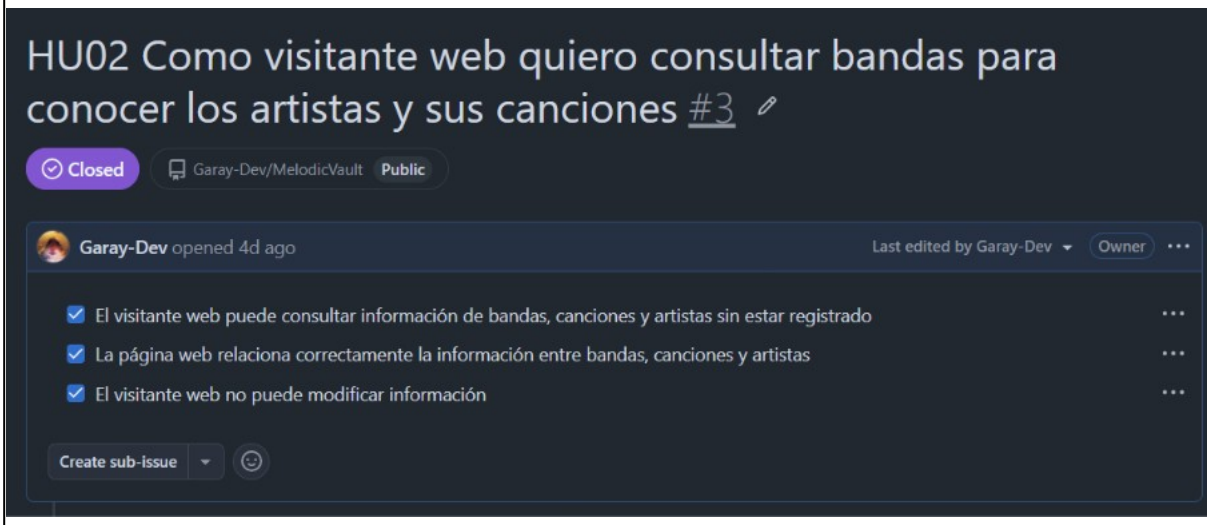
Figura 1 : HU01 Como administrador web quiero gestionar las bandas, álbumes y canciones en el sitio web (Sprint 1)



Interpretacion

El objetivo principal de este issue fue permitir que un usuario con el rol de administrador realizara la gestión integral (CRUD) del catálogo de música

Figura 2 : HU02 Como visitante web quiero consultar bandas para conocer los artistas y sus canciones (Sprint 1)



Interpretacion

El objetivo principal de este issue fue permitir que un visitante web consulte la información del catálogo de música de MeloticVault.

Para finalizar el primer sprint se elaboró el acta de Sprint Review que se muestra en la siguiente imagen:

Tabla *4*

Detalle del sprint

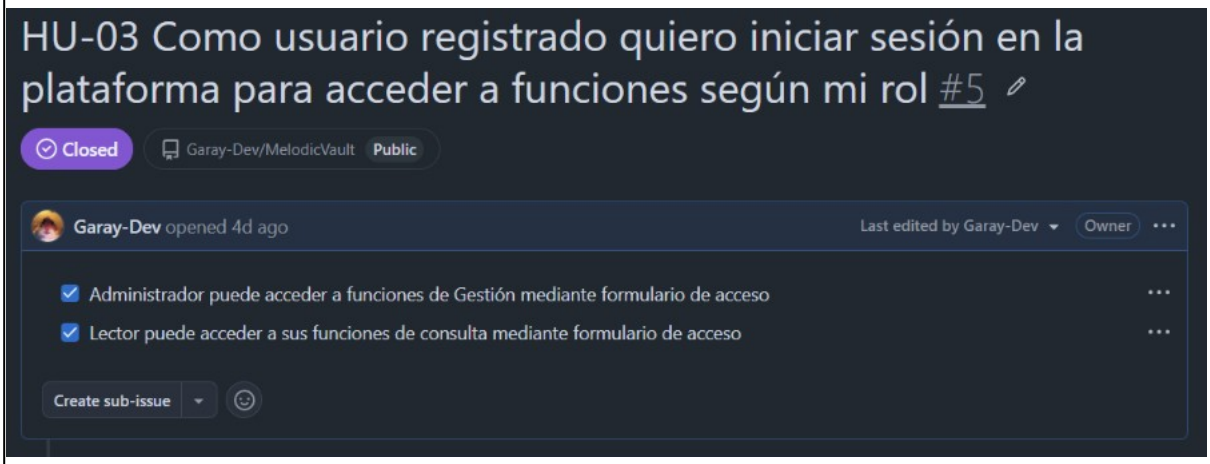
Dato	Detalle
Fecha y hora	Sin reunión
Duración del sprint	2 semanas
Equipo	Leonardo Garay (Product Owner), Fabrizzio Román (Scrum Master), Carlos Vidal y Cesar Burgos (desarrolladores)
Sprint Goal	El administrador gestiona bandas, álbumes y canciones desde el sitio web, y el visitante consulta esa información sin registrarse. Resultado: cumplido.

Desarrollo	Carlos Vidal construyó la versión base funcional (modelo de datos, mantenimientos de bandas y álbumes con búsqueda, vistas y estilos), entregada como punto de partida del proyecto. Leonardo Garay consolidó la versión en el repositorio compartido. Cesar Burgos la extendió con la gestión de canciones, el registro transaccional de un álbum con sus canciones y las estadísticas por banda.

7.1.2.2. Segundo sprint

Las imágenes muestran las historias de usuario del 2do sprint correspondiente a las fechas septiembre 29 a octubre 5

Figura 3 : HU-03 Como usuario registrado quiero iniciar sesión en la plataforma para acceder a funciones según mi rol (Sprint 2)



Interpretacion

El objetivo principal de este issue era terminar el modulo de acceso para el sistema mediante la separación de los roles Lector y Adminsitrador.

Para finalizar el segundo sprint se elaboró el acta de Sprint Review que se muestra en la siguiente imagen:

Tabla *5*

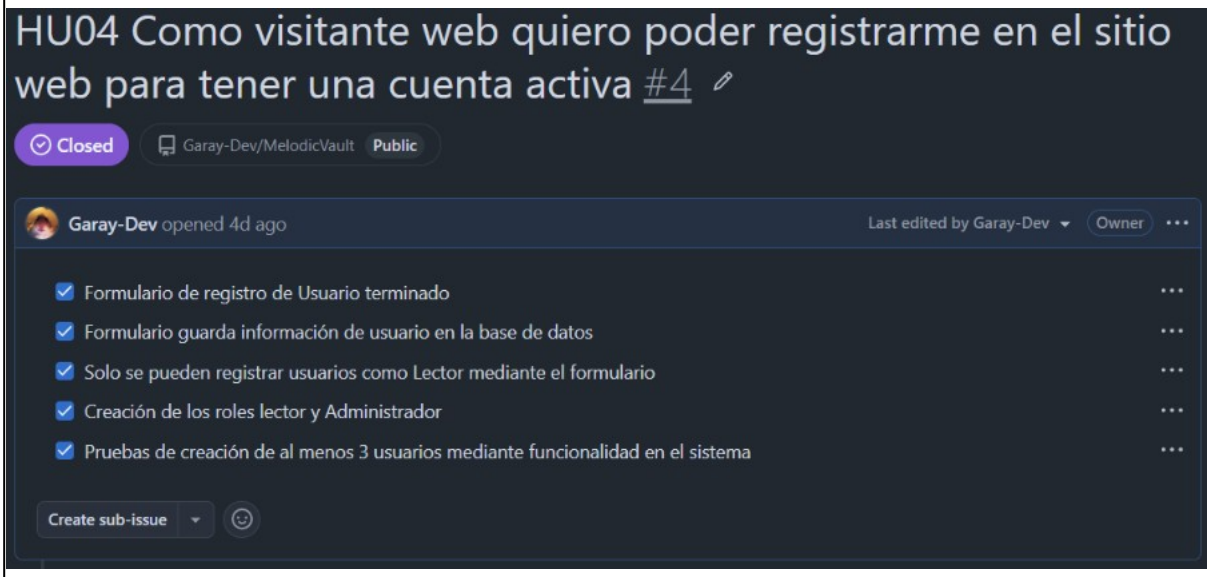
Detalle del sprint

Dato	Detalle
Fecha y hora	Sin reunión
Duración del sprint	1 semana
Equipo	Leonardo Garay (Product Owner), Fabrizzio Román (Scrum Master), Carlos Vidal y Cesar Burgos (desarrolladores)
Sprint Goal	Los usuarios con cuenta inician sesión y acceden a las funciones de su rol (Administrador y Lector). Resultado: cumplido.
Desarrollo	Cesar Burgos desarrolló el inicio de sesión y el control de acceso por roles (Administrador y Lector) con Spring Security y contraseñas cifradas, y reforzó la integridad del catálogo con la validación de pista única por álbum y la finalización del formulario de álbum. Carlos Vidal lo apoyó con consultas sobre el funcionamiento de su código base y la base de datos para evitar conflictos y hacer un trabajo solido

7.1.2.3. Tercer sprint

Las siguientes imágenes muestran las historias de usuario del tercer sprint, desarrollado del 6 al 10 de octubre

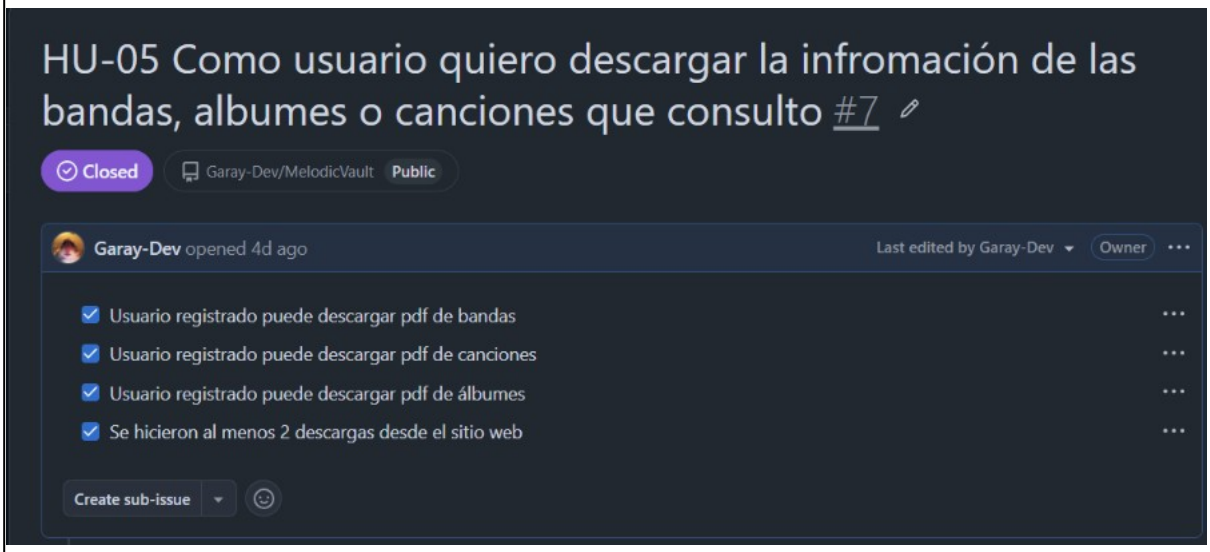
Figura 4 : HU04 Como visitante web quiero poder registrarme en el sitio web para tener una cuenta activa (Sprint 3)



Interpretacion

El objetivo principal de este issue era permitir que un visitante web pudiera registrar un usuario con el rol de lector en la página web mediante un formulario y que el administrador pueda acceder a sus funciones mediante un login de acceso.

Figura 5 : HU-05 Como usuario quiero descargar la infromación de las bandas, albumes o canciones que consulto (Sprint 3)



Interpretacion

El objetivo principal de este issue fue agregar la función de descargar la información de bandas, canciones y álbumes mediante JavaResports.

Para finalizar el tercer sprint se elaboró el acta de Sprint Review que se muestra en la siguiente imagen:

Tabla *6*

Detalle del sprint

Dato	Detalle
Fecha y hora	Sin reunión
Duración del sprint	5 días
Equipo	Leonardo Garay (Product Owner), Fabrizzio Román (Scrum Master), Carlos Vidal y Cesar Burgos (desarrolladores)

Sprint Goal	Un visitante se registra como lector y los usuarios registrados descargan en PDF los álbumes de una banda y las canciones de un álbum. Resultado: cumplido.
Desarrollo	Cesar Burgos desarrolló el módulo de registro de usuarios, que crea cuentas con rol Lector, y los dos reportes en PDF con JasperReports (canciones de un álbum y álbumes de una banda). Carlos Vidal elaboró el diagrama BPMN del proceso y actualizó la documentación. Leonardo Garay lideró la definición de los casos de uso y la documentación.

7.2. Análisis funcional del sistema

7.2.1. Requerimiento funcional

El sistema diferenció las funcionalidades disponibles según la autenticación y los roles. El Administrador dispuso de permisos para gestionar la información almacenada; el Lector, usuario con sesión iniciada, tuvo acceso a la consulta del catálogo y a la descarga de reportes; y el Visitante, sin sesión iniciada, pudo explorar el catálogo, iniciar sesión o crear una cuenta.

Visitante: persona que no había iniciado sesión. Pudo consultar el inicio, los listados, las búsquedas y los detalles de bandas, álbumes y canciones, así como iniciar sesión o registrarse.

Lector: usuario autenticado con rol LECTOR, que recibió toda cuenta creada desde el formulario de registro. Pudo realizar todas las acciones disponibles para el Visitante y, además, descargar reportes en PDF.

Administrador: usuario autenticado con rol ADMIN, asignado directamente en la base de datos. Pudo realizar todas las acciones disponibles para el Lector y, además, registrar, editar y eliminar bandas, álbumes y canciones.

En la tabla se presentaron los requerimientos funcionales de Melodic Vault. Cada requerimiento se identificó con un código, se asoció al actor que lo ejecutó (Visitante, Lector o Administrador) y se clasificó según su prioridad para el funcionamiento del sistema. El término «Todos» se refirió a los tres actores.

Tabla *7*

Requerimientos funcionales del sistema

Código	Requerimiento funcional	Actor relacionado	Prioridad
RF-01	El sistema permitió iniciar sesión con usuario y contraseña, con opción de cancelar y volver al inicio.	Visitante	Alta
RF-02	El sistema permitió cerrar sesión y volver a la página de inicio.	Lector / Administrador	Media
RF-03	El sistema restringió los mantenimientos al rol Administrador y ocultó los botones de registrar, editar y eliminar a quienes no tenían ese rol.	Administrador	Alta
RF-04	El sistema listó las bandas con sus datos principales.	Todos	Alta
RF-05	El sistema permitió buscar bandas por nombre, país o género y mostró un aviso cuando la búsqueda no devolvió resultados.	Todos	Alta
RF-06	El sistema mostró el detalle de una banda con sus datos, su biografía y su discografía (álbumes con año, tipo y rating), con opción de volver al catálogo.	Todos	Media
RF-07	El sistema permitió registrar, editar y eliminar bandas.	Administrador	Alta
RF-08	El sistema listó los álbumes y permitió buscarlos por título, tipo o banda.	Todos	Alta
RF-09	El sistema mostró el detalle de un álbum, con acceso a su banda y opción de volver.	Todos	Media

RF-10	El sistema permitió registrar, editar y eliminar álbumes asociados a una banda.	Administrador	Alta
RF-11	El sistema listó las canciones registradas.	Todos	Media
RF-12	El sistema permitió registrar, editar y eliminar canciones asociadas a un álbum.	Administrador	Alta
RF-13	El sistema permitió registrar un álbum con hasta cinco canciones por formulario en una sola operación transaccional; validó la duración y el título de cada canción y descartó las filas vacías.	Administrador	Alta
RF-14	El sistema mostró en la página de inicio las bandas y los álbumes destacados.	Todos	Media
RF-15	El sistema mostró indicadores generales: total de bandas, álbumes, países y géneros.	Todos	Alta
RF-16	El sistema mostró estadísticas por banda: cantidad de álbumes y rating promedio.	Todos	Media
RF-17	El sistema mostró la página "Acerca de" con la información del proyecto.	Todos	Baja
RF-18	El sistema mostró un menú de navegación (Inicio, Bandas, Álbumes, Canciones y Acerca de) que varió según la sesión: «Iniciar sesión» cuando no había una sesión activa y «Salir (usuario)» cuando sí la había.	Todos	Media
RF-19	El sistema permitió registrarse con usuario y contraseña (con confirmación), validó los datos, informó los errores, creó la cuenta	Visitante	Alta

	siempre con el rol Lector y luego redirigió al usuario al inicio de sesión.		
RF-20	El sistema permitió descargar en PDF el reporte de canciones de un álbum (pista, título y duración) desde el detalle del álbum, solo con la sesión iniciada.	Lector / Administrador	Media
RF-21	El sistema permitió descargar en PDF el reporte de álbumes de una banda (año, título, tipo y rating) desde el detalle de la banda, solo con la sesión iniciada.	Lector / Administrador	Media

7.2.1.1. Interpretación

En la tabla anterior se observó que el sistema cubrió 21 requerimientos funcionales. De ellos, 13 estuvieron disponibles sin sesión iniciada: 11 de consulta, abiertos a cualquier persona, el inicio de sesión y el registro de usuarios. Tres requirieron una sesión activa (el cierre de sesión y la descarga de los dos reportes en PDF) y cinco estuvieron reservados al Administrador (control de acceso; registro, edición y eliminación de bandas, álbumes y canciones; y registro transaccional de un álbum con sus canciones).

Esta distribución evidenció que la consulta del catálogo y la creación de cuentas fueron abiertas, que los reportes quedaron reservados a usuarios autenticados y que las operaciones que modificaron la información estuvieron restringidas por rol. Respecto a la prioridad, 11 requerimientos fueron de prioridad alta, nueve de prioridad media y uno de prioridad baja. Estos se concentraron en la autenticación, el registro, los mantenimientos y las consultas principales.

El sistema incluyó el registro de usuarios: toda cuenta creada desde el formulario recibió el rol Lector, que el servidor estableció sin permitir que el usuario lo eligiera. Las cuentas con rol Administrador se asignaron directamente en la base de datos. Como mejora futura, se recomendó incorporar la recuperación y el cambio de contraseña.

7.2.2. Requerimiento no funcional

En la tabla se detallaron los requerimientos no funcionales de Melodic Vault, es decir, las características que condicionaron el funcionamiento del sistema más allá de sus funcionalidades.

Cada requerimiento se identificó con un código y se clasificó según su categoría: seguridad, integridad, usabilidad, mantenibilidad, portabilidad, rendimiento y fiabilidad.

Tabla *8*

Requerimientos no funcionales del sistema

Código	Requerimiento funcional	Actor relacionado	Prioridad
RNF-01	Seguridad	Las contraseñas se almacenaron con hash BCrypt, nunca en texto plano.	Alta
RNF-02	Seguridad	El acceso a las rutas de mantenimiento se controló por roles con Spring Security.	Alta
RNF-03	Integridad	Las operaciones sobre varias tablas se ejecutaron en una transacción, con rollback si alguna de ellas fallaba.	Alta
RNF-04	Integridad	Los formularios validaron campos obligatorios, longitudes máximas (incluidas la descripción y las URL de imagen y portada), la duración obligatoria y no negativa de las canciones, y el rango del rating (0 a 10), con mensajes claros bajo cada campo.	Alta
RNF-05	Integridad	La base de datos impidió registrar dos	Alta

		canciones con el mismo número de pista en un mismo álbum, mediante una restricción única sobre álbum y número de pista; ante un duplicado, el formulario mostró un mensaje de error en lugar de una página de error.	
RNF-06	Usabilidad	La interfaz web usó plantillas Thymeleaf, Bootstrap 5, Bootstrap Icons y la hoja de estilos estilos.css, con navegación y mensajes de confirmación. Cuando un registro no existía, el sistema redirigió al listado con un mensaje de error. Cuando no había datos, mostró avisos distintos para una búsqueda sin resultados y para un catálogo vacío, y un álbum sin rating mostró «—» en lugar de un valor vacío.	Media
RNF-07	Mantenibilidad	La aplicación siguió una arquitectura MVC por capas (controller, service, repository y modelo) en paquetes separados.	Media
RNF-08	Portabilidad	La aplicación se construyó y empaquetó con Maven y pudo ejecutarse en cualquier equipo con JDK, MySQL y conexión a internet (los estilos se cargaron desde un CDN).	Baja
RNF-09	Seguridad	El rol de una cuenta nueva lo fijó el servidor (Lector); el formulario de registro no pudo elegirlo porque no recibió ese dato.	Alta

RNF-10	Integridad	El nombre de usuario fue único y la base de datos lo garantizó con una restricción única, incluso ante registros simultáneos; el formulario mostró el mensaje “Ese usuario ya existe”.	Alta
RNF-11	Rendimiento	Las plantillas de los reportes PDF se compilaron una sola vez al iniciar la aplicación, no en cada descarga.	Baja
RNF-12	Fiabilidad	Los reportes cargaron los datos que necesitaron en una sola consulta, sin depender de la configuración open-in-view de Spring Boot.	Media
RNF-13	Fiabilidad	Los errores se mostraron en una página comprensible (403, 404 u otro código), accesible sin sesión, en lugar de un error técnico. Un reporte solicitado con un identificador inexistente respondió con un 404.	Media
RNF-14	Integridad	Eliminar una banda o un álbum eliminó también sus registros dependientes, y editar un álbum conservó sus canciones.	Alta

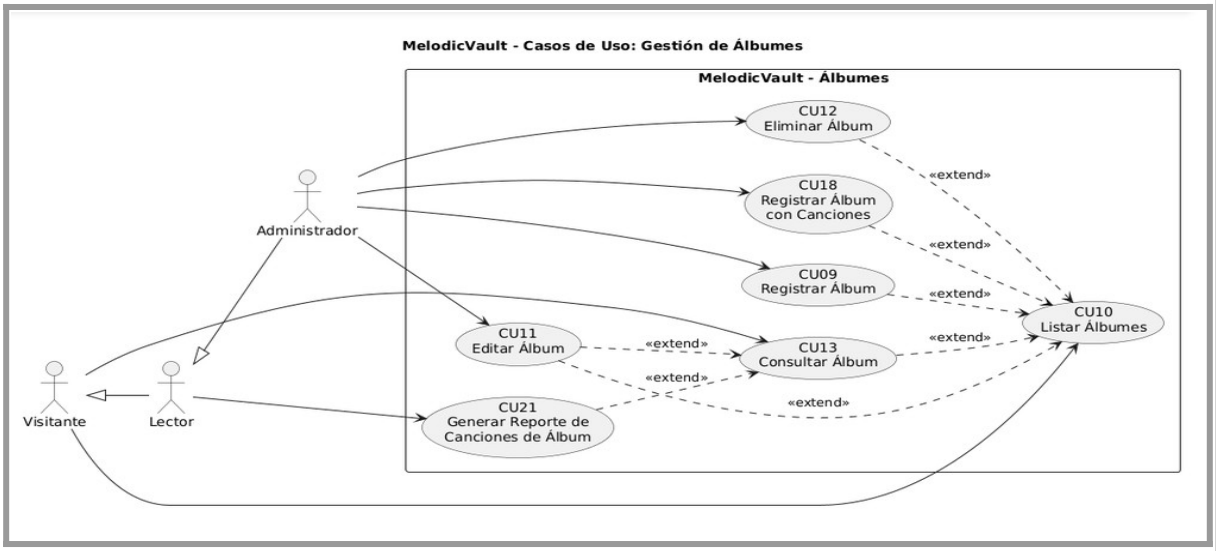
INTERPRETACION

Los 14 requerimientos no funcionales se concentran en la seguridad (contraseñas con hash, acceso por roles y rol fijado por el servidor) y en la integridad (transacciones, validaciones, restricciones únicas y eliminación en cascada). El resto cubre usabilidad, mantenibilidad, portabilidad, rendimiento y fiabilidad.

7.2.3. Casos de uso y reglas de negocio

7.2.3.1. Diagrama general de casos de uso

Diagrama 2 : CU – Gestión de álbumes

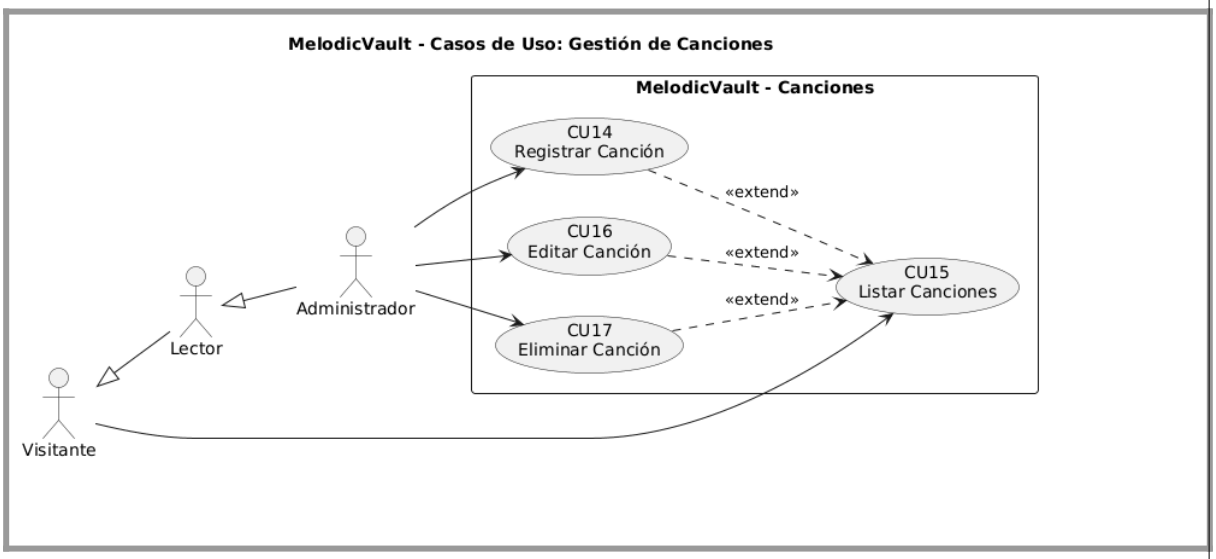


INTERPRETACION

Sigue el mismo patrón: el Visitante lista y consulta, el Lector además genera el reporte de canciones del álbum desde el detalle, y el Administrador registra, edita y elimina.

Se destaca Registrar Álbum con Canciones (CU18), un caso aparte del registro simple porque guarda el álbum y hasta 5 canciones en una sola transacción: si algo falla, no se guarda nada.

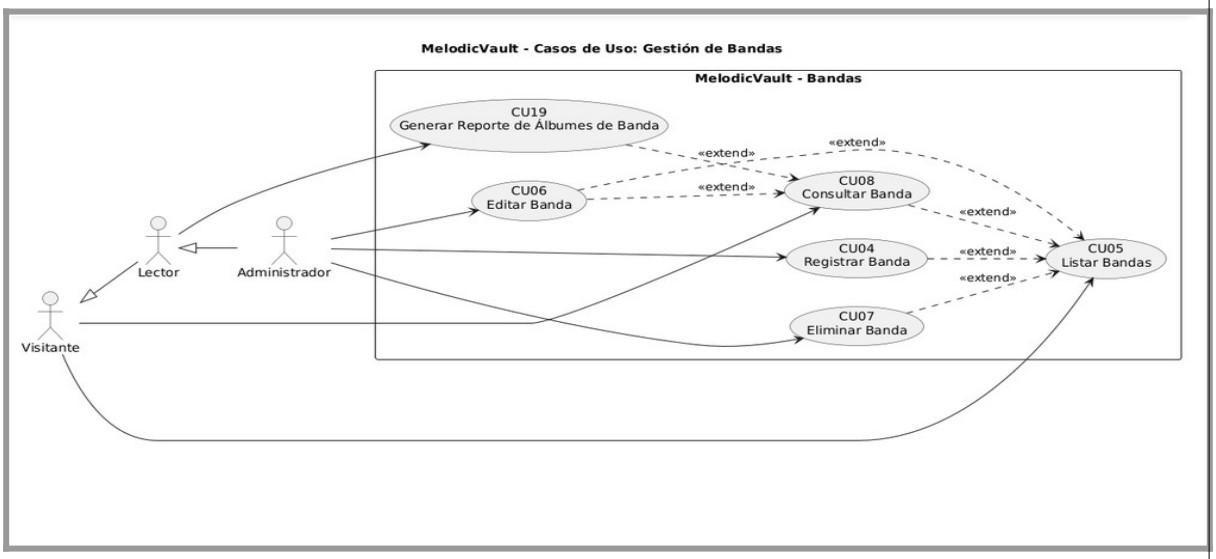
Diagrama 3 : CU – Gestión de canciones



INTERPRETACION

Es el diagrama más simple. Listar canciones es público, y registrar, editar y eliminar son solo del Administrador. No hay consulta de detalle ni reporte propio, lo que coincide con el sistema real, donde una canción solo se ve en el listado y en el detalle de su álbum.

Diagrama 4 : CU – Gestión de Bandas

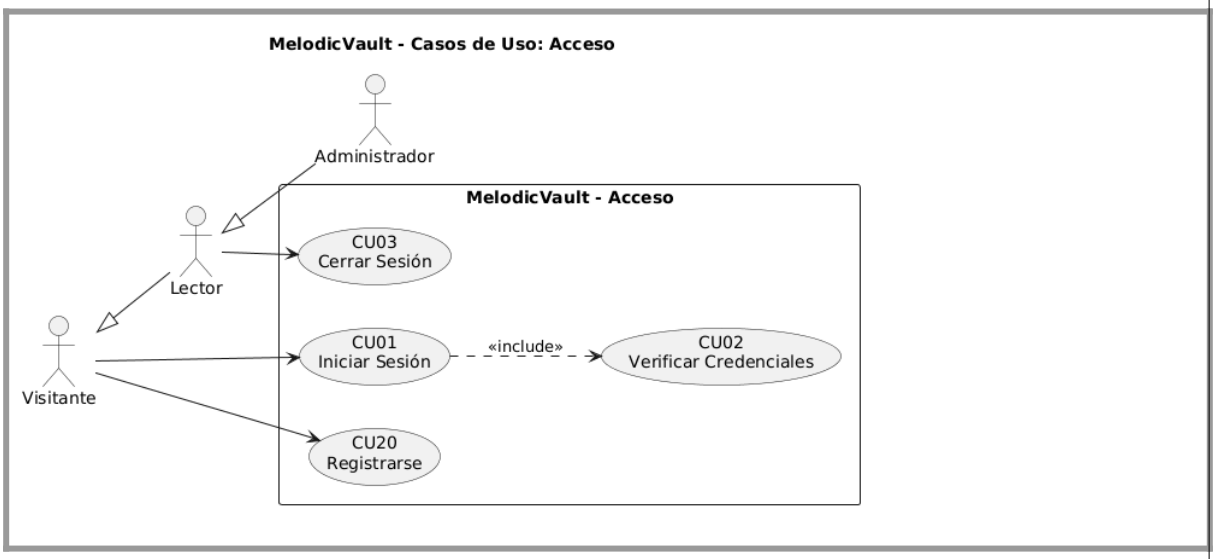


INTERPRETACION

Listar y consultar bandas están abiertos a todos, incluido el Visitante. Registrar, editar y eliminar son exclusivos del Administrador, y sus extensiones hacia el listado reflejan que esas acciones se ejecutan desde la pantalla de bandas.

Generar el reporte de álbumes de una banda es exclusivo del Lector (y por herencia del Administrador) y parte del detalle de la banda, por eso extiende a Consultar Banda. El Visitante lo ve pero no puede descargarlo: necesita sesión.

Diagrama 5 : CU – Acceso



INTERPRETACION

El diagrama muestra la puerta de entrada del sistema. El Visitante puede iniciar sesión o registrarse, y la cuenta creada siempre nace como Lector. Iniciar sesión incluye siempre la verificación de credenciales, que compara la contraseña contra el hash guardado.

Cerrar sesión lo hace cualquier usuario autenticado. La herencia Visitante → Lector → Administrador indica que cada rol conserva todo lo del anterior y suma permisos.

7.2.3.2. Reglas de negocio

Tabla *9*

Reglas de negocio

Dato	Detalle
RN-01	Las operaciones de registro, edición y eliminación de información requieren permisos de Administrador.
RN-02	El nombre de usuario debe ser único.
RN-03	Una banda debe tener nombre, país, género, año de formación y estado.

RN-04	El nombre de una banda no puede superar los 80 caracteres.
RN-05	El país y género de una banda no pueden superar los 60 caracteres.
RN-06	La descripción de una banda no puede superar los 500 caracteres.
RN-07	Una banda puede estar asociada a múltiples álbumes.
RN-08	Un álbum debe tener titulo, año y tipo.
RN-09	El titulo de un álbum no puede superar los 100 caracteres.
RN-10	La valoración de un álbum debe estar entre 0 y 10.
RN-11	Todo álbum debe estar asociado obligatoriamente a una banda.
RN-12	Una canción debe tener titulo y numero de pista.
RN-13	Toda canción debe estar asociada obligatoriamente a un álbum.
RN-14	El numero de pista debe ser igual o mayor que 1.
RN-15	La duración de una canción es obligatoria y no puede ser negativa.
RN-16	El registro completo de un álbum requiere al menos una canción válida.
RN-17	Las filas de canciones sin título se ignoran durante el registro completo del álbum.
RN-18	Si una canción con título no tiene número de pista, el sistema le asigna el número de orden de su fila entre las filas con título.
RN-19	Durante el registro completo, un numero de pista menor o igual a 0 invalida la operación.
RN-20	El nombre de usuario tiene entre 4 y 50 caracteres y solo puede contener letras, números, punto, guion y guion bajo.
RN-21	La contraseña tiene entre 6 y 72 caracteres y se almacena cifrada.
RN-22	La confirmación de la contraseña debe coincidir con la contraseña.
RN-23	Toda cuenta creada mediante el registro recibe el rol Lector.
RN-24	La descarga de reportes requiere una sesión iniciada (Lector o Administrador).
RN-25	No pueden existir dos canciones con el mismo número de pista en un mismo álbum.
RN-26	El registro completo de un álbum admite hasta 5 canciones por formulario.
RN-27	Al eliminar una banda se eliminan sus álbumes y canciones; al eliminar un álbum, se eliminan sus canciones.
RN-28	En el registro completo de un álbum, toda canción con título debe tener una duración no negativa y su título no puede superar los 100 caracteres.

RN-29	La URL de imagen de una banda y la URL de portada de un álbum no pueden superar los 255 caracteres.
RN-30	La reseña (descripción) de un álbum no puede superar los 500 caracteres.

INTERPRETACION

Las reglas definen cómo se comporta MelodicVault en tres frentes.

En acceso, solo el Administrador modifica el catálogo, toda cuenta registrada nace como Lector porque el servidor fija el rol y el formulario no puede elegirlo, y los reportes PDF exigen sesión iniciada.

En el catálogo, la estructura es una jerarquía estricta banda → álbum → canción: todo álbum pertenece a una banda y toda canción a un álbum, así que no hay registros huérfanos. Las reglas de obligatoriedad, longitud y rango replican las validaciones de las entidades y el tamaño de las columnas en MySQL. La pista única por álbum la garantiza una restricción en la base de datos, y la eliminación en cascada evita datos colgados.

En el registro completo de álbum, se guarda el álbum con hasta 5 canciones en una sola transacción: si una falla, no se guarda nada. Como las filas pueden quedar vacías, esas canciones no pasan por la validación automática, por eso el servicio ignora filas sin título, asigna pista según el orden de fila y valida duración y largo del título.

En conjunto, las reglas se aplican en interfaz, servidor y base de datos, manteniendo la consistencia aunque una capa falle.

7.2.3.3. Especificaciones de caso de uso

7.2.3.3.1. Acceso

Tabla *10*

CU-01 - Iniciar sesión

Campo	**Contenido**
Código	CU-01
Nombre	Iniciar sesión
Actor	Visitante

Descripción	Permite al usuario ingresar a MelodicVault con su usuario y contraseña. El sistema verifica las credenciales (CU02) y habilita las funcionalidades según el rol de la cuenta (Lector o Administrador).
Precondiciones	• El usuario debe estar registrado en el sistema. • El usuario debe contar con un nombre de usuario y contraseña válidos. • El sistema debe encontrarse disponible.
Flujo básico	1. El visitante selecciona “Iniciar sesión” en el menú. 2. sistema muestra el formulario de acceso con los campos usuario y contraseña. 3. El visitante completa ambos campos y presiona «Entrar». 4. El sistema verifica las credenciales (incluye CU-02). 5. El sistema identifica el rol del usuario (Administrador o Lector). 6. El sistema redirige a la página principal, con el menú y los botones correspondientes al rol.
Flujos alternativos	FA-1 Credenciales incorrectas: el sistema rechaza el acceso, vuelve al formulario y muestra «Usuario o contraseña incorrectos.». FA-2 Usuario no registrado: el sistema muestra el mismo mensaje que en FA-1. FA-3 Campos vacíos: el navegador impide el envío y solicita completar el campo, porque ambos son obligatorios en el formulario. FA-4 Cancelar: el visitante cancela y vuelve a la página de inicio. FA-5 Ruta protegida sin sesión: si el visitante abre una ruta que exige sesión, el sistema lo redirige al formulario de acceso. FA-6 Sin cuenta: el visitante selecciona «Regístrate» y continúa en CU20.
Postcondiciones	El usuario queda autenticado y accede a las funcionalidades de su rol.
Reglas de negocio	RN-21.
Relaciones	Incluye CU02 – Verificar credenciales.

Tabla *11*

CU-02 - Verificar credenciales

Campo	**Contenido**
Código	CU-02
Nombre	Verificar credenciales
Actor	Sistema
Descripción	Verifica que el usuario y la contraseña enviados desde el formulario de acceso correspondan a una cuenta registrada y obtiene su rol.
Precondiciones	1. El visitante envió su usuario y contraseña desde el formulario de acceso (CU01).

	2. El sistema puede consultar las cuentas registradas.
Flujo básico	1. El sistema recibe el usuario y la contraseña ingresados. 2. El sistema busca la cuenta por nombre de usuario. 3. El sistema compara la contraseña ingresada con la contraseña almacenada cifrada (BCrypt). 4. El sistema obtiene el rol de la cuenta (ADMIN o LECTOR). 5. El sistema devuelve a CU01 el resultado exitoso y el rol.
Flujos alternativos	FA-1 Usuario inexistente: el sistema devuelve resultado fallido a CU-01, que rechaza el acceso (CU-01 FA-2). FA-2 Contraseña incorrecta: el sistema devuelve resultado fallido a CU-01, que rechaza el acceso (CU-01 FA-1).
Postcondiciones	CU-01 conoce si las credenciales son válidas y, de serlo, el rol del usuario.
Reglas de negocio	RN-21.
Relaciones	Incluido por CU01 – Iniciar sesión.

Tabla *12*

CU-03 – Cerrar sesión

Campo	**Contenido**
Código	CU-03
Nombre	Cerrar sesion
Actor	Lector / Administrador
Descripción	Permite al usuario finalizar su sesión y volver a la condición de Visitante.
Precondiciones	El usuario tiene una sesion iniciada
Flujo básico	1. El usuario selecciona “Salir (usuario)” en el menú. 2. El sistema cierra la sesión. 3. El sistema redirige a la página de inicio. 4. El menú vuelve a mostrar “Iniciar sesión” y se ocultan los botones de mantenimiento y “Descargar PDF”.
Flujos alternativos	FA-1 Acceso posterior: si después de cerrar sesión el usuario abre una ruta que exige sesión, el sistema lo redirige al formulario de acceso (CU01).
Postcondiciones	La sesión queda cerrada y el usuario navega como Visitante.
Reglas de negocio	ninguna específica.
Relaciones	No presenta relaciones adicionales.

Tabla *13*

CU-20 – Registrarse

Campo	Contenido
Código	CU-20
Nombre	Registrarse
Actor	Visitante
Descripción	Permite crear una cuenta de acceso. La cuenta se crea siempre con rol Lector; el visitante no puede elegir el rol.
Precondiciones	1. El visitante no tiene una sesión iniciada. 2. El sistema está disponible.
Flujo básico	1. En el formulario de acceso, el visitante selecciona “Regístrate“. 2. El sistema muestra el formulario de registro con los campos usuario, contraseña y confirmar contraseña. 3. El visitante completa los tres campos y presiona “Crear cuenta”. 4. El sistema valida el formato del usuario y de la contraseña (RN-20, RN-21). 5. El sistema verifica que las contraseñas coincidan (RN-22) y que el usuario no exista (RN-02). 6. El sistema cifra la contraseña y crea la cuenta con rol Lector (RN-23). 7. El sistema redirige al formulario de acceso con el mensaje «Cuenta creada. Ya puedes iniciar sesión.». No inicia sesión automáticamente.
Flujos alternativos	FA-1 Datos inválidos: el sistema vuelve al formulario con el mensaje bajo cada campo (por ejemplo «El usuario es obligatorio», «Entre 4 y 50 caracteres», «Solo letras, números, punto, guion y guion bajo», «Entre 6 y 72 caracteres»), conserva el usuario escrito y vacía las contraseñas. FA-2 Contraseñas distintas: el sistema muestra «Las contraseñas no coinciden» bajo la confirmación. Solo se evalúa cuando los demás campos son válidos. FA-3 Usuario existente: el sistema muestra «Ese usuario ya existe». Lo mismo ocurre si otra persona registra el mismo usuario al mismo tiempo, porque la base de datos lo impide. FA-4 Cancelar: el visitante presiona «Cancelar» y vuelve al formulario de acceso.
Postcondiciones	La cuenta con rol Lector queda creada. El usuario debe iniciar sesión para ingresar.
Reglas de negocio	RN-02, RN-20, RN-21, RN-22 y RN-23.

Relaciones	No presenta relaciones adicionales.

7.2.3.3.2. Bandas

Tabla *14*

CU04 – Registrar banda

Campo	Contenido
Código	CU04
Nombre	Registrar banda
Actor	Administrador
Descripción	Permite registrar una nueva banda en el catálogo de MelodicVault con sus datos principales.
Precondiciones	1. El administrador ha iniciado sesión. 2. El sistema está disponible.
Flujo básico	1. En el módulo Bandas, el administrador selecciona Registrar banda. 2. El sistema muestra el formulario vacío. 3. El administrador ingresa nombre, país, género (de una lista), año de formación, estado (Activa, Separada o En pausa), URL de imagen y descripción. 4. El administrador presiona Guardar banda. 5. El sistema valida los datos (RN-03 a RN-06, RN-29). 6. El sistema registra la banda. 7. El sistema redirige al listado de bandas con el mensaje Banda registrada correctamente.
Flujos alternativos	FA-1 Datos obligatorios incompletos: el sistema vuelve al formulario con el mensaje bajo cada campo («El nombre es obligatorio», «El país es obligatorio», «El género es obligatorio», «El año de formación es obligatorio») y conserva lo escrito. FA-2 Longitudes excedidas: si el nombre, país, género, descripción o la URL superan el límite, el sistema rechaza el registro y muestra el mensaje bajo el campo. FA-3 Cancelar: el administrador presiona «Cancelar» o «Volver» y regresa al listado sin guardar. FA-4 Sin permisos: el usuario no puede ejecutar la operación
Postcondiciones	La banda queda registrada y disponible en el catálogo.
Reglas de negocio	RN-01, RN-03, RN-04, RN-05, RN-06 y RN-29.

Relaciones	Extiende CU05 – Listar bandas.

Tabla 15

CU05 – Listar bandas

Campo	Contenido
Código	CU-05
Nombre	Listar bandas
Actor	Visitante / Lector / Administrador
Descripción	Permite ver el catálogo de bandas registradas y buscarlas por nombre, país o género.
Precondiciones	1. El sistema está disponible.
Flujo básico	1. El usuario selecciona «Bandas» en el menú. 2. El sistema muestra el total de bandas registradas y una tarjeta por banda con imagen, género, nombre, país, año de formación y descripción. 3. El usuario puede escribir un criterio en el buscador y presionar Buscar. 4. El sistema muestra las bandas cuyo nombre, país o género contienen el texto, sin distinguir mayúsculas. 5. El usuario puede seleccionar «Ver detalle» para consultar una banda (CU08). 6. Si el usuario es Administrador, el sistema muestra además «Registrar banda» y, en cada tarjeta, «Editar» y «Eliminar».
Flujos alternativos	FA-1 Catálogo vacío: el sistema muestra «Aún no hay bandas registradas.». El administrador ve además el botón «Registrar primera banda». FA-2 Búsqueda sin resultados: el sistema muestra «No se encontraron bandas con ese criterio.». FA-3 Buscador vacío: el sistema lista todas las bandas.
Postcondiciones	El usuario visualiza las bandas disponibles o los resultados de su búsqueda.
Reglas de negocio	RN-07: una banda puede estar asociada a múltiples álbumes.
Relaciones	Es extendido por CU04, CU06, CU07 y CU08.

Tabla 16

CU06 – Editar banda

Campo	Contenido
Código	CU-06
Nombre	Editar banda
Actor	Administrador
Descripción	Permite modificar los datos de una banda ya registrada.
Precondiciones	1. El administrador ha iniciado sesión. 2. La banda que se desea modificar existe.
Flujo básico	1. El administrador selecciona «Editar» en la tarjeta de la banda (listado) o «Editar banda» en su detalle. 2. El sistema muestra el formulario con los datos actuales de la banda. 3. El administrador modifica los campos necesarios. 4. El administrador presiona «Guardar cambios». 5. El sistema valida los datos (RN-03 a RN-06, RN-29). 6. El sistema actualiza la banda. 7. El sistema redirige al listado de bandas con el mensaje «Banda actualizada correctamente.».
Flujos alternativos	FA-1 Datos inválidos: el sistema vuelve al formulario con los mensajes de validación bajo cada campo y conserva lo escrito. FA-2 Banda inexistente: el sistema redirige al listado de bandas con el mensaje «La banda solicitada no existe.». FA-3 Cancelar: el administrador presiona «Cancelar» o «Volver» y regresa al listado sin guardar. FA-4 Sin permisos: el usuario no puede ejecutar la operación (ver criterios generales).
Postcondiciones	Los datos de la banda quedan actualizados.
Reglas de negocio	RN-01, RN-03, RN-04, RN-05, RN-06 y RN-29.
Relaciones	Extiende CU05 – Listar bandas y CU08 – Consultar banda.

Tabla 17

CU-07 - Eliminar banda

Campo	Contenido
Código	CU-07

Nombre	Eliminar banda
Actor	Administrador
Descripción	Permite eliminar una banda del catálogo junto con sus álbumes y las canciones de esos álbumes.
Precondiciones	1. El administrador ha iniciado sesión. 2. Debe existir la banda.
Flujo básico	1. En el listado de bandas, el administrador selecciona «Eliminar» en la tarjeta de la banda. 2. El navegador solicita confirmación: «¿Seguro que deseas eliminar esta banda? También se eliminarán sus álbumes asociados.». 3. El administrador acepta. 4. El sistema verifica que la banda exista. 5. El sistema elimina la banda, sus álbumes y las canciones de esos álbumes (RN-27). 6. El sistema redirige al listado de bandas con el mensaje «Banda eliminada correctamente.».
Flujos alternativos	FA-1 Cancelar eliminación: si el administrador rechaza la confirmación, no se envía la petición y la banda se conserva. FA-2 Banda inexistente: si la banda ya no existe, el sistema redirige al listado con el mensaje «La banda no existe.». FA-3 Sin permisos: el usuario no puede ejecutar la operación
Postcondiciones	La banda, sus álbumes y sus canciones dejan de estar registrados.
Reglas de negocio	RN-01 y RN-27.
Relaciones	Extiende CU05 – Listar bandas.

Tabla 18

CU-08 – Consultar Banda

Campo	**Contenido**
Código	CU-08
Nombre	Consultar banda
Actor	Visitante / Lector / Administrador
Descripción	Permite ver la información detallada de una banda: sus datos, su biografía y su discografía.
Precondiciones	1. El sistema está disponible. 2. La banda que se desea consultar existe.
Flujo básico	1. El usuario selecciona «Ver detalle» en una banda del listado.

	2. El sistema muestra la ficha de la banda (imagen, nombre, país, género, año de formación y estado) y su biografía. 3. El sistema muestra la «Discografía destacada»: una tarjeta informativa por álbum con título, año, tipo y rating («—» si no tiene). 4. El usuario puede regresar con «Volver al catálogo». 5. Si el usuario tiene sesión, el sistema muestra «Descargar PDF» (CU19); si es Administrador, muestra además «Editar banda» (CU06).
Flujos alternativos	FA-1 Banda inexistente: el sistema redirige al listado de bandas con el mensaje «La banda solicitada no existe.». FA-2 Sin álbumes: el sistema muestra «No existen álbumes registrados para esta banda.». El administrador ve además el botón «Registrar álbum».
Postcondiciones	El usuario visualiza los datos de la banda y su discografía registrada.
Reglas de negocio	RN-07: una banda puede estar asociada a múltiples álbumes.
Relaciones	Extiende CU05 – Listar bandas. Es extendido por CU06 y CU19.

Tabla 19

CU-019 - Generar reporte de álbumes de una banda

Campo	**Contenido**
Código	CU-19
Nombre	Generar reporte de álbumes de una banda
Actor	Lector / Administrador
Descripción	Permite descargar en PDF el listado de álbumes de una banda desde su detalle.
Precondiciones	1-. El usuario ha iniciado sesión (Lector o Administrador). 2-.La banda existe y el usuario está en su detalle (CU08).
Flujo básico	1-. El usuario selecciona «Descargar PDF» en el detalle de la banda. 2. El sistema consulta la banda y sus álbumes 3- El sistema genera el reporte con el nombre, país, género y año de formación de la banda, y una tabla con año, título, tipo y rating de cada álbum. El rating se muestra con un decimal («-» si no tiene) y el pie dice «Melodic Vault - Página N». 4-. El sistema envía el PDF al navegador, que lo abre en una pestaña nueva.
Flujos alternativos	FA-1 Banda sin álbumes: el sistema genera el PDF con los datos de

	la banda y la cabecera de la tabla, sin filas y sin error. FA-2 Banda inexistente: si el usuario accede por URL a una banda que no existe, el sistema muestra la página de error 404. FA-3 Sin sesión: el botón «Descargar PDF» no se muestra; si el visitante abre la ruta directamente, el sistema lo redirige al formulario de acceso.
Postcondiciones	El usuario obtiene el PDF con los álbumes de la banda.
Reglas de negocio	RN-24: la descarga de reportes requiere una sesión iniciada.
Relaciones	Extiende CU08 – Consultar banda.

Tabla 20

7.2.3.3.3. Album

CU-09 - Registrar álbum

Campo	Contenido
Código	CU-09
Nombre	Registrar álbum
Actor	Administrador
Descripción	Permite registrar un álbum en el catálogo y asociarlo obligatoriamente a una banda existente.
Precondiciones	1. El administrador ha iniciado sesión. 2. Existe al menos una banda registrada.
Flujo básico	1. En el módulo «Álbumes», el administrador selecciona «Registrar álbum» 2. El sistema muestra el formulario vacío. 3. El administrador ingresa el título, selecciona la banda de la lista, y completa año, tipo (Studio Album, Live Album, EP, Compilation o Single), rating (0 a 10), formato (CD, Vinilo, Cassette o Digital), URL de portada y reseña. 4. El administrador presiona «Guardar álbum». 5. El sistema valida los datos (RN-08 a RN-11, RN-29, RN-30) y que se haya seleccionado una banda. 6. El sistema registra el álbum asociado a la banda. 7. El sistema redirige al listado de álbumes con el mensaje «Álbum registrado correctamente.».
Flujos alternativos	FA-1 Datos incompletos: el sistema vuelve al formulario con los mensajes bajo cada campo («El título es obligatorio», «El año es obligatorio») y conserva lo escrito. FA-2 Rating fuera de rango: el sistema muestra «El rating mínimo es 0» o «El rating máximo es 10». El campo además limita el valor entre 0 y 10 en el navegador.

	FA-3 Banda no seleccionada: el sistema muestra «Debe seleccionar una banda» y no guarda el álbum. FA-4 Longitudes excedidas: si el título, la reseña o la URL de portada superan el límite, el sistema rechaza el registro y muestra el mensaje bajo el campo. FA-5 Cancelar: el administrador presiona «Cancelar» o «Volver» y regresa al listado sin guardar. FA-6 Sin permisos: el usuario no puede ejecutar la operación
Postcondiciones	El álbum queda registrado y asociado a la banda seleccionada.
Reglas de negocio	RN-01, RN-08, RN-09, RN-10, RN-11, RN-29 y RN-30.
Relaciones	Extiende CU10 – Listar álbumes.

Tabla 21

CU-10 - Listar álbumes

Campo	**Contenido**
Código	CU-10
Nombre	Listar álbumes
Actor	Visitante / Lector / Administrador
Descripción	Permite ver el catálogo de álbumes registrados y buscarlos por título, tipo o banda.
Precondiciones	1. El sistema está disponible.
Flujo básico	1. El usuario selecciona «Álbumes» en el menú. 2. El sistema muestra el total de álbumes registrados y una tarjeta por álbum con portada (o el título si no tiene), tipo, título, banda y año, descripción, rating («—» si no tiene), formato y género. 3. El usuario puede escribir un criterio en el buscador y presionar «Buscar». 4. El sistema muestra los álbumes cuyo título, tipo o nombre de banda contienen el texto, sin distinguir mayúsculas. 5. El usuario puede seleccionar «Ver detalle» para consultar un álbum (CU13). 6. Si el usuario es Administrador, el sistema muestra además «Registrar álbum», «Álbum con canciones» (CU18) y, en cada tarjeta, «Editar» y «Eliminar».
Flujos alternativos	FA-1 Sin álbumes: el sistema muestra «Aún no hay álbumes registrados.». El administrador ve además el botón «Registrar primer álbum». FA-2. Búsqueda sin resultados: el sistema muestra «No se encontraron álbumes con ese criterio.». FA-3. Buscador vacío: el sistema lista todos los álbumes.

Postcondiciones	El usuario visualiza los álbumes disponibles o los resultados de su búsqueda.
Reglas de negocio	RN-11: todo álbum está asociado a una banda.
Relaciones	Es extendido por CU09, CU11, CU12, CU13 y CU18.

Tabla 22

CU-11 - Editar álbum

Campo	**Contenido**
Código	CU-11
Nombre	Editar álbum
Actor	Administrador
Descripción	Permite modificar los datos de un álbum ya registrado, incluida su banda. Las canciones del álbum se conservan.
Precondiciones	1. El administrador ha iniciado sesión. 2. El álbum que se desea modificar existe.
Flujo básico	1. El administrador selecciona «Editar» en la tarjeta del álbum (listado) o «Editar álbum» en su detalle.. 2. El sistema muestra el formulario con los datos actuales, con la banda del álbum ya seleccionada. 3. El administrador modifica los campos necesarios. 4. El administrador presiona «Guardar cambios». 5. El sistema valida los datos (RN-08 a RN-11, RN-29, RN-30) y que haya una banda seleccionada. 6. El sistema actualiza el álbum. 7. El sistema redirige al listado de álbumes con el mensaje «Álbum actualizado correctamente.».
Flujos alternativos	FA-1 Datos inválidos: el sistema vuelve al formulario con los mensajes bajo cada campo y conserva lo escrito. FA-2 Rating fuera de rango: el sistema muestra «El rating mínimo es 0» o «El rating máximo es 10». FA-3 Banda no seleccionada: si el administrador deja la banda en blanco, el sistema muestra «Debe seleccionar una banda». FA-4 Álbum inexistente: el sistema redirige al listado de álbumes con el mensaje «El álbum solicitado no existe.». FA-5 Cancelar: el administrador presiona «Cancelar» o «Volver» y regresa al listado sin guardar. FA-6 Sin permisos: el usuario no puede ejecutar la operación
Postcondiciones	El álbum queda actualizado y conserva sus canciones.
Reglas de negocio	RN-01, RN-08, RN-09, RN-10, RN-11, RN-29 y RN-30.

Relaciones	Extiende CU10 – Listar álbumes y CU13 – Consultar álbum.

Tabla 23

CU-12 - Eliminar álbum

Campo	Contenido
Código	CU-12
Nombre	Eliminar álbum
Actor	Administrador
Descripción	Permite eliminar un álbum del catálogo junto con sus canciones.
Precondiciones	1. El administrador ha iniciado sesión. 2. El álbum que se desea eliminar existe.
Flujo básico	1. En el listado de álbumes, el administrador selecciona «Eliminar» en la tarjeta del álbum. 2. El navegador solicita confirmación: «¿Seguro que deseas eliminar este álbum? Esta acción no se puede deshacer.». 3. El administrador acepta. 4. El sistema verifica que el álbum exista. 5. El sistema elimina el álbum y sus canciones (RN-27). 6. El sistema redirige al listado de álbumes con el mensaje «Álbum eliminado correctamente.».
Flujos alternativos	FA-1 Cancelar eliminación: si el administrador rechaza la confirmación, no se envía la petición y el álbum se conserva. FA-2 Álbum inexistente: el sistema redirige al listado con el mensaje «El álbum no existe.». FA-3 Sin permisos: el usuario no puede ejecutar la operación (ver criterios generales).
Postcondiciones	El álbum y sus canciones dejan de estar registrados.
Reglas de negocio	RN-01 y RN-27.
Relaciones	Extiende CU10 – Listar álbumes.

Tabla 24

CU-13 - Consultar álbum

Campo	Contenido
Código	CU-13
Nombre	Consultar álbum

Actor	Visitante / Lector / Administrador
Descripción	Permite ver la información detallada de un álbum, su banda y sus canciones.
Precondiciones	1. El sistema está disponible. 2. El álbum que se desea consultar existe.
Flujo básico	1. El usuario selecciona «Ver detalle» en un álbum del listado, o el título del álbum en el listado de canciones (CU15). 2. El sistema muestra la portada, el tipo, el título, la banda y el año, las etiquetas de rating («—» si no tiene), formato, género y país, y la descripción. 3. El sistema muestra la tabla de canciones con número de pista, título y duración en formato m:ss, ordenadas por pista. 4. El usuario puede seleccionar «Ver banda» (CU08) o «Volver a álbumes». 5. Si el usuario tiene sesión, el sistema muestra «Descargar PDF» (CU21); si es Administrador, muestra además «Editar álbum» (CU11).
Flujos alternativos	FA-1 Álbum inexistente: el sistema redirige al listado de álbumes con el mensaje «El álbum solicitado no existe.». FA-2 Sin canciones: el sistema muestra «Este álbum aún no tiene canciones registradas.».
Postcondiciones	El usuario visualiza los datos del álbum, su banda y sus canciones.
Reglas de negocio	RN-10 y RN-11.
Relaciones	Extiende CU10 – Listar álbumes. Es extendido por CU11 y CU21.

Tabla 25

CU-18 Registrar álbum con canciones

Campo	**Contenido**
Código	CU-18
Nombre	Registrar álbum con canciones
Actor	Administrador
Descripción	Permite registrar un álbum junto con hasta cinco canciones en una sola operación transaccional: si algo falla, no se guarda nada.
Precondiciones	1. El administrador ha iniciado sesión. 2. Existe al menos una banda registrada.
Flujo básico	1. En el módulo «Álbumes», el administrador selecciona «Álbum con canciones».

	2. El sistema muestra el formulario con los datos del álbum (título, banda, año, tipo, rating, formato y URL de portada) y cinco filas de canciones con pista, título y duración en segundos. 3. El administrador completa el álbum y las filas que necesite, y presiona «Guardar todo». 4. El sistema valida los datos del álbum (RN-08 a RN-11) y que se haya seleccionado una banda. 5. El sistema ignora las filas de canciones sin título (RN-17). 6. A cada fila con título que no tiene número de pista, el sistema le asigna el número de orden de su fila entre las filas con título (RN-18). 7. El sistema verifica que quede al menos una canción (RN-16). 8. En una sola transacción, el sistema registra el álbum y luego cada canción, validando la pista (RN-19), la duración y el título (RN-28) y que la pista no se repita en el álbum (RN-25). 9. El sistema redirige al listado de álbumes con el mensaje «Álbum y canciones registrados correctamente.».
Flujos alternativos	FA-1 Datos del álbum inválidos: el sistema vuelve al formulario con los mensajes bajo cada campo («El título es obligatorio», «El año es obligatorio», «Debe seleccionar una banda») y conserva lo escrito. FA-2 Sin canciones: si ninguna fila tiene título, el sistema muestra «Agrega al menos una canción.» y no guarda. FA-3 Canción inválida: si una fila con título tiene pista menor o igual a 0, no tiene duración (o es negativa), o su título supera los 100 caracteres, el sistema deshace toda la operación y muestra «No se guardó nada:» seguido del motivo (por ejemplo «Pista inválida: …», «Falta la duración (o es negativa) en: …», «Hay un título de más de 100 caracteres»), conservando lo escrito. FA-4 Pista repetida: si una pista se repite dentro del álbum, el sistema no guarda nada y muestra «No se guardó nada: revisa que no haya pistas repetidas en el álbum.», conservando lo escrito. FA-5 Cancelar: el administrador presiona «Cancelar» y regresa sin guardar. FA-6 Sin permisos: el usuario no puede ejecutar la operación
Postcondiciones	El álbum y sus canciones quedan registrados; si algo falla, no queda ningún registro nuevo.
Reglas de negocio	RN-01, RN-08 a RN-11, RN-16, RN-17, RN-18, RN-19, RN-25, RN-26 y RN-28.
Relaciones	Extiende CU10 – Listar álbumes.

Tabla 26

CU21 – Generar reporte de canciones de un álbum

Campo	**Contenido**

Código	CU-21
Nombre	Generar reporte de canciones de un álbum
Actor	Lector / Administrador
Descripción	Permite descargar en PDF la lista de canciones de un álbum desde su detalle.
Precondiciones	1. El usuario ha iniciado sesión (Lector o Administrador). 2. El álbum existe y el usuario está en su detalle (CU13).
Flujo básico	1. El usuario selecciona «Descargar PDF» en el detalle del álbum. 2. El sistema consulta las canciones del álbum, ordenadas por número de pista. 3. El sistema genera el reporte con el título del álbum, su banda y una tabla con pista, título y duración en formato m:ss («-» si no tiene). El pie dice «Melodic Vault - Página N». 4. El sistema envía el PDF al navegador, que lo abre en una pestaña nueva.
Flujos alternativos	FA-1 Álbum sin canciones: el sistema genera el PDF con el título, la banda y la cabecera de la tabla, sin filas. FA-2 Álbum inexistente: si el usuario accede por URL a un álbum que no existe, el sistema muestra la página de error 404. FA-3 Sin sesión: el botón «Descargar PDF» no se muestra; si el visitante abre la ruta directamente, el sistema lo redirige al formulario de acceso.
Postcondiciones	El usuario obtiene el PDF con las canciones del álbum.
Reglas de negocio	RN-24: la descarga de reportes requiere una sesión iniciada.
Relaciones	Extiende CU13 – Consultar álbum.

7.2.3.3.4. Canciones

Tabla 27

CU14 – Registrar canción

Campo	**Contenido**
Código	CU-14
Nombre	Registrar canción
Actor	Administrador
Descripción	Permite registrar una canción en el catálogo y asociarla obligatoriamente a un álbum existente.
Precondiciones	1. El administrador ha iniciado sesión.

	2. Existe al menos un álbum registrado. Si no hay álbumes, el formulario muestra «Todavía no hay álbumes. Registra uno antes de agregar canciones.».
Flujo básico	1. En el módulo «Canciones», el administrador selecciona «Nueva canción». 2. El sistema muestra el formulario vacío. 3. El administrador ingresa el título, selecciona el álbum de la lista (con formato «Álbum — Banda»), e ingresa el número de pista y la duración en segundos. El formulario muestra la duración equivalente en m:ss. 4. El administrador presiona «Guardar canción». 5. El sistema valida los campos (RN-12, RN-14, RN-15) y que se haya seleccionado un álbum (RN-13). 6. El sistema verifica que el número de pista no esté repetido en el álbum (RN-25). 7. El sistema registra la canción. 8. El sistema redirige al listado de canciones con el mensaje «Canción registrada correctamente.».
Flujos alternativos	FA-1 Datos incompletos: el sistema vuelve al formulario con los mensajes bajo cada campo («El título es obligatorio», «El número de pista es obligatorio», «La duración es obligatoria») y conserva lo escrito. FA-2 Pista inválida: si el número de pista es menor que 1, el sistema muestra «La pista debe ser 1 o mayor». FA-3 Duración negativa: el sistema muestra «La duración no puede ser negativa». FA-4 Pista duplicada: si ya existe esa pista en el álbum, el sistema no guarda y muestra «No se pudo guardar. Revisa que la pista no esté repetida en este álbum.» bajo el campo «Número de pista», conservando lo escrito. FA-5 Álbum no seleccionado: el sistema muestra «Debe seleccionar un álbum». FA-6 Cancelar: el administrador presiona «Cancelar» o «Volver» y regresa al listado sin guardar. FA-7 Sin permisos: el usuario no puede ejecutar la operación
Postcondiciones	La canción queda registrada y asociada al álbum seleccionado.
Reglas de negocio	RN-01, RN-12, RN-13, RN-14, RN-15 y RN-25.
Relaciones	Extiende CU15 – Listar canciones.

Tabla 28

CU15 – Listar canciones

Campo	Contenido
Código	CU0-15
Nombre	Listar canciones
Actor	Visitante / Lector / Administrador
Descripción	Permite ver todas las canciones registradas, con su álbum y su banda. El listado no tiene buscador ni página de detalle de canción.
Precondiciones	1. El sistema está disponible.
Flujo básico	1. El usuario selecciona «Canciones» en el menú. 2. El sistema muestra «N canciones en el catálogo» y una tabla con pista, título, álbum, banda y duración en formato m:ss. 3. El usuario puede seleccionar el título de un álbum para abrir su detalle (CU13). 4. Si el usuario es Administrador, el sistema muestra además «Nueva canción» y, en cada fila, «Editar» y «Eliminar».
Flujos alternativos	FA-1 Sin canciones: el sistema muestra «Aún no hay canciones». El administrador ve además «Registra la primera con el botón “Nueva canción”.».
Postcondiciones	El usuario visualiza las canciones registradas y el álbum y la banda a los que pertenece cada una.
Reglas de negocio	RN-13: toda canción está asociada a un álbum.
Relaciones	Es extendido por CU14, CU16 y CU17.

Tabla 29

CU16 – Editar cancion

Campo	Contenido
Código	CU-16
Nombre	Editar canción
Actor	Administrador
Descripción	Permite modificar los datos de una canción ya registrada, incluido su álbum.
Precondiciones	1. El administrador ha iniciado sesión.] 2. La canción que se desea modificar existe.
Flujo básico	1. En el listado de canciones, el administrador selecciona «Editar» en la fila de la canción. 2. El sistema muestra el formulario con los datos actuales de la

	canción. 3. El administrador modifica los campos necesarios. 4. El administrador presiona «Guardar cambios». 5. El sistema valida los campos (RN-12, RN-14, RN-15) y que haya un álbum seleccionado (RN-13). 6. El sistema verifica que el número de pista no esté repetido en el álbum (RN-25). 7. El sistema actualiza la canción. 8. El sistema redirige al listado de canciones con el mensaje «Canción actualizada correctamente.».
Flujos alternativos	FA-1. Datos incompletos: el sistema vuelve al formulario con los mensajes bajo cada campo y conserva lo escrito. FA-2. Pista inválida: si el número de pista es menor que 1, el sistema muestra «La pista debe ser 1 o mayor». FA-3. Duración inválida: si la duración falta o es negativa, el sistema muestra «La duración es obligatoria» o «La duración no puede ser negativa». FA-4 Pista duplicada: si la pista ya está asignada a otra canción del mismo álbum, el sistema muestra «No se pudo guardar. Revisa que la pista no esté repetida en este álbum.» bajo el campo «Número de pista». FA-5 Canción inexistente: el sistema redirige al listado de canciones con el mensaje «La canción solicitada no existe.». FA-6 Cancelar: el administrador presiona «Cancelar» o «Volver» y regresa al listado sin guardar. FA-7 Sin permisos: el usuario no puede ejecutar la operación (ver criterios generales).
Postcondiciones	Los datos de la canción quedan actualizados.
Reglas de negocio	RN-01, RN-12, RN-13, RN-14, RN-15 y RN-25.
Relaciones	Extiende CU15 – Listar canciones.

Tabla 30

CU17 – Eliminar canción

Campo	**Contenido**
Código	CU-17
Nombre	Eliminar canción
Actor	Administrador
Descripción	Permite eliminar una canción del catálogo. Solo se elimina esa canción.
Precondiciones	1. El administrador debe haber iniciado sesión.

	2. La canción que se desea eliminar existe.
Flujo básico	1. En el listado de canciones, el administrador selecciona «Eliminar» en la fila de la canción. 2. El navegador solicita confirmación: «¿Eliminar esta canción?». 3. El administrador acepta. 4. El sistema verifica que la canción exista. 5. El sistema elimina la canción. 6. El sistema redirige al listado de canciones con el mensaje «Canción eliminada correctamente.».
Flujos alternativos	FA-1 Cancelar eliminación: si el administrador rechaza la confirmación, no se envía la petición y la canción se conserva. FA-2 Canción inexistente: el sistema redirige al listado con el mensaje «La canción no existe.». FA-3 Sin permisos: el usuario no puede ejecutar la operación (ver criterios generales).
Postcondiciones	La canción deja de estar registrada y no aparece en el catálogo.
Reglas de negocio	RN-01: solo el Administrador puede eliminar información.
Relaciones	Extiende CU15 – Listar canciones.

7.3. Diseño de base de datos y persistencia JPA

7.3.1. Introducción

Melodic Vault fue una aplicación web de gestión de un catálogo musical, desarrollada para atender la dispersión de la información sobre bandas, álbumes y canciones. La persistencia de esa información se implementó sobre una base de datos relacional MySQL, conectada a la aplicación mediante Spring Data JPA e Hibernate. El modelo se diseñó a partir de las entidades necesarias para representar el catálogo, las relaciones entre ellas y las cuentas de acceso al sistema.

7.3.2. Modelo logico de base de datos

7.3.2.1. Entidades identificadas

Tabla *31*

Entidades del modelo lógico

Entidad	Detalle	Identificador
Banda	Grupos o artistas registrados en el catálogo. El género musical se guardó como atributo de la banda.	id_banda
Álbum	Producciones discográficas asociadas a una banda.	id_album
Canción	Pistas de un álbum, identificadas por su número de pista.	id_cancion
Usuario	Credenciales de las personas con cuenta en el sistema y su rol (ADMIN o LECTOR), que determinó el nivel de acceso.	id_usuario

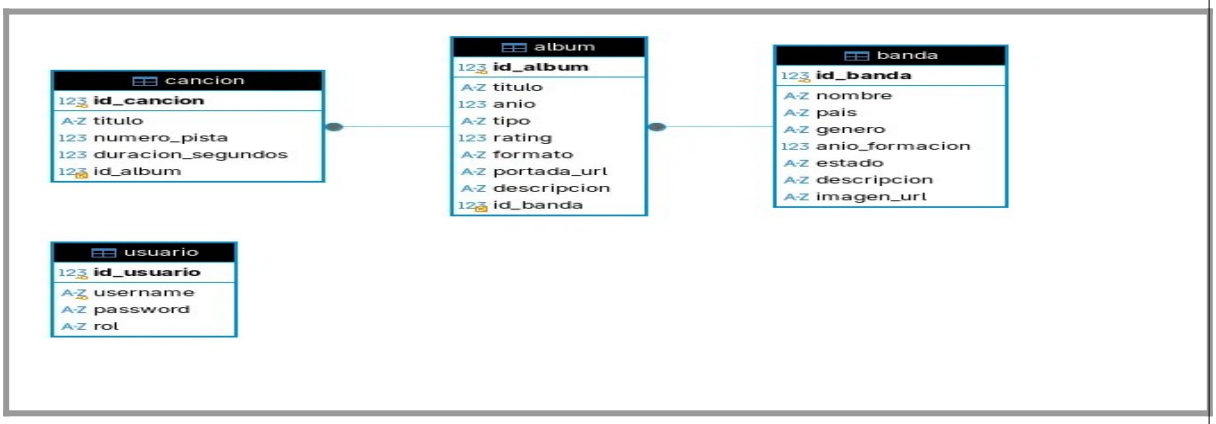
INTERPRETACION

La tabla delimitó el alcance del modelo: tres entidades describieron el catálogo (Banda, Álbum y Canción) y una describió la seguridad (Usuario). Cada entidad tuvo un identificador propio, que luego se convirtió en la clave primaria de su tabla.

7.3.2.2. Diagrama entidad-relación

Las relaciones fueron Banda 1:N Álbum y Álbum 1:N Canción: una banda registró múltiples álbumes y un álbum contuvo múltiples canciones, mientras que cada álbum perteneció a una banda y cada canción a un álbum. Usuario no se relacionó con las demás entidades.

Figura 6 : Modelo lógico de Melodic Vault



INTERPRETACION

El diagrama mostró cuatro entidades. Banda se relacionó 1:N con Álbum y Álbum 1:N con Canción, de modo que el catálogo se organizó en cadena: una canción no pudo existir sin álbum, ni un álbum sin banda. Usuario apareció aislado porque solo sostuvo las credenciales de acceso y no formó parte del catálogo.

7.3.3. Modelo físico de base de datos

7.3.3.1. Motor y base de datos

El modelo físico se implementó en MySQL, en la base de datos bd_melodicvault. Se utilizó el motor por defecto de MySQL, que soporta claves foráneas y eliminación en cascada. Las claves primarias fueron enteros autoincrementales.

7.3.3.2. Claves y restricciones

Tabla *32*

Restricciones definidas en el mod

Restricción	Tabla	Tipo	Columna(s)	Efecto
(PK)	banda	Clave primaria	id_banda	Identificó cada banda.
(PK)	album	Clave primaria	id_album	Identificó cada álbum.
(PK)	cancion	Clave primaria	id_cancion	Identificó cada canción.
(PK)	usuario	Clave primaria	id_usuario	Identificó cada usuario.
fk_album_banda	album	Clave foránea	id_banda → banda(id_banda)	Vinculó el álbum con su banda; ON DELETE CASCADE.
fk_cancion_album	cancion	Clave foránea	id_album → album(id_album)	Vinculó la canción con su álbum; ON DELETE CASCADE.
uq_cancion_album_pista	cancion	Única	(id_album, numero_pista)	Impidió repetir el número de pista dentro de un mismo álbum.
(UNIQUE)	usuario	Única	username	Impidió nombres de usuario repetidos.

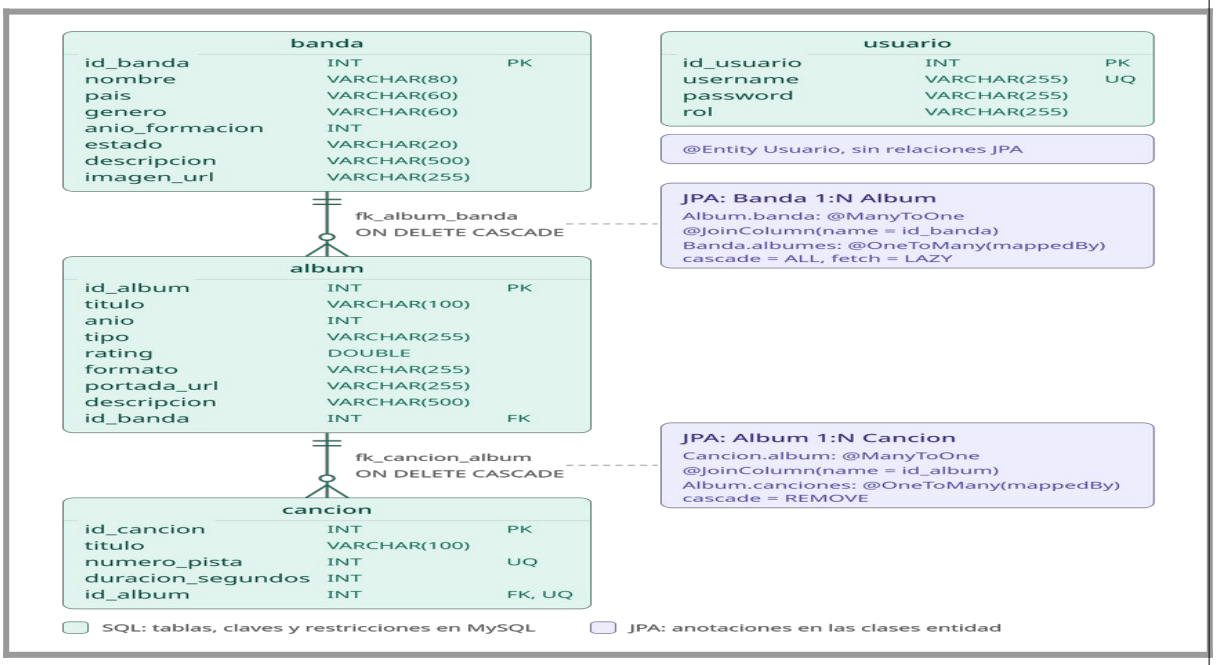
INTERPRETACION

Las claves primarias identificaron cada registro; las dos claves foráneas encadenaron banda, álbum y canción con eliminación en cascada, y las dos restricciones únicas protegieron las reglas de negocio: un solo usuario por nombre y una sola canción por número de pista dentro de cada álbum.

La unicidad de la pista se definió sobre el par álbum + pista, de modo que la pista 1 del álbum A no bloqueó la pista 1 del álbum b

7.3.3.3. Diagrama del modelo físico

Figura 7 : Modelo fisico de Melodic Vault



INTERPRETACION

El diagrama combinó dos capas. La capa SQL (tablas en verde azulado) mostró el modelo físico: tipos de dato, claves primarias, claves foráneas (fk_album_banda y fk_cancion_album), la restricción única de cancion y la eliminación en cascada; esas relaciones quedaron declaradas de forma explícita en el script.

La capa JPA (recuadros en morado) mostró cómo cada relación se reprodujo en las clases entidad: el lado propietario con @ManyToOne y @JoinColumn, el lado inverso con @OneToMany(mappedBy), y las reglas de cascada

Lo que JPA agregó al modelo fue la navegación en ambos sentidos entre objetos (por ejemplo, de una banda a sus álbumes), la cascada a nivel de aplicación y la validación de datos mediante Bean Validation, que el SQL por sí solo no expresa.

7.3.3.4. Decisiones de tipos de dato

• rating se definió como DOUBLE para almacenar valores con decimales (por ejemplo, 9.2). El rango de 0 a 10 se validó en la aplicación, no con una restricción en la base de datos

• duracion_segundos se almacenó como entero en segundos; la aplicación y los reportes la mostraron en formato m:ss.

• password se definió como VARCHAR(255) para guardar el hash BCrypt (de unos 60 caracteres), nunca la contraseña en texto plano.

• rol se guardó sin el prefijo ROLE_, que Spring Security agrega automáticamente.

• Las URL de imágenes (imagen_url, portada_url) se guardaron como texto (VARCHAR(255)); la aplicación no almacenó archivos binarios.

7.3.4. Diccionario de datos

Tabla *33*

Diccionario de datos de la tabla banda

Campo	Tipo de dato	Clave	Restricción	Descripción
id_banda	INT	PK	Autoincremental	Identificador único de la banda.
nombre	VARCHAR(80)		NOT NULL	Nombre de la banda o artista.
pais	VARCHAR(60)		NOT NULL	País de procedencia.
genero	VARCHAR(60)		NOT NULL	Género musical principal de la banda.
anio_formacion	INT		NOT NULL	Año de formación de la banda.

estado	VARCHAR(20)		NOT NULL	Estado de la banda (por ejemplo, Activa).
descripcion	VARCHAR(500)		Opcional	Reseña o biografía de la banda.
imagen_url	VARCHAR(255)		Opcional	Dirección de la imagen de la banda.

INTERPRETACION

banda fue la tabla raíz del catálogo y no dependió de ninguna otra. Los cinco campos obligatorios (nombre, país, género, año de formación y estado) garantizaron una ficha mínima completa; la descripción y la imagen quedaron opcionales.

Tabla *34*

Diccionario de datos de la tabla cancion

Campo	Tipo de dato	Clave	Restricción	Descripción
id_cancion	INT	PK	Autoincremental	Identificador único de la canción.
titulo	VARCHAR(100)		NOT NULL	Título de la canción.
numero_pista	INT		NOT NULL; mínimo 1 validado en la aplicación; único junto con id_album	Posición de la canción dentro del álbum.
duracion_segundos	INT		Obligatorio; mínimo 0 validado en la aplicación	Duración en segundos.
id_album	INT	FK	NOT NULL; referencia a album con eliminación en cascada	Álbum al que pertenece.

INTERPRETACION

Cancion fue la tabla más dependiente del modelo: id_album es obligatorio y el par (id_album, numero_pista) es único, de modo que ninguna canción puede existir sin álbum ni repetir pista dentro de él.

El mínimo de la pista y de la duración no lo restringe la base de datos (en el caso de duracion solo exige que no sea null), lo valida la aplicación.

Tabla *35*

Diccionario de datos de la tabla album

Campo	Tipo de dato	Clave	Restricción	Descripción
id_album	INT	PK	Autoincremental	Identificador único del álbum.
titulo	VARCHAR(100)		NOT NULL	Título del álbum.
anio	INT		NOT NULL	Año de lanzamiento.
tipo	VARCHAR(255)		NOT NULL	Tipo de producción discográfica (por ejemplo, Studio Album).
rating	DOUBLE		Opcional; rango 0.0 a 10.0 validado en la aplicación	Calificación del álbum.
formato	VARCHAR(255)		Opcional	Formato de la edición (por ejemplo, Vinilo o CD).
portada_url	VARCHAR(255)		Opcional	Dirección de la imagen de portada.
descripcion	VARCHAR(500)		Opcional	Reseña del álbum.

id_banda	INT	FK	NOT NULL; referencia a banda con eliminación en cascada	Banda a la que pertenece el álbum.

INTERPRETACION

album ocupó el centro de la cadena del catálogo: dependió de banda mediante id_banda, que es obligatoria, y de él dependieron las canciones.

Los campos obligatorios (título, año, tipo y banda) garantizaron una ficha mínima; rating, formato, portada y descripción quedaron opcionales.

El rango del rating (0.0 a 10.0) no lo restringe la base de datos: lo aplicó la validación de la entidad con @DecimalMin y @DecimalMax.

Tabla *36*

Diccionario de datos de la tabla usuario

Campo	Tipo de dato	Clave	Restricción	Descripción
id_usuario	INT	PK	Autoincremental	Identificador único del usuario.
username	VARCHAR(255)	UQ	NOT NULL; único	Nombre con el que se inicia sesión.
password	VARCHAR(255)		NOT NULL	Hash BCrypt de la contraseña.
rol	VARCHAR(255)		NOT NULL	Rol de acceso: ADMIN o LECTOR (sin el prefijo ROLE_).

INTERPRETACION

usuario no tuvo claves foráneas porque no se relacionó con el catálogo. username fue único, password almacenó solo el hash BCrypt, nunca la contraseña en texto plano, y rol tomó los valores ADMIN o LECTOR.

La base de datos no restringe esos dos valores: los fija la aplicación, que asigna LECTOR en el registro.

7.3.5. Script SQL de creación de tablas

Figura 8 : Script Creacion de Tablas

```sql
-- ============ TABLAS ============

CREATE TABLE banda (
    id_banda         INT AUTO_INCREMENT PRIMARY KEY,
    nombre           VARCHAR(80)  NOT NULL,
    pais             VARCHAR(60)  NOT NULL,
    genero           VARCHAR(60)  NOT NULL,
    anio_formacion   INT          NOT NULL,
    estado           VARCHAR(20)  NOT NULL,
    descripcion      VARCHAR(500),
    imagen_url       VARCHAR(255)
);

CREATE TABLE album (
    id_album      INT AUTO_INCREMENT PRIMARY KEY,
    titulo        VARCHAR(100) NOT NULL,
    anio          INT          NOT NULL,
    tipo          VARCHAR(255) NOT NULL,
    rating        DOUBLE,
    formato       VARCHAR(255),
    portada_url   VARCHAR(255),
    descripcion   VARCHAR(500),
    id_banda      INT          NOT NULL,
    CONSTRAINT fk_album_banda FOREIGN KEY (id_banda) REFERENCES banda(id_banda)
        ON DELETE CASCADE
);

CREATE TABLE usuario (
    id_usuario  INT AUTO_INCREMENT PRIMARY KEY,
    username    VARCHAR(255) NOT NULL UNIQUE,
    password    VARCHAR(255) NOT NULL,
    rol         VARCHAR(255) NOT NULL
);

CREATE TABLE cancion (
    id_cancion          INT AUTO_INCREMENT PRIMARY KEY,
    titulo              VARCHAR(100) NOT NULL,
    numero_pista        INT          NOT NULL,
    duracion_segundos   INT,
    id_album            INT          NOT NULL,
    CONSTRAINT fk_cancion_album FOREIGN KEY (id_album) REFERENCES album(id_album)
        ON DELETE CASCADE,
    CONSTRAINT uq_cancion_album_pista UNIQUE (id_album, numero_pista)
);
```

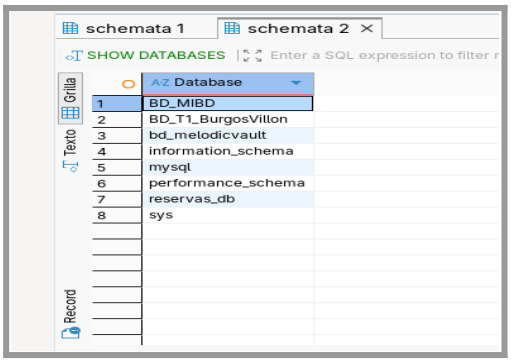
INTERPRETACION

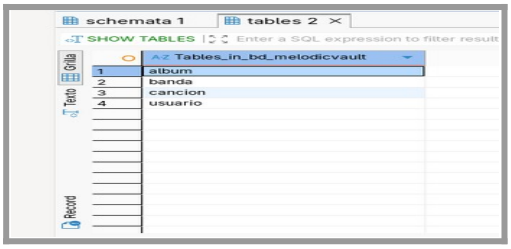
La restricción única de canciones se incluyó dentro del CREATE TABLE porque spring.jpa.hibernate.ddl-auto=update no es confiable para crear o modificar restricciones sobre tablas ya existentes. Por ello, el script SQL fue la fuente oficial del esquema.

7.3.6. Verificación de la base de datos en MySQL

La creación de la base de datos se comprobó ejecutando el script en MySQL y consultando su estructura y contenido

Figura 9 : Script - Mostrar base de datos

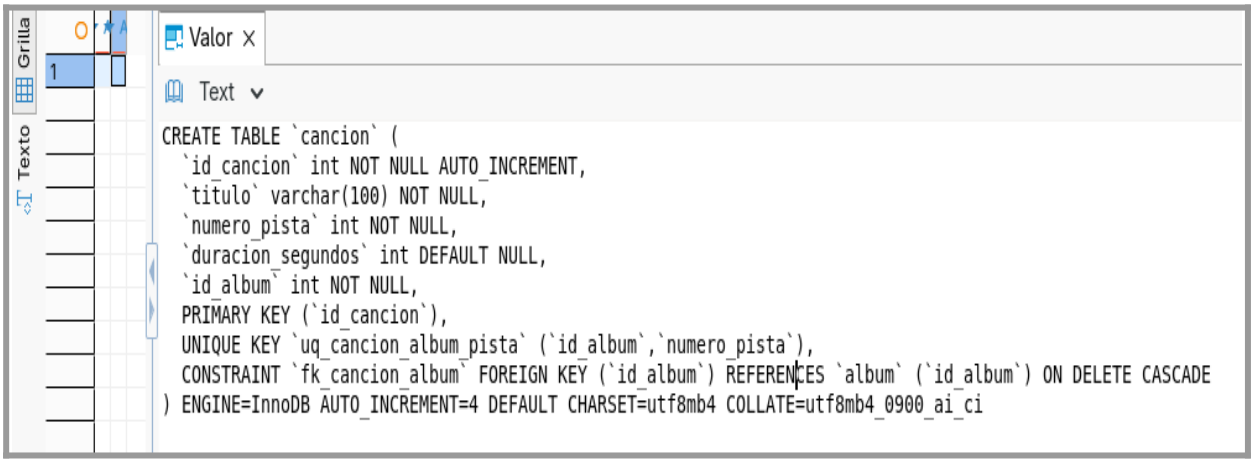




INTERPRETACION

La base `bd_melodicvault` apareció en la lista y contuvo exactamente cuatro tablas: `banda`, `album`, `cancion` y `usuario`.

Figura 10 : Restricción única y claves foráneas de cancion

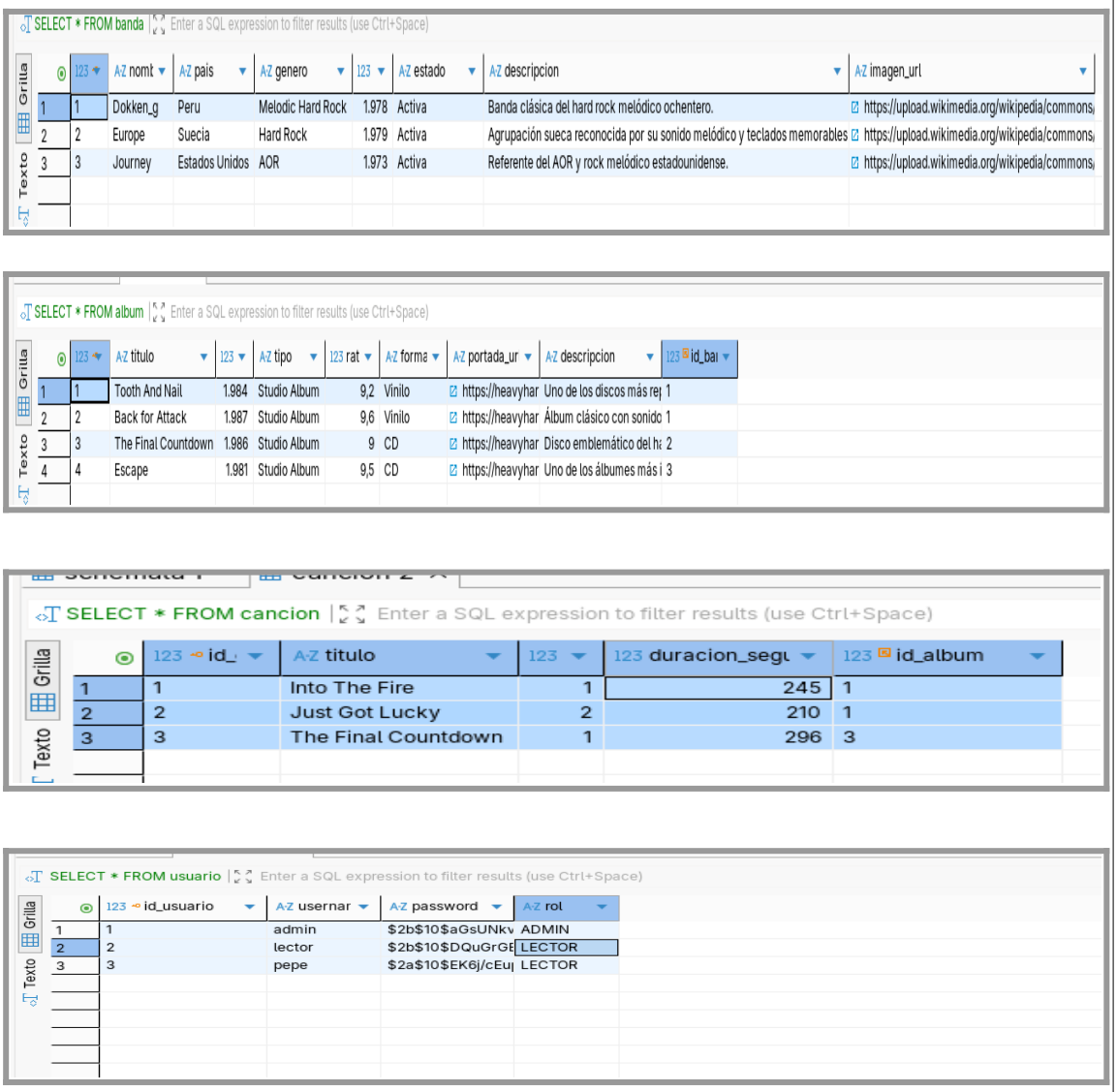


INTERPRETACION

El CREATE TABLE de cancion muestra una única restricción única, uq_cancion_album_pista, y la clave foránea con ON DELETE CASCADE.

La unicidad está definida solo en el script SQL; la entidad no la duplica, por lo que Hibernate no creó ninguna restricción adicional al arrancar la aplicación

Figura 11 :Datos de prueba cargados



INTERPRETACION

Los datos de prueba se cargaron completos (3 bandas, 4 álbumes, 3 canciones y 3 usuarios) y las contraseñas aparecieron como hash, no como texto.

7.3.7. Persistencia con JPA

Figura 12 :Imagen- Application properties



INTERPRETACION

La configuración reunida en application.properties enlazó la aplicación Spring Boot con la base de datos MySQL bd_melodicvault, de modo que la capa JPA pudo persistir y recuperar las entidades del dominio. Primero se fijó el puerto de servicio (8090), después se definió la URL JDBC de conexión al motor MySQL, luego se establecieron las credenciales de acceso .

Así, la persistencia quedó centralizada y reproducible desde un único archivo de configuración.

7.3.8. Clases entidad del proyecto

7.3.8.1. Entidad Banda

Figura 13 :Cap entitie banda

```java
@Entity
@Table(name = "banda")
public class Banda {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer idBanda;

    @NotBlank(message = "El nombre es obligatorio")
    @Size(max = 80)
    private String nombre;

    @NotBlank(message = "El país es obligatorio")
    @Size(max = 60)
    private String pais;

    @NotBlank(message = "El género es obligatorio")
    @Size(max = 60)
    private String genero;

    @NotNull(message = "El año de formación es obligatorio")
    private Integer anioFormacion;

    @NotBlank(message = "El estado es obligatorio")
    @Size(max = 20)
    private String estado;

    @Size(max = 500)
    private String descripcion;

    @Size(max = 255)
    private String imagenUrl;

    @OneToMany(mappedBy = "banda", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
    private List<Album> albumes;


    //getters y setters
```

INTERPRETACION

Banda se mapeó a la tabla banda mediante @Entity y @Table(name = “banda”).

@Id con @GeneratedValue(strategy = IDENTITY) delegó la generación de id_banda al autoincremental de MySQL.

Los atributos no llevaron @Column: Spring Boot convirtió automáticamente los nombres en camelCase (idBanda, anioFormacion, imagenUrl) a los nombres de columna en snake_case (id_banda, anio_formacion, imagen_url).

Las anotaciones de Bean Validation (@NotBlank, @NotNull, @Size) replicaron en Java los NOT NULL y las longitudes de la tabla. @OneToMany(mappedBy = “banda”) expuso la lista de álbumes sin crear una columna adicional.

7.3.8.2. Entidad Album

Figura 14 :Cap entitie album

```
@Table(name = "album")
public class Album {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer idAlbum;

    @NotBlank(message = "El título es obligatorio")
    @Size(max = 100)
    private String titulo;

    @NotNull(message = "El año es obligatorio")
    private Integer anio;

    @NotBlank(message = "El tipo es obligatorio")
    private String tipo;

    @DecimalMin(value = "0.0", message = "El rating mínimo es 0")
    @DecimalMax(value = "10.0", message = "El rating máximo es 10")
    private Double rating;

    private String formato;

    @Size(max = 255)
    private String portadaUrl;

    @Size(max = 500)
    private String descripcion;

    @NotNull(message = "Debe seleccionar una banda")
    @ManyToOne
    @JoinColumn(name = "id_banda", nullable = false)
    private Banda banda;

    public Integer getIdAlbum() {
        return idAlbum;
    }
```

INTERPRETACION

Album se mapeó a la tabla album. @ManyToOne con @JoinColumn(name = id_banda) representó la clave foránea hacia banda, y @OneToMany(mappedBy = album) expuso sus canciones ordenadas por pista mediante @OrderBy. @DecimalMin y @DecimalMax aplicaron el rango del rating, que la base de datos no restringe.

7.3.8.3. Entidad Cancion

Figura 15 :Cap entitie cancion

```
17 @Entity
18 @Table(name = "cancion")
19
20 public class Cancion {
21     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
22     private Integer idCancion;
23     @NotBlank(message = "El título es obligatorio")
24     @Size(max = 100)
25     private String titulo;
26
27     @NotNull(message = "El número de pista es obligatorio")
28     @Min(value = 1, message = "La pista debe ser 1 o mayor")
29     private Integer numeroPista;
30
31     @NotNull(message = "La duración es obligatoria")
32     @Min(value = 0, message = "La duración no puede ser negativa")
33     private Integer duracionSegundos;
34
35     @NotNull(message = "Debe seleccionar un álbum")
36     @ManyToOne
37     @JoinColumn(name = "id_album", nullable = false)
38     private Album album;
```

INTERPRETACION

Cancion se mapeó a la tabla cancion mediante @Entity y @Table(name = “cancion”). @ManyToOne con @JoinColumn(name = “id_album”, nullable = false) representó la clave foránea hacia album, de modo que toda canción quedó obligatoriamente asociada a un álbum.

Los atributos no llevaron @Column: Spring Boot convirtió automáticamente los nombres en camelCase (numeroPista, duracionSegundos) a los nombres de columna en snake_case (numero_pista, duracion_segundos).

Las anotaciones de Bean Validation (@NotBlank, @Size, @NotNull, @Min) replicaron en Java las reglas de la tabla. La unicidad de (id_album, numero_pista) no se declaró en la entidad: la garantiza únicamente la restricción uq_cancion_album_pista del script SQL, por lo que Hibernate no creó ninguna restricción adicional.

7.3.8.4. Entidad Usuario

Figura 16 :Cap entitie usuario

```java
@Entity
@Table(name = "usuario")
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer idUsuario;
    @Column(unique = true, nullable = false)
    private String username;
    @Column(nullable = false)
    private String password;
    @Column(nullable = false)
    private String rol;
```

INTERPRETACION

Usuario se mapeó a la tabla usuario y no tuvo anotaciones de relación. @Column(unique = true) reflejó la restricción UNIQUE de username, y password guardó el hash BCrypt con una longitud suficiente para almacenarlo.

7.3.9. Anotaciones utilizadas

Tabla *37*

Anotaciones JPA, Bean Validation y Spring Data utilizadas

Anotación	Origen	Dónde se usó	Para qué se usó

@Entity	JPA	Las cuatro clases del modelo	Identificó la clase como persistente.
@Table	JPA	Las cuatro clases	Especificó la tabla asociada a cada entidad.
@Id	JPA	Clave primaria de cada entidad	Identificó la clave primaria.
@GeneratedValue(strategy = IDENTITY)	JPA	Clave primaria de cada entidad	Generó el identificador de forma automática (autoincremental de MySQL).
@Column	JPA	Atributos con nombre distinto al de la columna o con restricciones	Declaró la unicidad del nombre de usuario
@OneToMany(mappedBy, cascade, fetch)	JPA	Banda.albumes, Album.canciones	Representó el lado "uno" de la relación 1:N.
@ManyToOne	JPA	Album.banda, Cancion.album	Representó el lado "muchos" de la relación.
@JoinColumn	JPA	Album.banda, Cancion.album	Indicó la columna de la clave foránea.
@OrderBy	JPA	Album.canciones	Ordenó las canciones por número de pista.
@NotBlank, @NotNull	Bean Validation	Atributos obligatorios	Rechazaron valores vacíos o nulos antes de guardar.
@Size, @Min	Bean Validation	Textos, pista y duración	Controlaron longitudes máximas y valores mínimos.
@DecimalMin,	Bean	Album.rating	Limitaron el rating entre

@DecimalMax	Validation		0.0 y 10.0.
@Query	Spring Data	Repositorios IAlbum	Definió consultas JPQL específicas.
@EntityGraph	Spring Data	IBanda	Cargó la banda y sus álbumes en una sola consulta.
@Transactional	Spring	AlbumTransaccionalService	Ejecutó el registro de álbum con canciones como una sola transacción.

INTERPRETACION

Las anotaciones cumplieron tres propósitos: @Entity, @Table, @Id, @GeneratedValue mapearon clases y atributos a tablas y columnas; @OneToMany, @ManyToOne y @JoinColumn reprodujeron las claves foráneas; y Bean Validation, @Query, @EntityGraph y @Transactional cubrieron la validación, las consultas y las transacciones.

Las anotaciones @OneToOne y @ManyToMany no se utilizaron porque el modelo no tuvo relaciones de esos tipos: la entidad Usuario no se relacionó con las demás y ninguna relación del catálogo fue de muchos a muchos.

7.3.10. Relaciones entre entidades

Tabla *38*

Relaciones JPA del proyecto

Relación	Lado propietario (clave foránea)	Lado inverso	Cascada en JPA	Cascada en MySQL
Banda 1:N Álbum	Album.banda con @ManyToOne y @JoinColumn(name = "id_banda")	Banda.albumes con @OneToMany(mappedBy	CascadeType.ALL, carga LAZY	fk_album_banda con ON DELETE CASCADE

		= "banda")		
Álbum 1:N Canción	Cancion.album con @ManyToOne y @JoinColumn(name = "id_album")	Album.canciones con @OneToMany(mappedBy = "album")	CascadeType.REMOVE	fk_cancion_album con ON DELETE CASCADE

INTERPRETACION

En ambas relaciones el lado propietario fue el que contiene la clave foránea (`Album.banda` y `Cancion.album`). La cascada se definió en JPA y en MySQL a la vez: eliminar una banda eliminó sus álbumes y canciones tanto desde la aplicación como desde la base de datos.

No se activó orphanRemoval porque el formulario de edición del álbum no envía las canciones; con esa opción, guardar el álbum las habría borrado.

Eliminar un álbum sigue arrastrando sus canciones por CascadeType.REMOVE

7.3.11. Repositorios y consultas

Figura 17 : Imagen – Consultas JPQL

```
public interface IAlbum extends CrudRepository<Album, Integer> {

    List<Album> findByTituloContainingIgnoreCaseOrTipoContainingIgnoreCaseOrBandaNombreContainingIgnoreCase(
            String titulo, String tipo, String banda
    );

    @Query("SELECT a.banda.nombre, COUNT(a), AVG(a.rating) FROM Album a GROUP BY a.banda.nombre ORDER BY COUNT(a) DESC")
    List<Object[]> estadisticasPorBanda();

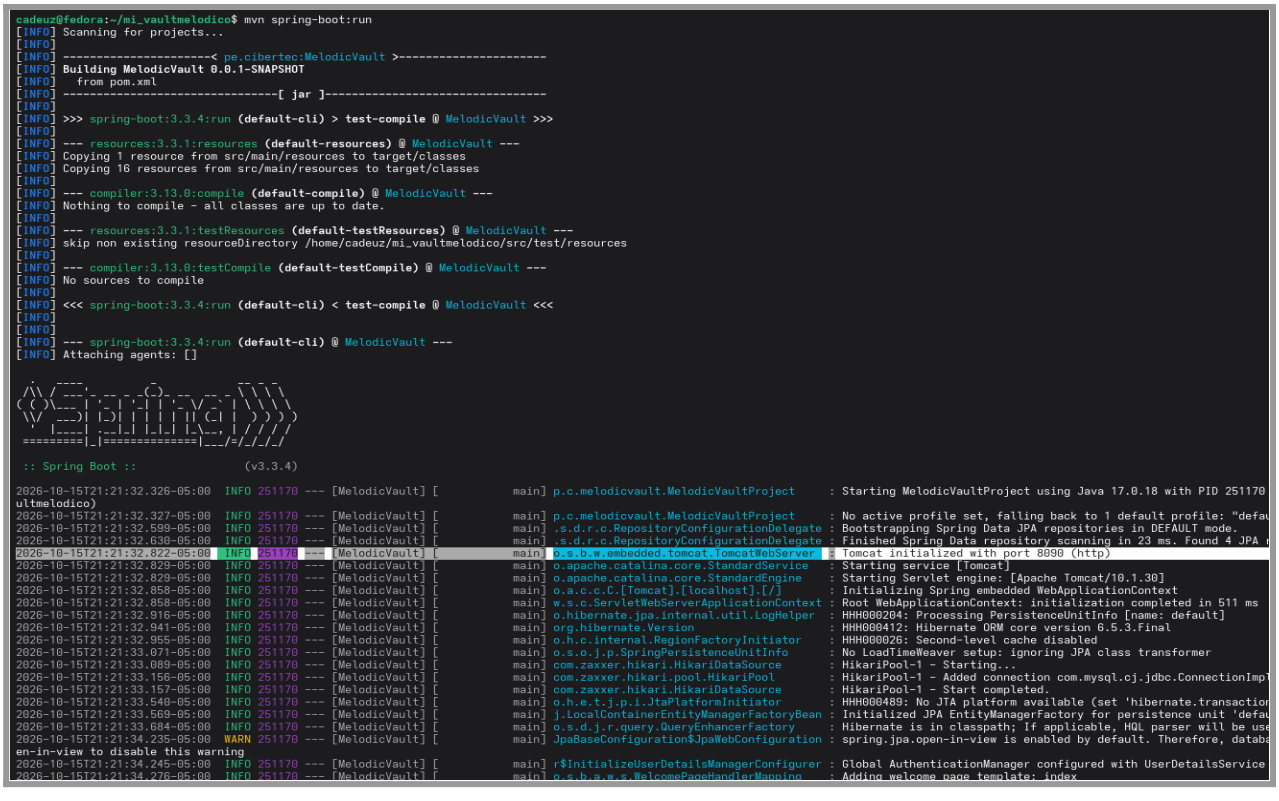
    @Query("SELECT c FROM Cancion c WHERE c.album.idAlbum = :id ORDER BY c.numeroPista")
    List<Cancion> listadoDeCanciones(@Param("id") int id);
}
```

INTERPRETACION

El acceso a la información se realizó mediante repositorios que extendieron CrudRepository, lo que permitió ejecutar operaciones CRUD (save, findById, findAll, deleteById, count) sin implementar manualmente las instrucciones SQL. Las búsquedas se resolvieron con métodos derivados de Spring Data, y las consultas más específicas con @Query en JPQL.

7.3.12. Evidencia de conexión entre la aplicación y base de datos

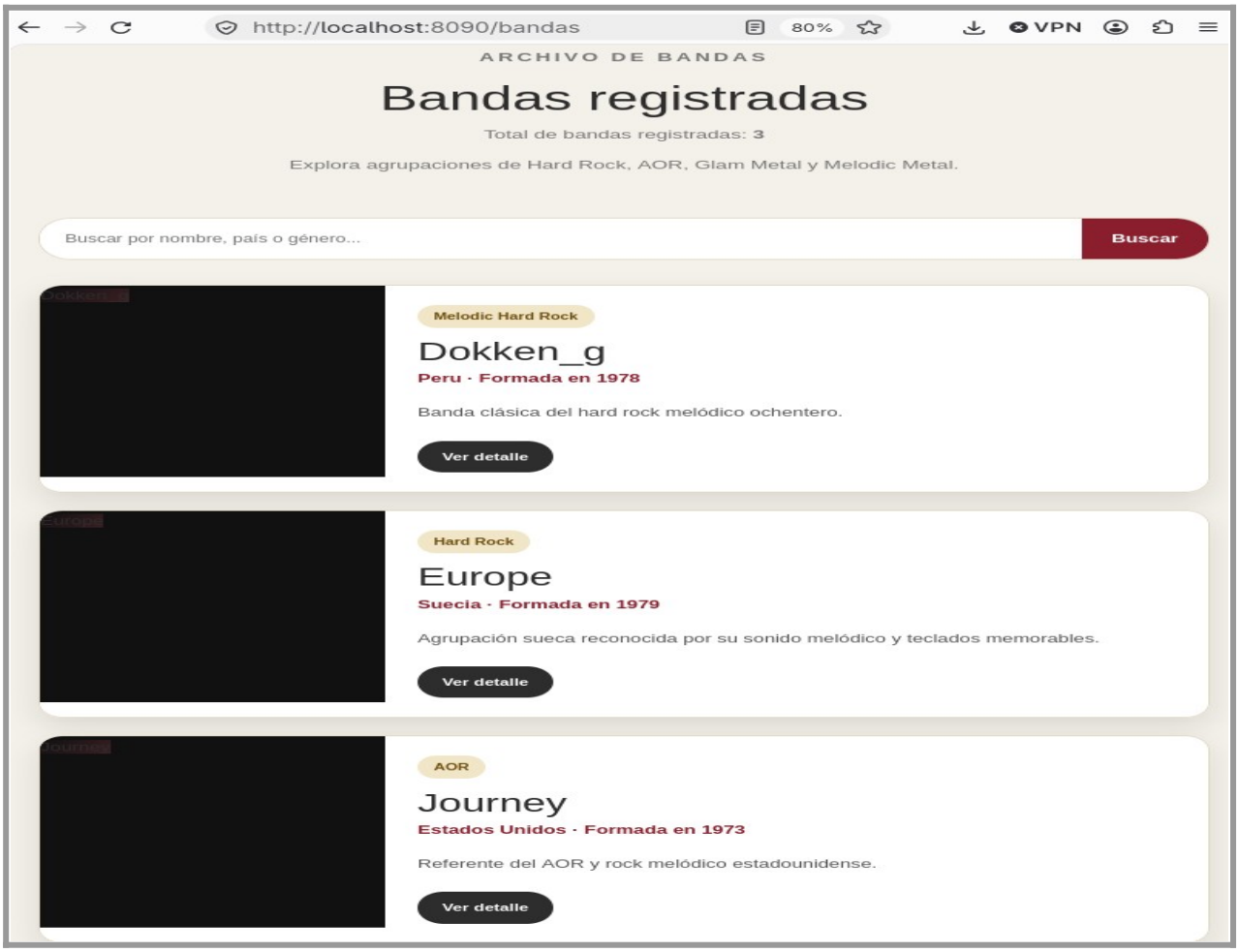
Figura 18 : Imagen – Arranque



INTERPRETACION

La consola de arranque mostró el pool de conexiones iniciado sin errores y el servidor activo en el puerto 8090, es decir, la aplicación se conectó a MySQL al arrancar.

Figura 19 : Imagen – Vista froentend listado bandas



INTERPRETACION

Los registros que mostró la aplicación (Dokken, Europe y Journey) coincidieron con los del SELECT en MySQL: la lectura funcionó.

7.3.13. Conclusiones de la sección

El diseño de la base de datos representó el catálogo con tres tablas relacionadas (banda, álbum y canción) y una tabla independiente de usuarios, con integridad garantizada por claves foráneas, restricciones únicas y eliminación en cascada. Cada tabla se asoció a una clase entidad JPA mediante @Entity y @Table, y las relaciones 1:N se mapearon con @OneToMany y @ManyToOne. Los repositorios de Spring Data permitieron operar sobre objetos Java, y la evidencia de conexión demostró que los datos registrados desde la aplicación se almacenaron y se leyeron correctamente en MySQL.

7.4. Configuración del proyecto con Maven

Maven fue la herramienta de construcción y de gestión de dependencias de MelodicVault. El proyecto se definió con las coordenadas pe.cibertec:MelodicVault:0.0.1-SNAPSHOT, tomó como padre a spring-boot-starter-parent 3.3.4 y se compiló con Java 17.

7.4.1. Configuración del proyecto con Maven

El proyecto respetó la estructura estándar de Maven. El código fuente se organizó bajo el paquete base pe.cibertec.melodicvault, cuya clase de arranque fue MelodicVaultProject, y los subpaquetes se separaron por capa.

Tabla *39*

Directorios principales del proyecto Maven

Ruta (los paquetes parten de src/main/java/pe/cibertec/melodicvault)	Qué aportó en MelodicVault	Con qué se conectó
pom.xml	Fijó Spring Boot 3.3.4 y Java 17, declaró las dependencias y el plugin que empaquetó el .jar.	Maven lo leyó para descargar las dependencias y llenar target/; los starters tomaron su versión del

		parent.
config	SecurityConfig: definió qué rutas eran públicas, cuáles exigían sesión y cuáles solo ADMIN, y el cifrado BCrypt.	Interceptó las peticiones antes de que llegaran a controller y autenticó con UsuarioDetailsService.
controller	Ocho controladores (HomeController, LoginController, RegistroController, BandaController, AlbumController, AlbumCompletoController, CancionController, ReporteController) que atendieron las rutas del sistema.	Recibieron la petición, llamaron a los servicios y respondieron con una vista de templates o con un PDF.
dto	RegistroForm: recibió usuario, contraseña y confirmación, sin campo rol.	Lo usó RegistroController; el servidor fijó el rol LECTOR, así nadie pudo enviarse ADMIN desde el formulario.
interfaces	Repositorios IBanda, IAlbum, ICancion e IUsuario (extendieron CrudRepository), con las consultas del sistema: búsquedas, estadísticas, canciones por álbum y usuarios.	Los usaron los service; trabajaron sobre las entidades de modelo y Hibernate los tradujo a SQL contra MySQL.
interfacesService	Contratos IBandaService, IAlbumService e ICancionService.	Los controladores invocaron estos contratos; sus implementaciones

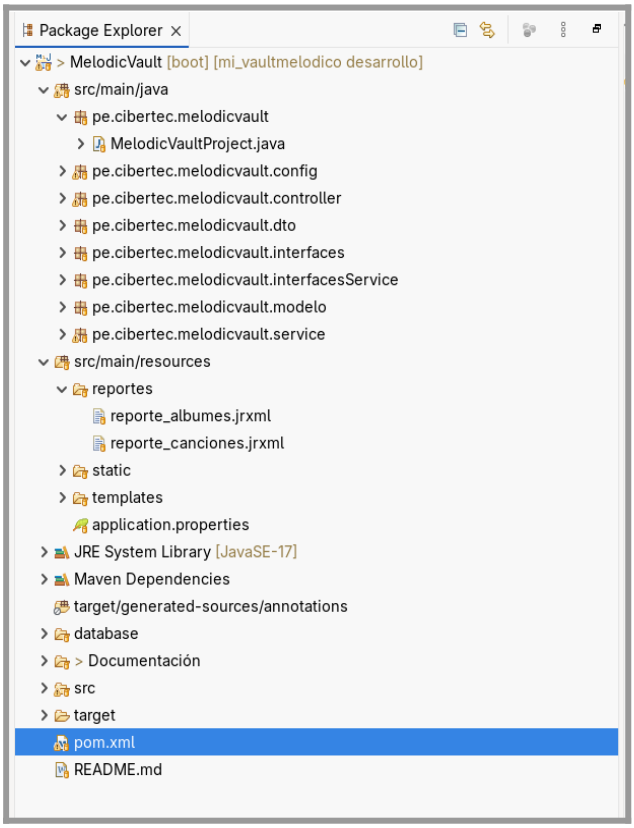
		estuvieron en service.
service	BandaService, AlbumService y CancionService; AlbumTransaccionalService (guardó álbum y canciones en una sola transacción); UsuarioDetailsService (login contra la base) y ReporteService (generó los PDF).	Llamaron a los repositorios de interfaces; SecurityConfig usó UsuarioDetailsService y ReporteService leyó las plantillas de resources/reportes.
modelo	Entidades Banda, Album, Cancion y Usuario, con sus validaciones y la relación banda → álbumes → canciones con eliminación en cascada.	Hibernate las mapeó a las tablas de bd_melodicvault.
src/main/resources/ templates	Las 13 vistas HTML: inicio, listados, detalles, formularios, login y registro.	Recibieron del controlador los datos en el Model, mostraron u ocultaron botones con sec:authorize según el rol y cargaron la hoja de estilos de static.
src/main/resources/ reportes	reporte_canciones.jrxml y reporte_albumes.jrxml.	ReporteService los compiló una sola vez al iniciar; sus campos coincidieron con los atributos de las entidades de modelo.
src/main/resources/ static	css/estilos.css, img/ y js/.	Las vistas los cargaron; SecurityConfig los dejó públicos para que se vieran sin sesión.

src/main/resources/ application.properties	Puerto 8090, conexión a bd_melodicvault y ddl-auto=update.	Spring Boot lo leyó al arrancar; el conector mysql-connector-j usó esa URL para que Hibernate llegara a MySQL.
target/	Clases compiladas y .jar generados por Maven.	Se llenó al ejecutar compile, package e install.

INTERPRETACION

La tabla mostró que el proyecto siguió la convención de Maven: código en src/main/java, recursos en src/main/resources y artefactos generados en target. Dentro del paquete base, cada capa del patrón MVC quedó en su propio subpaquete. El proyecto no tuvo carpeta src/test, por lo que no incluyó pruebas automatizadas.

Figura 20 : Árbol del proyecto MelodicVault en Eclipse IDE



INTERPRETACION

La figura mostró que el proyecto conservó el esqueleto que genera Spring Initializr al crear un proyecto Spring Boot con Maven: pom.xml en la raíz con spring-boot-starter-parent como padre, src/main/java con la clase de arranque en el paquete base, y src/main/resources con static, templates y application.properties.

Sobre esa base se agregaron elementos propios del sistema, como los subpaquetes por capa, la carpeta reportes y dependencias como Spring Security y JasperReports, sin alterar la estructura original.

Esto fue posible porque Maven se rigió por convenciones: bastó declarar las nuevas dependencias en el pom.xml y ubicar los archivos en las carpetas estándar para que se compilaran y empaquetaran sin configuración adicional.

Y, como MelodicVaultProject quedó en el paquete base, Spring Boot detectó automáticamente los componentes de todos los subpaquetes agregados, y las plantillas .jrxml ubicadas en resources/reportes quedaron en el classpath, por lo que ReporteService pudo cargarlas al iniciar la aplicación.

7.4.2. Archivo pom.xml

Figura 21 : Proyecto padre, coordenadas y versión de Java en el pom.xml

```
7   <parent>
8       <groupId>org.springframework.boot</groupId>
9       <artifactId>spring-boot-starter-parent</artifactId>
10      <version>3.3.4</version> Upgrade to the Latest Patch
11      <relativePath/>
12  </parent>
13
14  <groupId>pe.cibertec</groupId>
15  <artifactId>MelodicVault</artifactId>
16  <version>0.0.1-SNAPSHOT</version>
17  <name>MelodicVault</name>
18  <description>Catálogo web de bandas, álbumes y canciones</description>
19
20  <properties>
21      <java.version>17</java.version>
22  </properties>
```

INTERPRETACION

Al heredar de spring-boot-starter-parent, el proyecto recibió un conjunto de versiones probadas entre sí (Spring, Hibernate, Thymeleaf, Tomcat y el conector de MySQL), por eso la mayoría de dependencias no declararon <version> y no hubo conflictos entre librerías.

Las tres coordenadas (groupId, artifactId y version) identificaron de forma única al proyecto.

La propiedad java.version fijó Java 17, que el padre usó para configurar el compilador; fue el mínimo que exigió Spring Boot 3.

Figura 22 : Distintas formas de declarar una dependencia en el pom.xml

MelodicVault/pom.xml

```
23
24    <dependencies>
25        <!-- Sin <version>: la versión la administra el parent -->
26        <dependency>
27            <groupId>org.springframework.boot</groupId>
28            <artifactId>spring-boot-starter-security</artifactId>
29        </dependency>
30
```

```
64
65        <!-- Con <version>: JasperReports no está administrada por Spring Boot -->
66        <dependency>
67            <groupId>net.sf.jasperreports</groupId>
68            <artifactId>jasperreports</artifactId>
69            <version>6.20.6</version>
70        </dependency>
71        <dependency>
72            <groupId>net.sf.jasperreports</groupId>
73            <artifactId>jasperreports-fonts</artifactId>
74            <version>6.20.6</version>
75        </dependency>
76
77    </dependencies>
```

INTERPRETACION

Cada forma respondió a una necesidad distinta. Sin <version>, Maven tomó la versión del padre, de modo que una actualización de Spring Boot habría actualizado todas las librerías de forma coherente.

JasperReports no perteneció al conjunto administrado por Spring Boot, así que su versión se fijó a mano en 6.20.6, y jasperreports-fonts usó la misma para que ambas fueran compatibles.

Figura 23 : Plugin de construcción del proyecto en el pom.xml

```
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
```

INTERPRETACION

Este plugin se declaró sin versión ni configuración, porque el padre ya las definía. Cumplió dos funciones: habilitó el goal spring-boot:run para ejecutar la aplicación desde Maven y, durante package, convirtió el .jar común en uno ejecutable que incluyó las dependencias y el servidor Tomcat embebido, conservando el original con la extensión .original. Sin este plugin, package habría generado un .jar que no se habría podido ejecutar por sí solo con java -jar.

7.4.3. Dependencias utilizadas

Tabla *40*

Dependencias principales utilizadas en el proyecto

Dependencia	Propósito dentro del proyecto
Spring Web (spring-boot-starter-web)	Permitió desarrollar los controladores y las rutas web (/bandas, /albumes, /reportes/...) y aportó el servidor Tomcat embebido en el que corre la aplicación.
Spring Data JPA (spring-boot-starter-data-jpa)	Facilitó la persistencia mediante los repositorios IBanda, IAlbum, ICancion e IUsuario, con métodos derivados, consultas

	@Query y soporte para @Transactional.
Hibernate (incluido en Spring Data JPA)	Implementó el mapeo objeto-relacional de Banda, Album, Cancion y Usuario, con sus relaciones y cascadas. No se declaró por separado: llega como dependencia transitiva del starter.
MySQL Connector/J (mysql-connector-j, runtime)	Permitió la conexión con la base de datos bd_melodicvault.
Thymeleaf (spring-boot-starter-thymeleaf)	Permitió desarrollar las vistas dinamicas HTML integradas con Spring.
JasperReports (jasperreports 6.20.6)	Permitió compilar las plantillas .jrxml y generar los dos reportes PDF: canciones de un álbum y álbumes de una banda.
JasperReports Fonts (jasperreports-fonts 6.20.6)	Aportó las fuentes necesarias para exportar a PDF
Spring Security (spring-boot-starter-security)	Implementó el inicio de sesión por formulario, los roles ADMIN y LECTOR, la protección de rutas y el cifrado de contraseñas con BCrypt.
Thymeleaf Extras Spring Security 6 (thymeleaf-extras-springsecurity6)	Habilitó sec:authorize en las plantillas para mostrar u ocultar botones según la sesión y el rol.
Validation (spring-boot-starter-validation)	Permitió validar los formularios con anotaciones (@NotBlank, @Size, @Pattern, entre otras) y @Valid.
Spring Boot Starter Test (test)	Dependencia de pruebas incluida en el proyecto; no se implementaron clases de

	prueba.

INTERPRETACION

La tabla relacionó cada dependencia con una funcionalidad implementada: persistencia, vistas, seguridad por roles, validación de formularios y reportes PDF. Todas las dependencias, salvo spring-boot-starter-test, sostuvieron una función que el sistema usó. Hibernate se documentó dentro de Spring Data JPA porque el pom.xml no lo declaró de forma independiente.

Se usó la serie 6.x de JasperReports porque fue estable con Spring Boot 3.3.4 y Java 17; la serie 7.x cambió la estructura de módulos y el formato del .jrxml. Bootstrap y Bootstrap Icons no se agregaron como dependencias Maven: se cargaron desde un CDN.

7.4.4. Uso del ciclo de vida Maven

Los goals se ejecutaron desde Eclipse con Run As → Maven build…, indicando en el campo Goals el goal correspondiente y como directorio base ${project_loc:MelodicVault}. Se ejecutaron en orden: validate, compile, package e install

El goal test no se ejecutó de forma explícita porque el proyecto no tiene pruebas

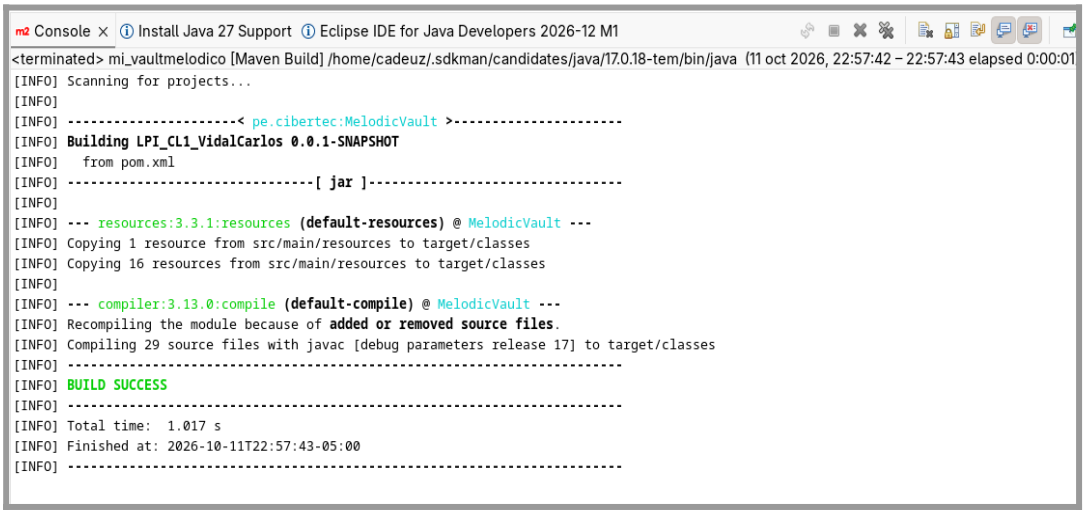
Tabla *41*

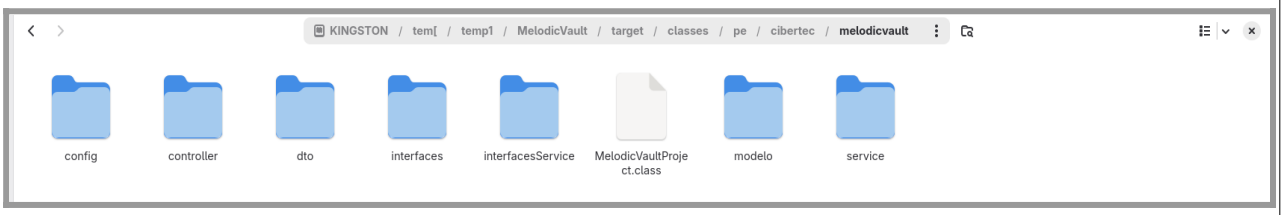
Goals del ciclo de vida Maven ejecutados en el proyecto

Goal	Resultado en el proyecto
validate	Verificó que el pom.xml y la estructura del proyecto fueran correctos.
compile	Compiló las clases de src/main/java y copió los recursos a target/classes.
package	Generó el .jar ejecutable de la aplicación en target.
install	Copió el .jar y el pom.xml al repositorio local de Maven (~/.m2).

La tabla mostró que cada goal agregó un resultado verificable: proyecto válido, clases compiladas, .jar generado y artefacto instalado

Figura 24 : Ejecución del goal compile y clases generadas





INTERPRETACION

mvn compile compiló el código fuente y verificó errores de compilación. Se ejecutó con éxito:

Maven copió los recursos de src/main/resources y compiló los 29 archivos fuente con javac para Java 17, convirtiendo los archivos .java en .class, ubicados en target/classes.

Figura 25 : Ejecución del goal package y artefacto generado





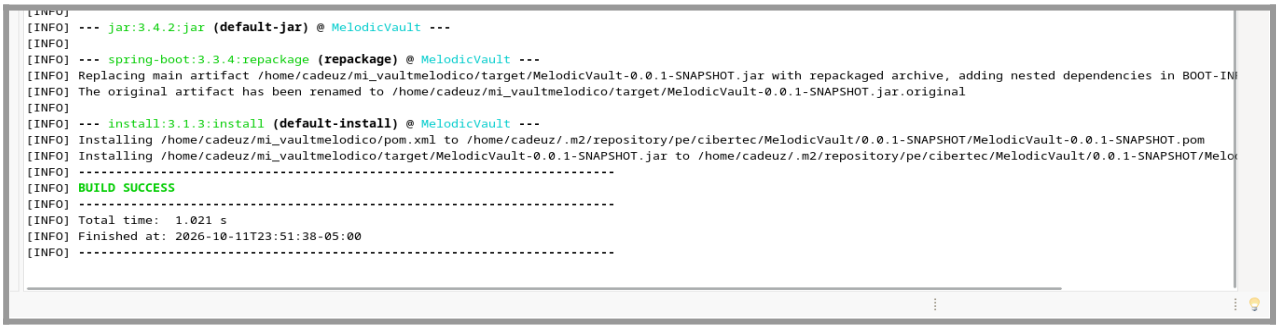
INTERPRETACION

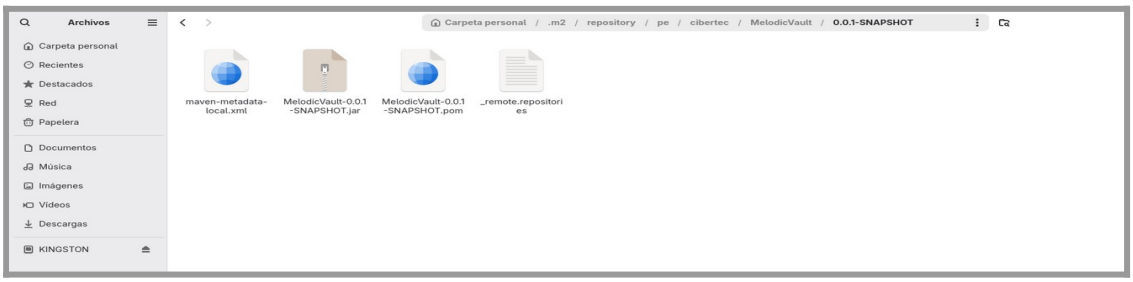
mvn package compiló el proyecto, ejecutó las fases anteriores del ciclo de vida y empaquetó el resultado en un archivo distribuible.

Se ejecutó con éxito: el spring-boot-maven-plugin reempaquetó el .jar incorporando todas las dependencias, y renombró el .jar original (sin dependencias) con la extensión .jar.original.

Como resultado, en la carpeta target se generó MelodicVault-0.0.1-SNAPSHOT.jar, un archivo ejecutable que contuvo la aplicación completa junto con el servidor Tomcat embebido, por lo que pudo ejecutarse en cualquier equipo con Java 17 usando java -jar.

Figura 26 : Ejecución del goal install y artefacto en el repositorio local





INTERPRETACION

La figura mostró el goal install y la carpeta del repositorio local donde quedó el artefacto. La ruta pe/cibertec/MelodicVault/0.0.1-SNAPSHOT reprodujo el groupId, el artifactId y la versión del pom.xml, y contuvo el .jar y el .pom instalados.

7.5. Control de versiones con Git y GitHub

Durante el desarrollo del sistema web Melodic Vault, se emplearon Git y GitHub como herramientas para gestionar el control de versiones, mantener un historial de modificaciones y facilitar la integración de los aportes realizados por los integrantes del equipo.

Git permitió registrar los cambios efectuados sobre el código fuente y la documentación técnica mediante confirmaciones (commits), mientras que GitHub proporcionó

un repositorio remoto centralizado para compartir, consultar y sincronizar las diferentes versiones del proyecto.

Repositorio remoto. El código y la documentación del proyecto están en GitHub: https://github.com/Garay-Dev/MelodicVault

7.5.1. Creación del repositorio local

La implementación del control de versiones comenzó con la inicialización de un repositorio Git dentro del directorio principal del proyecto Melodic Vault. Para ello, se utilizó el comando git init, cuya ejecución permitió establecer la estructura necesaria para registrar y administrar las modificaciones del proyecto.

7.5.1.1. Comando: git init

Figura 27 : Ejecución del goal install y artefacto en el repositorio local



INTERPRETACION

La ejecución del comando git init permitió convertir el directorio del proyecto en un repositorio Git mediante la creación de la carpeta interna .git. Esta carpeta almacena la información necesaria para administrar las referencias, configuraciones e historial de versiones del proyecto.

La evidencia muestra el mensaje de inicialización satisfactoria del repositorio, confirmando que Git quedó habilitado para realizar el seguimiento de los archivos del

sistema.Este procedimiento constituyó el punto de partida para incorporar prácticas de control de versiones al desarrollo de Melodic Vault, permitiendo que las modificaciones posteriores pudieran registrarse, identificarse y recuperarse mediante el historial de Git.

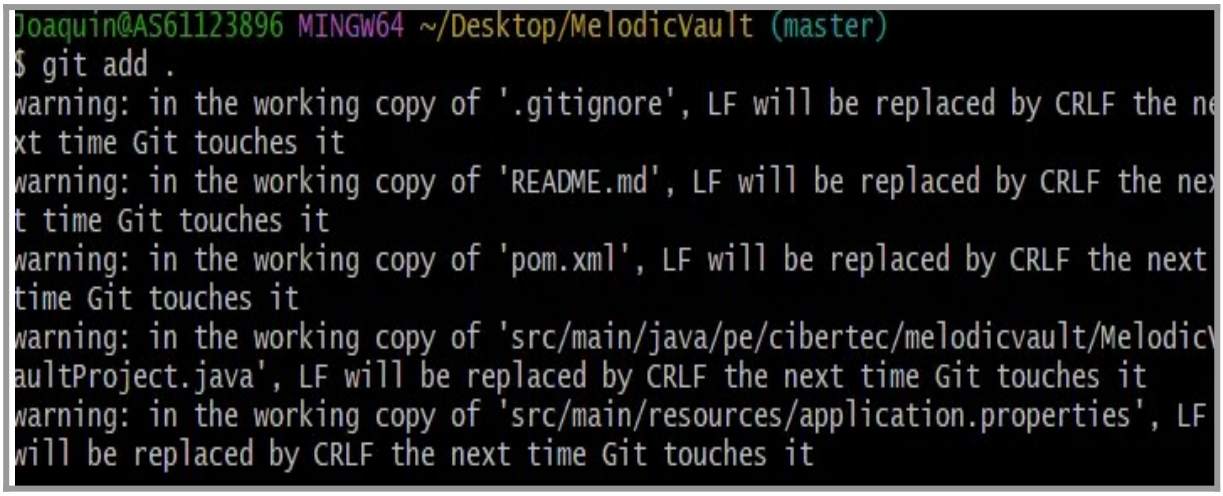
7.5.2. Preparación de archivos mediantes Git Add

Una vez inicializado el repositorio local, se preparó los archivos del proyecto Melodic Vault para su incorporación al sistema de control de versiones. Se utilizó el comando git add cuya finalidad es agregar al área de preparación (Staging Area) los cambios detectados dentro del directorio actual y sus subdirectorios.

Este procedimiento permitió preparar los archivos correspondientes a la estructura inicial del sistema, incluyendo componentes del backend desarrollados en Java, archivos de configuración de Spring Boot, dependencias Maven y documentación del proyecto.

7.5.2.1. Comando: git add .

Figura 28 : Preparación inicial de archivos del proyecto mediante el comando git add.



INTERPRETACION

La ejecución del comando git add . es fundamental dentro del proceso de control de versiones, debido a que permite seleccionar y preparar los cambios que posteriormente serán registrados mediante una confirmación (commit).

En la evidencia presentada se identifican archivos como README.md, pom.xml, application.properties y componentes Java ubicados dentro de la estructura src/main/java. Estos elementos forman parte de la configuración, documentación y código fuente del sistema Melodic Vault.

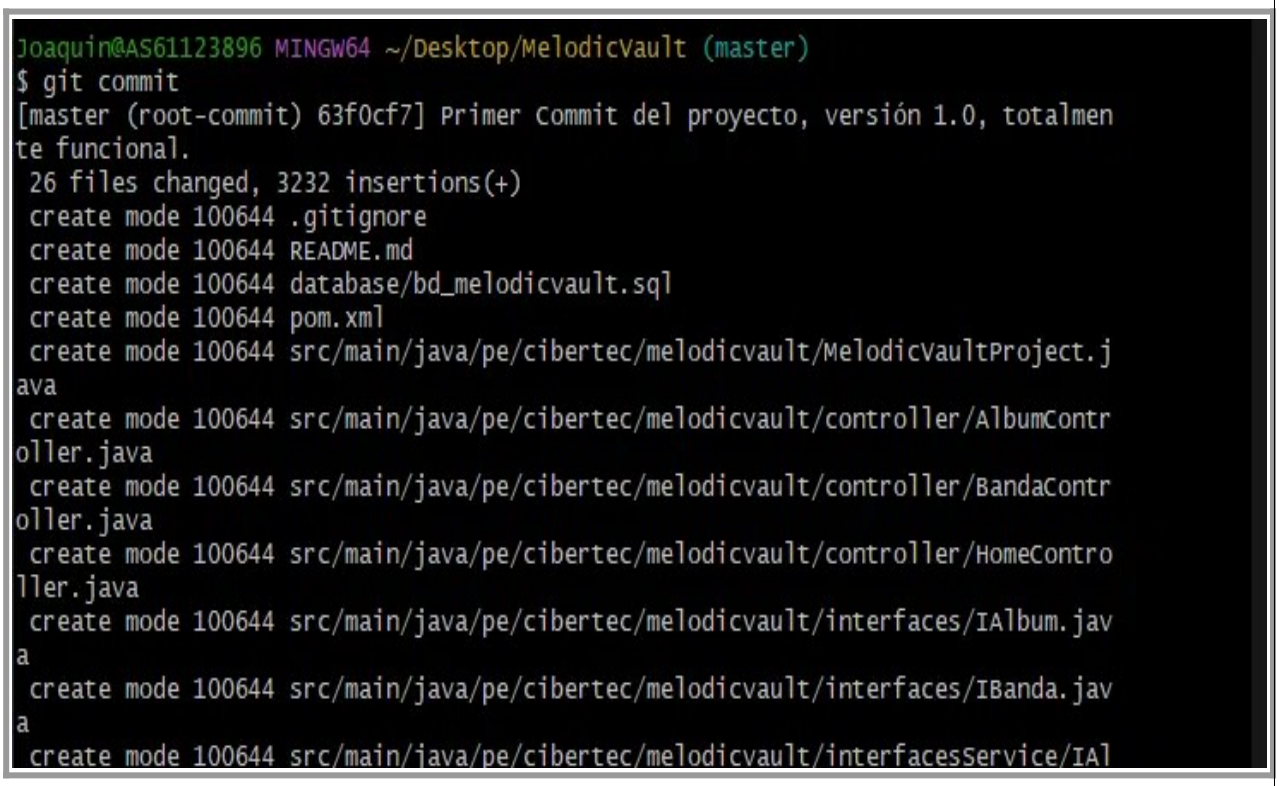
Durante la ejecución se generaron advertencias relacionadas con los formatos de finalización de línea LF (Line Feed) y CRLF (Carriage Return + Line Feed). Estas advertencias corresponden a diferencias en las convenciones utilizadas por distintos sistemas operativos y a la configuración de normalización de finales de línea de Git.

7.5.3. Registro de la primera versión del proyecto mediante Git Commit

Después de preparar los archivos mediante el comando git add ., se procedió a registrar la primera versión del sistema web Melodic Vault en el repositorio local. Para ello, se utilizó el comando git commit, que permite generar un punto de control dentro del historial de versiones, conservando una referencia identificable del estado de los archivos preparados.

Comando utilizado: git commit -m "Primer Commit del proyecto, versión 1.0, totalmente funcional."

Figura 29 : Registro del primer commit del proyecto mediante Git Bash



INTERPRETACION

La ejecución del comando git commit permitió registrar formalmente la primera versión del proyecto dentro del historial local de Git. Como resultado, se generó el identificador abreviado 63f0cf7, correspondiente al commit inicial, que constituye una referencia para identificar y consultar el estado del proyecto en ese momento.

La salida de la terminal muestra que se incorporaron 26 archivos, con un total de 3232 líneas añadidas. Entre los elementos registrados se identifican archivos de configuración, dependencias Maven, clases Java, controladores, interfaces y scripts relacionados con la base de datos.

El proyecto había comenzado a desarrollarse antes de que el equipo incorporara GitHub. En particular, Carlos Vidal Merino había trabajado previamente en la estructura y funcionalidades iniciales de la aplicación, que posteriormente fueron incorporadas al

repositorio por el equipo. Esta situación explica el volumen de archivos y líneas registradas en la primera confirmación, y que ahí comprendimos la importancia de GitHub y de ir subiendo cada versión.

7.5.4. Vinculación del repositorio local con el repositorio remoto

Una vez registrada la primera versión mediante Git, se estableció la vinculación entre el repositorio local y el repositorio remoto alojado en GitHub. Esta configuración permitió preparar el entorno para la publicación y posterior sincronización de los cambios realizados durante el desarrollo del proyecto.

Para establecer dicha conexión, se utilizó el comando git remote add origin, especificando la dirección URL del repositorio remoto creado para Melodic Vault.

7.5.4.1. Comando: git remote add origin

https://github.com/Garay-Dev/MelodicVault.git

Figura 30 : Configuración del repositorio remoto mediante Git Bash



INTERPRETACION

Git almacena esta configuración en el archivo .git/config, permitiendo que los comandos posteriores de sincronización utilicen el nombre origin en lugar de escribir nuevamente la dirección completa del repositorio.

La evidencia muestra la configuración de la dirección https://github.com/Garay-Dev/MelodicVault.git, correspondiente al repositorio destinado a centralizar el código fuente y la documentación técnica del sistema Melodic Vault.

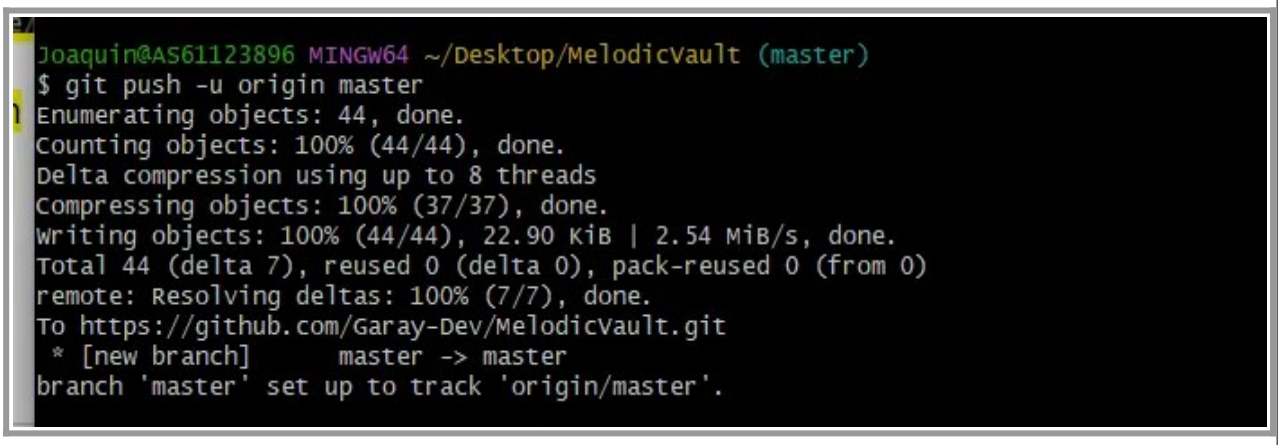
Este procedimiento no realiza automáticamente la transferencia de archivos ni publica los commits locales. Su función consiste exclusivamente en registrar la ubicación del repositorio remoto, estableciendo la configuración necesaria para ejecutar posteriormente operaciones como git push, git fetch y git pull.

7.5.5. Primera publicación del proyecto en GitHub

Después de configurar la vinculación entre el repositorio local y el repositorio remoto, se procedió a realizar la primera publicación del sistema web en GitHub. Esta actividad tuvo como finalidad transferir las versiones registradas localmente hacia una plataforma centralizada, permitiendo que los integrantes del equipo pudieran acceder al código fuente, consultar el historial de modificaciones y colaborar en las siguientes etapas de desarrollo.

7.5.5.1. Comando: git push -u origin master

Figura 31 : Primera publicación del repositorio en GitHub mediante Git Bash



INTERPRETACION

El comando git push permite transferir los objetos de Git necesarios para actualizar las referencias del repositorio remoto, incluyendo los commits, árboles y contenidos asociados que todavía no se encuentren disponibles en el destino, permitiendo que el repositorio remoto incorpore el historial publicado.

El parámetro -u, equivalente a --set-upstream, permite configurar el seguimiento de la rama local respecto a su correspondiente rama remota. Esta asociación facilita las operaciones posteriores de sincronización, ya que Git puede identificar la rama de referencia configurada para determinados comandos de consulta, recuperación y publicación.

El comando utilizado corresponde a la publicación inicial de la rama denominada master. La utilización posterior de otras ramas, como main, desarrollo y hotfix, forma parte de la evolución del repositorio y será analizada en el apartado correspondiente a la gestión de ramas.

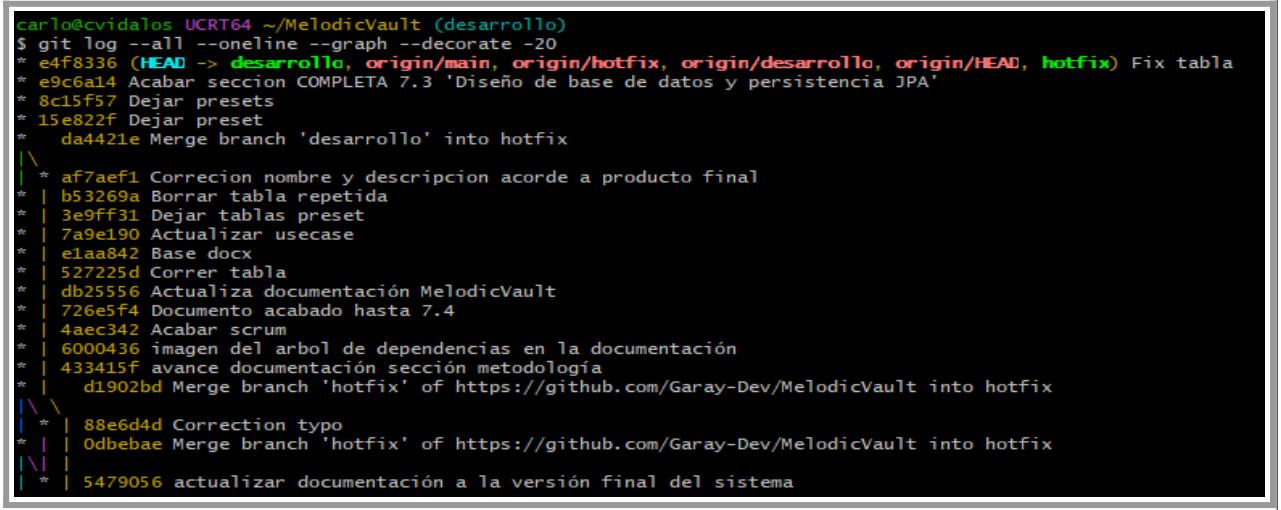
De esta manera, el equipo incorporó un mecanismo de colaboración que facilitó la consulta del historial, la sincronización de versiones y la posterior integración de los aportes realizados durante el desarrollo del sistema web.

7.5.6. Gestión de ramas

La gestión de ramas permitió trabajar con distintas líneas de desarrollo y realizar operaciones de integración sin depender exclusivamente de una única rama para registrar todas las modificaciones del proyecto.

7.5.6.1. Comando: git log --all --oneline --graph --decorate -20

Figura 32 : Visualización gráfica del historial de commits y relaciones entre ramas del proyecto Melodic Vault

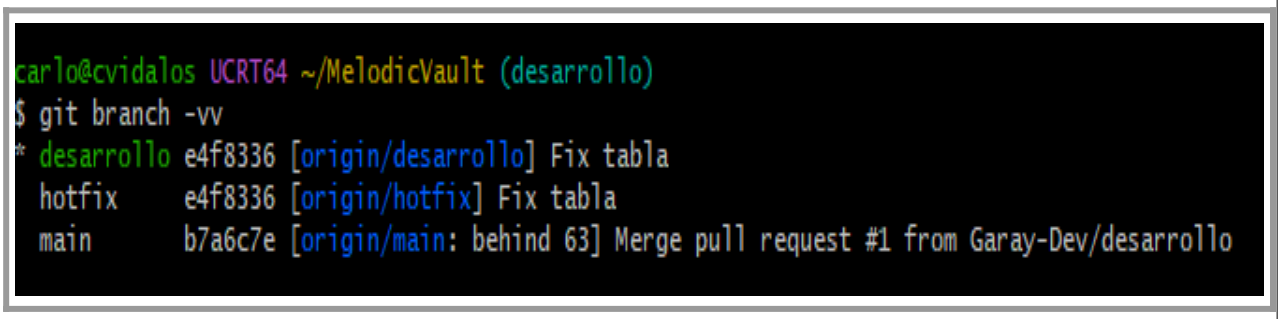


INTERPRETACION

En la evidencia se observa que el commit e4f8336, identificado mediante el mensaje “Fix tabla”, constituye el punto al que apuntan las referencias desarrollo, hotfix, origin/desarrollo y origin/hotfix.

7.5.6.2. Comando: git branch -vv

Figura 33 : Identificación y estado de las ramas locales del repositorio Melodic Vault



INTERPRETACION

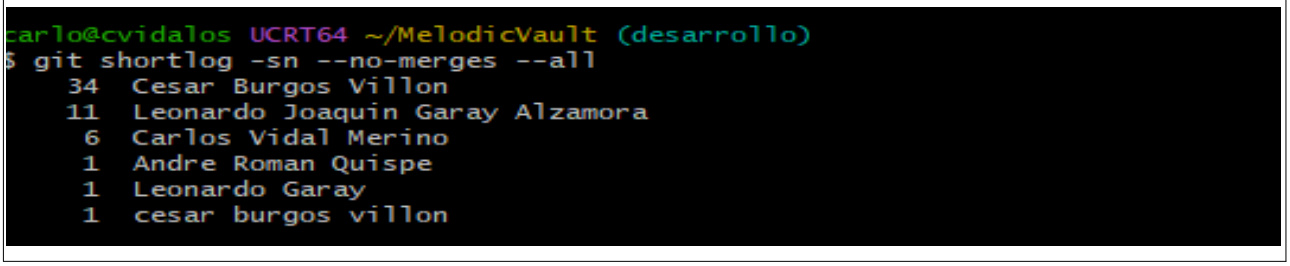
Ramas utilizadas.

- main : version estable
- desarrollo : iteracion codigo
- hotfix: documentación (requisitos, matriz de CU, diagramas, BPMN y el documento Word).
- docs/andre-casos-uso: rama temporal de un integrante para completar reglas de negocio y casos de uso. Se eliminó después de integrarla.

7.5.7. Historial y participación de los integrantes

7.5.7.1. Comando: git shortlog -sn --no-merges –all

Figura 34 : Distribución de commits registrados por autor en el repositorio Melodic Vault



INTERPRETACION

El resultado monstruo un total de 54 commits , distribuidas inicialmente entre seis identidades de autor.

Sin embargo, se identificaron diferencias en la forma de registrar los nombres de algunos integrantes.

En particular, César Burgos Villón aparece con dos variantes de escritura, al igual que Leonardo Garay Alzamora se producieron, por una discrepancia temporal que hubo entre definición local y global de username

Carlos Vidal aparece con solo 6 commits aunque desarrolló la base inicial del proyecto, porque esa base se subió en un único commit consolidado al inicializar el repositorio.

Fue algo que ocurrio una única vez en el proyecto, ya que GitHub se enseñó tarde en el curso; desde entonces cada aporte quedó registrado por integrante.

Gracias a esto aprendimos que usar GitHub desde el inicio permite segregar las ediciones por autor y conservar la trazabilidad del aporte real de cada integrante.

7.5.7.2. Comando: git log --all --no-merges --format='%ae| %an' | awk -F'|' '{e=tolower($1); c[e]++; if(!(e in n)) n[e]=$2} END{for(e in c) print c[e], n[e]}' | sort -nr | awk '{v[NR]=$1; l[NR]=$0; if($1>m)m=$1} END{for(i=1;i<=NR;i++){b=""; for(j=0;j<int(v[i]*40/m+0.5);j++) b=b"#"; printf "%-38s %s\n", l[i], b}}'

Figura 35 : Distribución de commits por integrante mediante la normalización de identidades de autor

```
carlo@cvidalos UCRT64 ~/MelodicVault (desarrollo)
$ git log --all --no-merges --format='%ae|%an' | awk -F'|' '{e=tolower($1); c[e]++
; if(!(e in n)) n[e]=$2} END{for(e in c) print c[e], n[e]}' | sort -nr | awk '{v[N
R]=$1; l[NR]=$0; if($1>m)m=$1} END{for(i=1;i<=NR;i++){b=""; for(j=0;j<int(v[i]*40/
m+0.5);j++) b=b"#"; printf "%-38s %s\n", l[i], b}}'
35 Cesar Burgos Villon                 ########################################
12 Leonardo Joaquin Garay Alzamora     ##############
6 Carlos Vidal Merino                  #######
1 Andre Roman Quispe                   #
```

INTERPRETACION

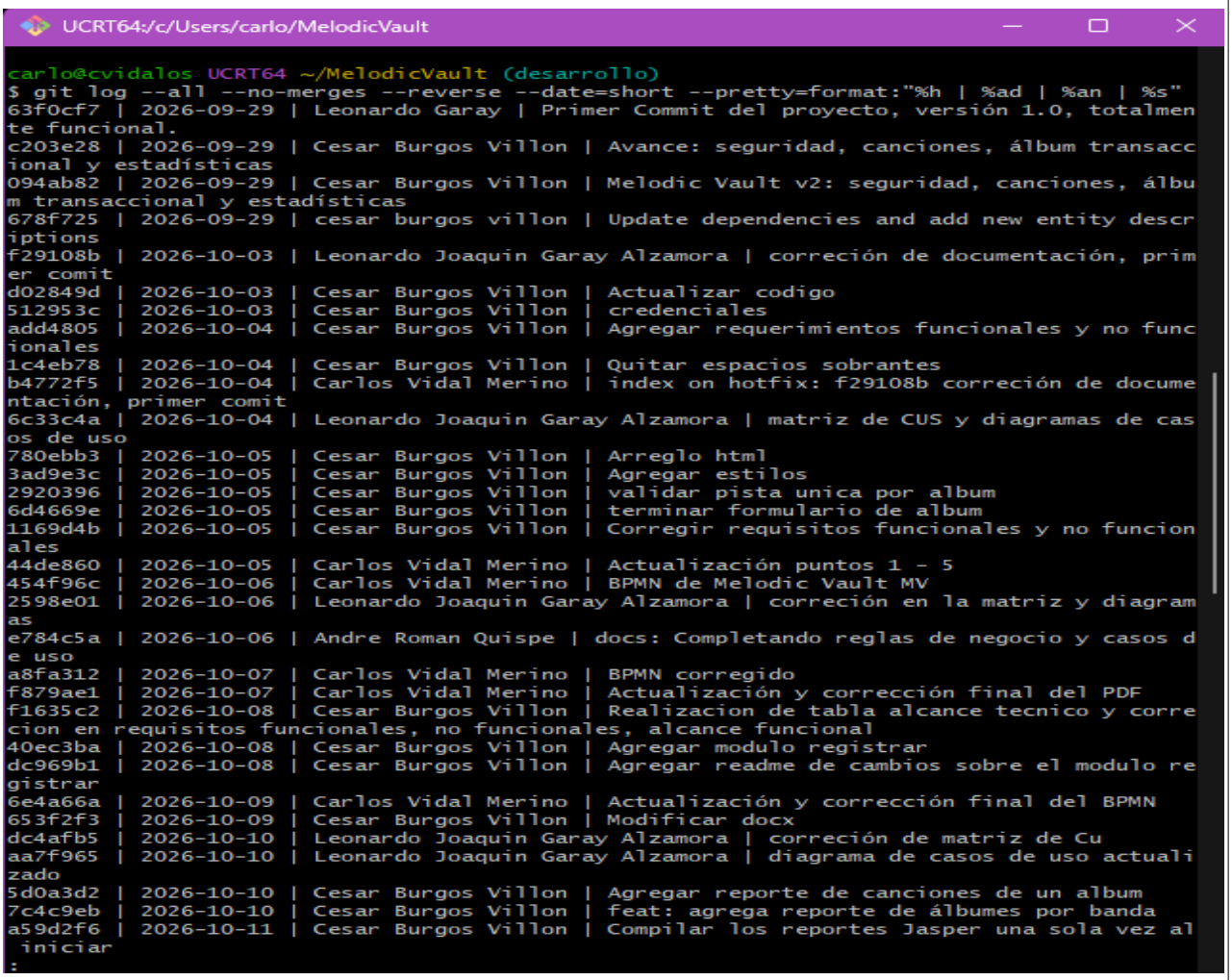
La ejecución del comando facilitó analizar cuantitativamente las confirmaciones registradas por los integrantes del equipo de desarrollo de Melodic Vault, utilizando el correo electrónico de cada autor como criterio de agrupación.

Al comparar esta evidencia con el resultado del git shortlog se observa que inicialmente aparecían seis identidades de autor, debido a diferencias en la forma de registrar los nombres de César Burgos Villón y Leonardo Garay Alzamora.

Mediante la agrupación por correo electrónico fue posible consolidar dichas variantes y obtener una representación correspondiente a cuatro integrantes, manteniendo el total de 54 confirmaciones.

7.5.7.3. Comando: log --all --no-merges --reverse --date=short --pretty=format:"%h | %ad | %an | %s"

Figura 36 : Historial cronológico de commits del proyecto Melodic Vault



INTERPRETACION

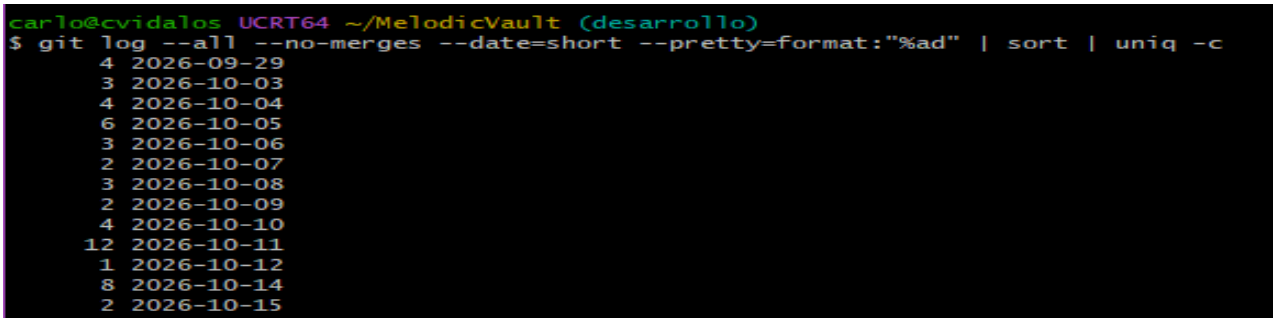
En la evidencia se identifica como primer registro el commit 63f0cf7, fechado el 29 de septiembre de 2026, con el mensaje “Primer Commit del proyecto, versión 1.0, totalmente funcional”.

Es importante señalar que Melodic Vault ya contaba con una base funcional desarrollada antes de que el equipo comenzara a utilizar GitHub sistemáticamente. Por lo tanto, la fecha del primer commit no representa necesariamente el inicio real del desarrollo del proyecto, sino el momento en que se registró su primera versión dentro del repositorio.

Posteriormente, durante el 29 de septiembre, se observan confirmaciones relacionadas con seguridad, canciones, operaciones transaccionales de álbumes, estadísticas y actualización de dependencias. Estos mensajes permiten identificar actividades orientadas a ampliar y ajustar componentes de la aplicación.

7.5.7.4. Comando: log --all --no-merges --date=short --pretty=format:"%ad" | sort | uniq -c

Figura 37 : Distribución cronológica de commits registrados por día en el repositorio Melodic Vault



INTERPRETACION

Esta distribución permite identificar que las confirmaciones no se concentraron exclusivamente en una única jornada, sino que fueron registradas en diferentes momentos del desarrollo del proyecto.

Esto permite demostrar que el repositorio fue utilizado para registrar modificaciones sucesivas y no únicamente para almacenar una versión final del proyecto.

7.5.8. Integración de cambios

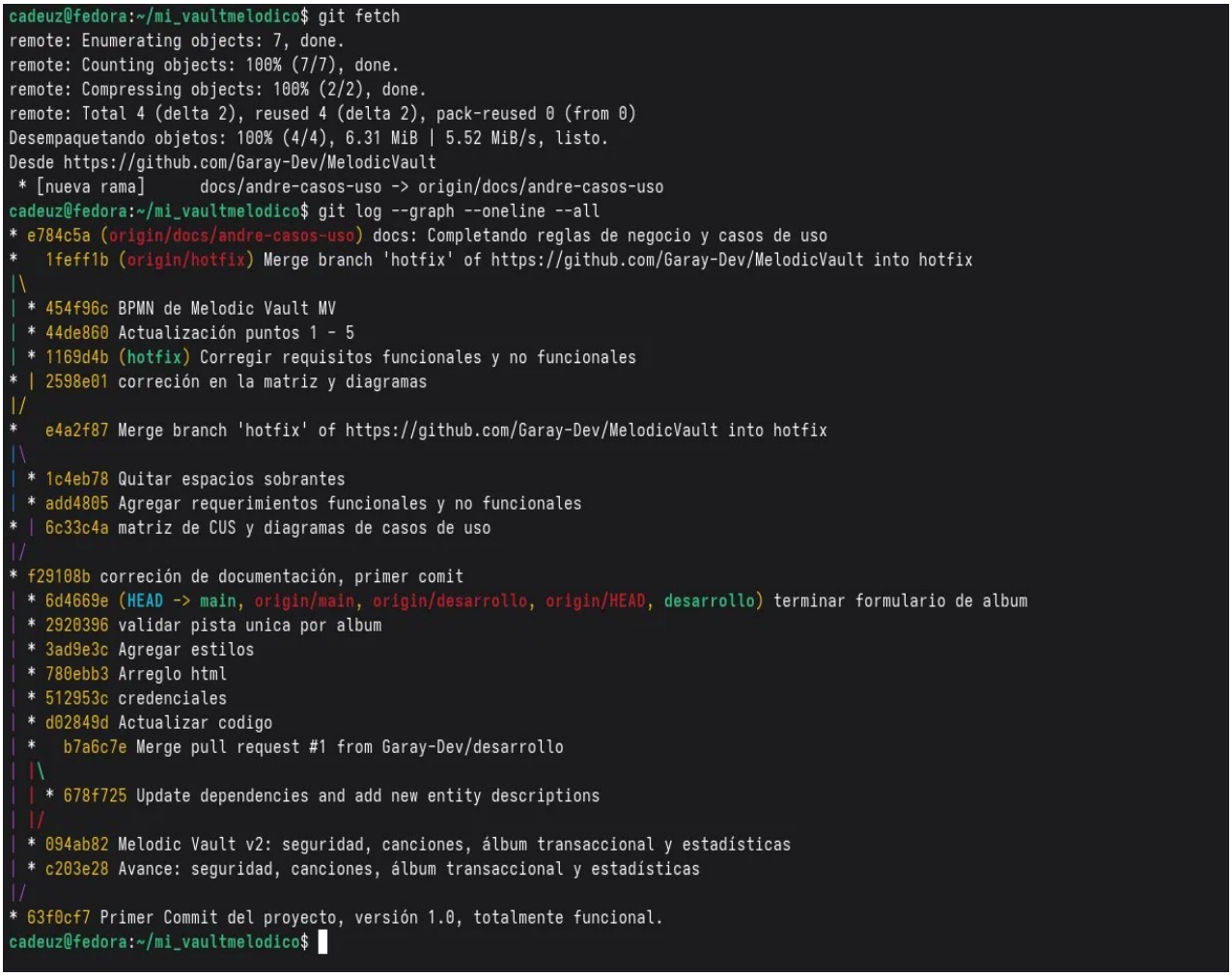
Durante el desarrollo colaborativo del sistema web Melodic Vault, se realizaron operaciones de integración y sincronización para incorporar los aportes de los integrantes al repositorio compartido.

Uno de los casos documentados correspondió a la incorporación del trabajo realizado en la rama temporal docs/andre-casos-uso, utilizada para desarrollar documentación relacionada con las reglas de negocio y los casos de uso del sistema. Estos cambios debían integrarse en la rama hotfix, donde el equipo mantenía avances relacionados con la documentación del proyecto.

Para realizar este procedimiento, se utilizaron diferentes comandos de Git

7.5.8.1. Comando: fetch

Figura 38 : Recuperación de referencias remotas mediante Git Fetch



INTERPRETACION

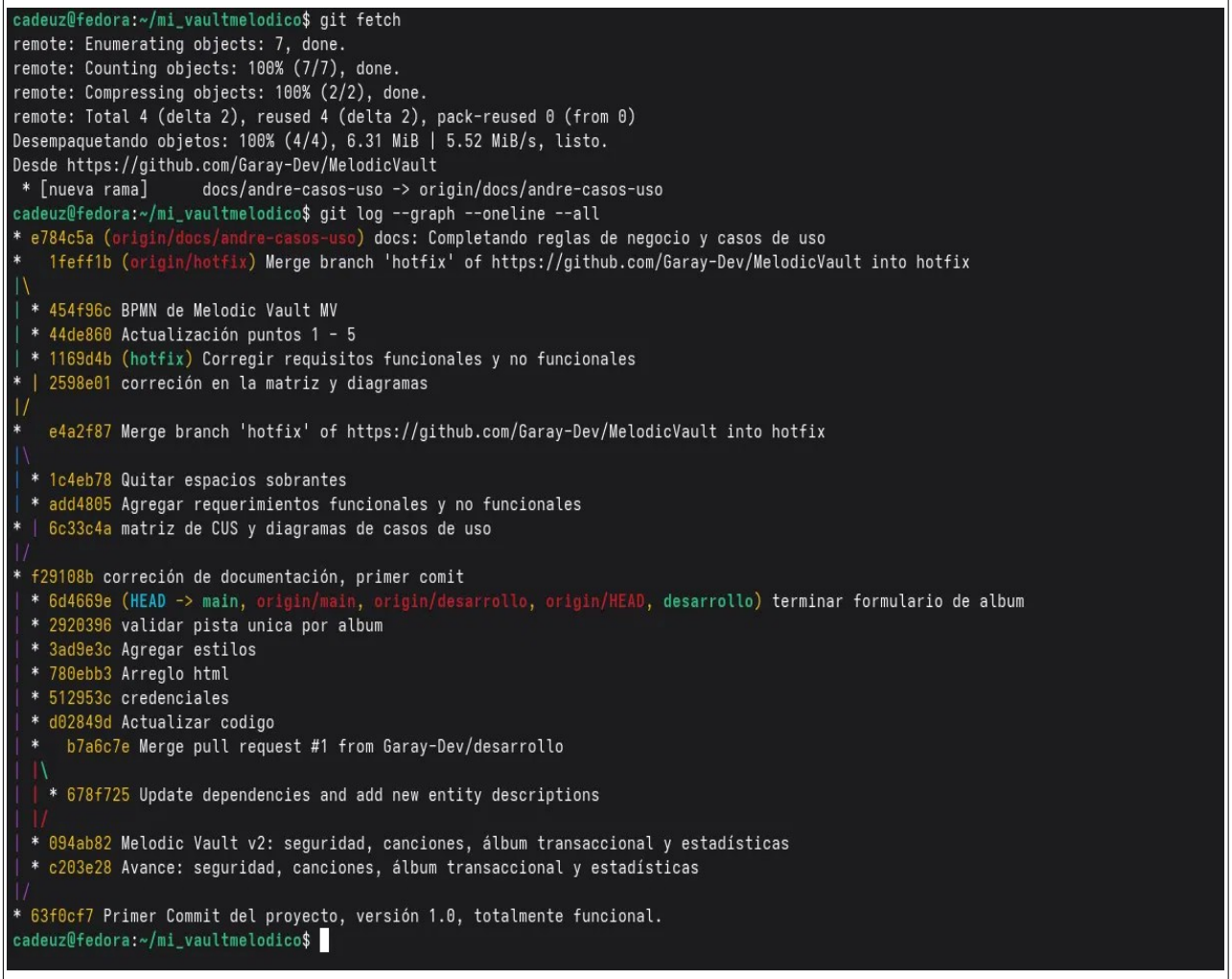
La ejecución de git fetch permitió recuperar la información disponible en el repositorio remoto, incluyendo las referencias necesarias para identificar el trabajo desarrollado en docs/andre-casos-uso.

Esta operación consultó los cambios remotos sin incorporarlos automáticamente a la rama activa ni modificar los archivos del directorio de trabajo. Es decir, este comando descarga la rama nueva (origin/docs/andre-casos-uso) sin tocar el trabajo local.

De esta manera, el equipo pudo revisar previamente el estado del repositorio y determinar cómo realizar la integración.

7.5.8.2. Comando: git log --graph --oneline –all

Figura 39 : Verificación del historial de commits y la rama temporal



INTERPRETACION

La visualización del historial facilitó identificar la relación entre los commits de la rama temporal y la rama de destino. Según la evidencia histórica, el commit e784c5a se encontraba en una línea de desarrollo que permitía su incorporación mediante avance rápido.

Esta revisión fue importante porque permitió determinar que la integración podía realizarse actualizando la referencia de la rama de destino, sin necesidad de generar un commit adicional de fusión.

7.5.8.3. Comando: switch hotfix + merge --ff-only origin/hotfix

Figura 40 : Actualización de la rama Hotfix e integración del trabajo del integrante



INTERPRETACION

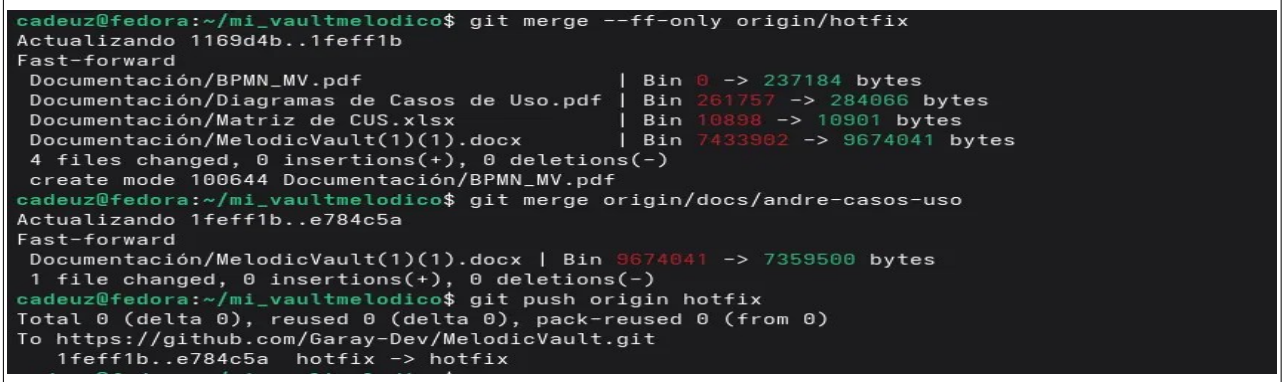
Durante esta operación, Git informó que la rama local se encontraba cuatro commits por detrás de origin/hotfix y que podía actualizarse mediante avance rápido (fast-forward).

Para resolver esta diferencia se ejecutó git merge --ff-only origin/hotfix, obteniéndose el resultado Fast-forward, con una actualización desde el commit 1169d4b hasta 1feff1b.

El resultado evidencia que fue posible actualizar la rama local sin generar un commit de fusión adicional, debido a que su historial podía avanzar directamente hasta el commit de la referencia remota.

7.5.8.4. Comando: merge origin/docs/andre-casos-uso

Figura 41 : Integración del trabajo desarrollado en docs/andre-casos-uso



INTERPRETACION

El resultado muestra una actualización desde el commit 1feff1b hasta e784c5a, acompañada nuevamente del mensaje Fast-forward.

Esta información permite determinar que el commit correspondiente al trabajo documental del integrante se encontraba en una trayectoria compatible con la rama de destino. Por ello, Git pudo actualizar directamente la referencia de hotfix, conservando el historial existente y sin generar un commit adicional de fusión.

7.5.8.5. Comando: push origin hotfix

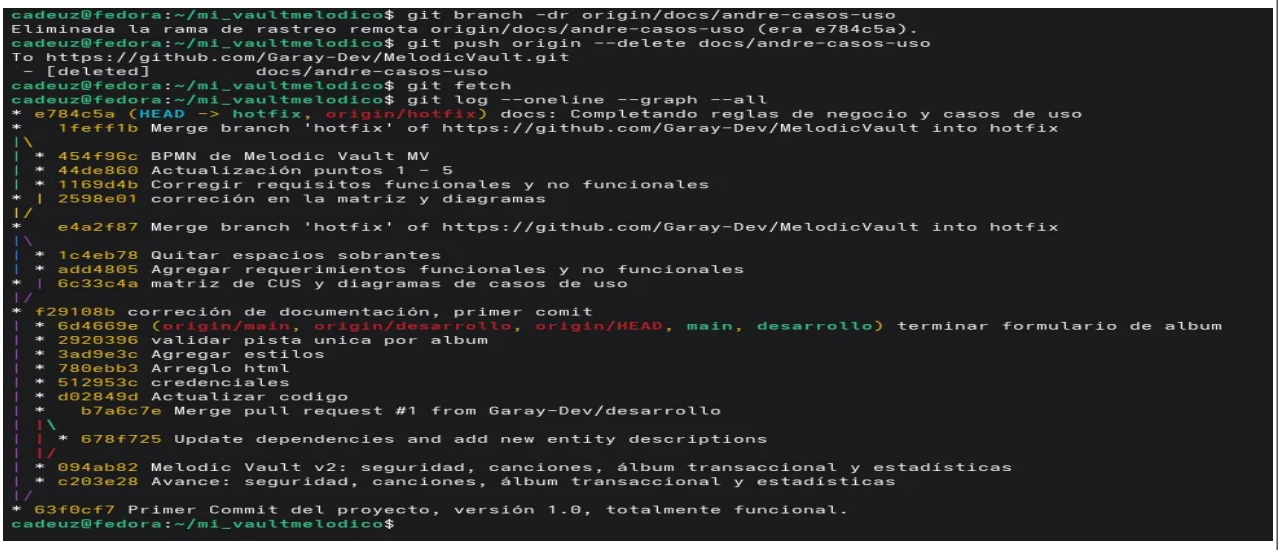
Figura 42 : Integración del trabajo en el repositorio remoto de GitHub

```
cadeuz@fedora:~/mi_vaultmelodico$ git merge --ff-only origin/hotfix
Actualizando 1169d4b..1feff1b
Fast-forward
 Documentación/BPMN_MV.pdf                   | Bin 0 -> 237184 bytes
 Documentación/Diagramas de Casos de Uso.pdf | Bin 261757 -> 284066 bytes
 Documentación/Matriz de CUS.xlsx            | Bin 10898 -> 10901 bytes
 Documentación/MelodicVault(1)(1).docx       | Bin 7433902 -> 9674041 bytes
 4 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Documentación/BPMN_MV.pdf
cadeuz@fedora:~/mi_vaultmelodico$ git merge origin/docs/andre-casos-uso
Actualizando 1feff1b..e784c5a
Fast-forward
 Documentación/MelodicVault(1)(1).docx | Bin 9674041 -> 7359500 bytes
 1 file changed, 0 insertions(+), 0 deletions(-)
cadeuz@fedora:~/mi_vaultmelodico$ git push origin hotfix
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Garay-Dev/MelodicVault.git
   1feff1b..e784c5a  hotfix -> hotfix
```

INTERPRETACION

Una vez incorporadas las modificaciones documentales en la rama local hotfix, se ejecutó el comando git push origin hotfix, con el propósito de publicar el resultado de la integración en el repositorio remoto de GitHub.

Figura 43 Eliminación de la rama temporal y verificación del historial



INTERPRETACION

La última evidencia presenta las actividades de limpieza y verificación realizadas después de integrar el trabajo documental del integrante.

Este resultado permite verificar que la rama local y su referencia remota quedaron alineadas en el mismo commit al finalizar el procedimiento.

La eliminación de ramas temporales después de su integración constituye una práctica de organización que contribuye a mantener un repositorio más claro y administrable.

La verificación posterior del historial permitió comprobar que el aporte del integrante permaneció registrado, que las referencias de la rama de destino coincidían y que el trabajo anterior continuaba disponible.

7.5.9. Resultado y aprendizaje

Integrar cambios no es solo ejecutar un merge. Es un procedimiento ordenado: recuperar referencias, revisar el historial, actualizar la rama destino, incorporar contribuciones, publicar y verificar.

7.6. Implementación web con Spring Boot, MVC, JPA e Hibernate

La implementación web de MelodicVault se desarrolló con Spring Boot siguiendo el patrón Model View Controller (MVC): las entidades JPA representan los datos, las vistas Thymeleaf los presentan y los controladores coordinan ambos.

Los datos se almacenaron en MySQL y se accedió a ellos mediante repositorios de Spring Data JPA, con Hibernate como implementación del mapeo objeto-relacional.

Para explicar la implementación se tomó como ejemplo el módulo de bandas; los módulos de álbumes y canciones siguieron la misma estructura.

7.6.1. Arquitectura del sistema

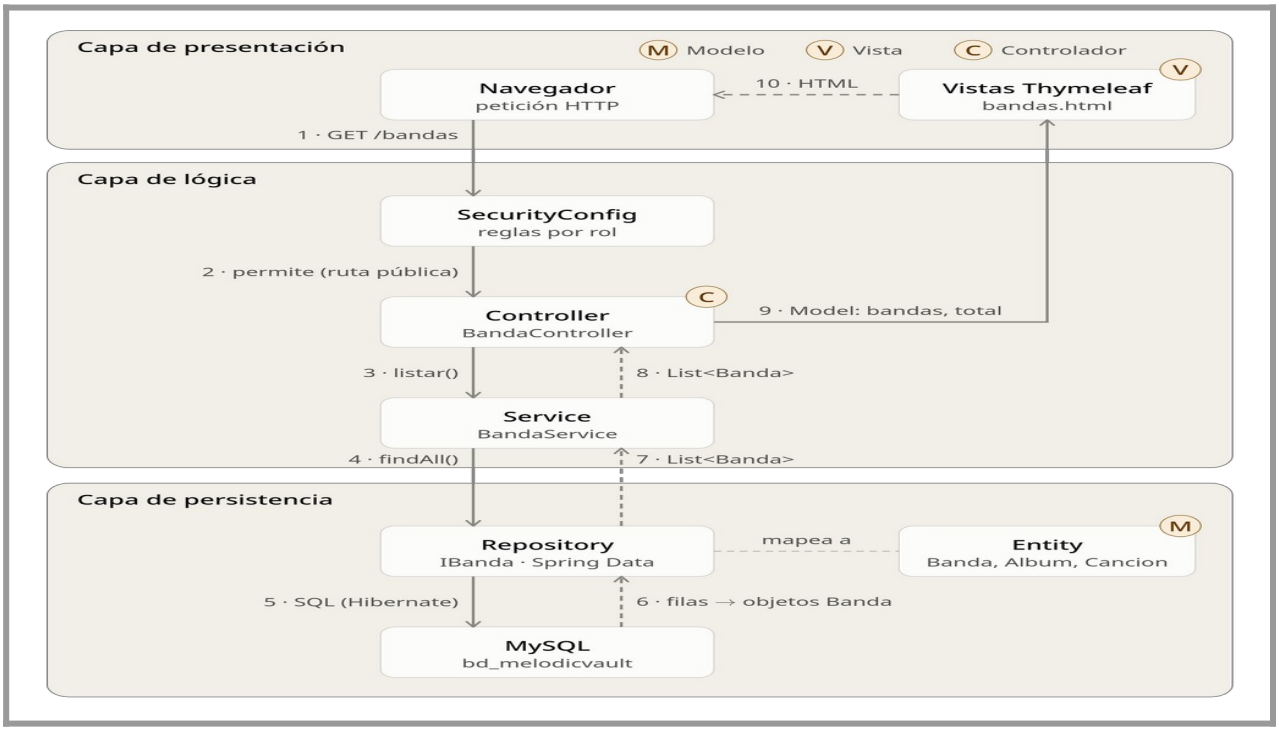
7.6.1.1. Arquitectura de tres capas y patrón MVC

La aplicación se organizó en tres capas.

La capa de presentación quedó formada por las vistas Thymeleaf; la capa de lógica, por los controladores y los servicios; y la capa de persistencia, por los repositorios, las entidades y la base de datos MySQL.

Sobre esa división se aplicó el patrón MVC: el Model lo constituyeron las entidades del paquete modelo, la View las plantillas de templates y el Controller las clases del paquete controller. Los servicios no corresponden a ninguna letra de MVC: se ubicaron entre el controlador y los repositorios para que el controlador no accediera a la base de datos directamente.

Figura 44 :Arquitectura en tres capas y patrón MVC de MelodicVault



INTERPRETACION

La figura muestra el recorrido completo de una petición a /bandas, numerado en diez pasos. La línea continua representa la ida: el navegador envió la petición (1), SecurityConfig verificó que la ruta fuera pública (2) y BandaController pidió los datos a BandaService (3), que los solicitó al repositorio IBanda (4); Hibernate tradujo esa llamada a una consulta SQL sobre MySQL (5).

La línea punteada representa la vuelta: las filas regresaron al repositorio convertidas en objetos Banda (6) y subieron como List<Banda> a través del servicio (7) hasta el controlador (8), que los entregó a la vista mediante el Model (9); Thymeleaf generó el HTML y lo devolvió al navegador (10), con lo que el ciclo quedó cerrado.

En el módulo de bandas, cada capa solo se comunicó con la inmediata: la vista nunca consultó la base de datos y el controlador nunca usó el repositorio directamente, sino a través del servicio.

Las etiquetas M, V y C indican qué elemento cumplió cada rol del patrón MVC: la entidad Banda como Model, las plantillas Thymeleaf como View y BandaController como Controller.

7.6.1.2. Componentes de la arquitectura

Tabla *42*

Componentes principales de la arquitectura del sistema

Componente	Ubicación en el proyecto	Función	Capa - Rol MVC
Entity	paquete modelo	Representa las tablas banda, album, cancion y usuario, con sus validaciones y relaciones.	Persistencia - Model
Repository	paquete interfaces	Gestiona las operaciones de persistencia contra MySQL; Spring Data genera su implementación.	Persistencia
Contrato de servicio	paquete interfacesService	Define las operaciones disponibles sobre cada entidad.	Lógica
Service	paquete service	Implementa los contratos apoyándose en los repositorios y agrupa las operaciones transaccionales.	Lógica
Controller	paquete controller	Atiende las rutas, prepara los datos con Model y decide qué vista o redirección devolver.	Lógica - Controller
View	src/main/ resources/ templates	Presenta las 13 interfaces en Thymeleaf y oculta botones según el rol.	Presentación - View
Configuración de seguridad	paquete config (SecurityConfig)	Define qué rutas son públicas, cuáles exigen sesión y cuáles solo ADMIN.	Lógica (transversal)»

application.properties	src/main/resources	Configura el puerto, la conexión a MySQL y el comportamiento de JPA e Hibernate.	Transversal

INTERPRETACION

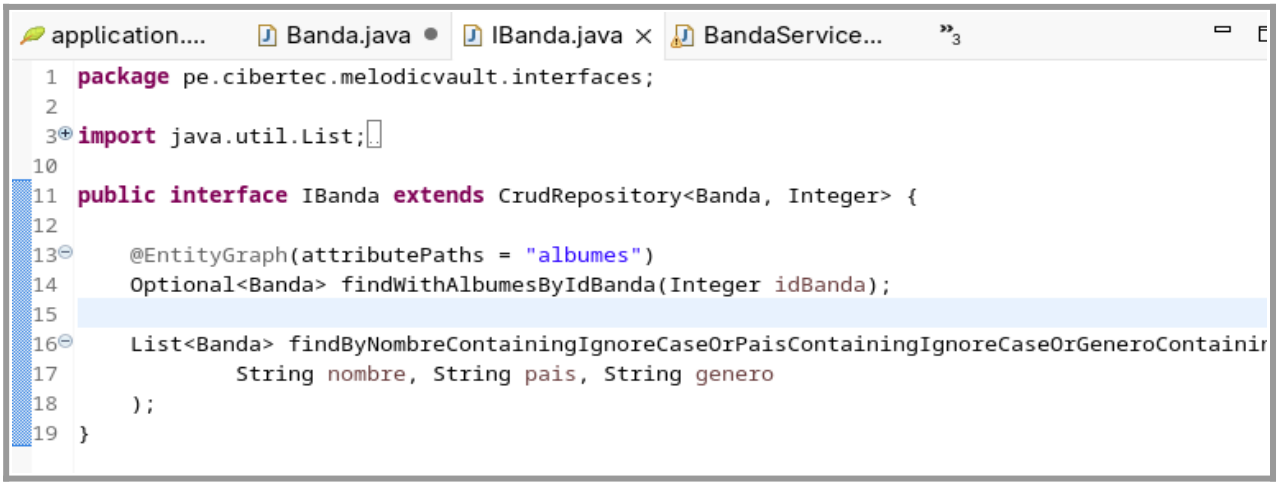
La tabla muestra que cada componente funcional pertenece a una sola capa y que el patrón MVC se reparte entre ellas: el Model son las entidades, la View son las plantillas y el Controller vive en la capa de lógica.

Los repositorios, los contratos y los servicios no tienen letra en MVC: son las piezas intermedias que mantienen al controlador con logica desacoplada.

SecurityConfig pertenece a la capa de lógica, pero actúa de forma transversal sobre toda la aplicación, igual que application.properties

7.6.1.3. Comunicación entre los componentes: módulo de bandas

Figura 45 :Codigo - iBanda



INTERPRETACION

Al extender CrudRepository<Banda, Integer>, IBanda heredó findAll, findById, save y deleteById sin escribir SQL.

El método derivado findBy...ContainingIgnoreCase se tradujo, a partir de su nombre, en una búsqueda parcial sin distinguir mayúsculas sobre nombre, país o género, y @EntityGraph cargó los álbumes junto con la banda en una sola consulta.

Figura 46 :Contrato IBandaService e implementación BandaService

```java
public interface IBandaService {

	List<Banda> listar();

	Optional<Banda> listarId(int id);

	int save(Banda b);

	void delete(int id);

	List<Banda> buscar(String texto);

	Optional<Banda> listarConAlbumes(int id);

}
```

```java
public class BandaService implements IBandaService {

    @Autowired
    private IBanda data;

    @Override
    public List<Banda> listar() {
        return (List<Banda>) data.findAll();
    }

    @Override
    public Optional<Banda> listarId(int id) {
        return data.findById(id);
    }

    @Override
    public int save(Banda b) {
        int res = 0;
        Banda banda = data.save(b);

        if (banda != null) {
            res = 1;
        }

        return res;
    }

    @Override
    public void delete(int id) {
        data.deleteById(id);
    }

    @Override
    public List<Banda> buscar(String texto) {
        return data.findByNombreContainingIgnoreCaseOrPaisContainingIgno
                texto, texto, texto
        );
    }

    @Override
    public Optional<Banda> listarConAlbumes(int id) {
        return data.findWithAlbumesByIdBanda(id);
    }
}
```

INTERPRETACION

La interfaz IBandaService definió el contrato, es decir, qué operaciones existen sobre las bandas, y BandaService definió cómo se cumplen, apoyándose en IBanda para hablar con la base de datos.

Como el controlador depende solo del contrato, la implementación podría cambiar sin tocar los controladores.

En buscar, el único texto del buscador se pasó tres veces al repositorio, lo que permitió que una sola caja filtrara por nombre, país o género.

Figura 47 :Controlador BandaController

```
@GetMapping({"", "/"})
public String listar(@RequestParam(value = "buscar", required = false) String buscar, Model model) {

    if (buscar != null && !buscar.trim().isEmpty()) {
        model.addAttribute("bandas", service.buscar(buscar));
        model.addAttribute("total", service.buscar(buscar).size());
        model.addAttribute("buscar", buscar);
    } else {
        model.addAttribute("bandas", service.listar());
        model.addAttribute("total", service.listar().size());
    }

    return "bandas";
}

@GetMapping("/nuevo")
public String nuevo(Model model) {
    model.addAttribute("banda", new Banda());
    model.addAttribute("modo", "registrar");
    return "form-banda";
}

@GetMapping("/editar/{id}")
public String editar(@PathVariable("id") int id, Model model, RedirectAttributes flash) {
    Optional<Banda> banda = service.listarId(id);

    if (banda.isEmpty()) {
        flash.addFlashAttribute("error", "La banda solicitada no existe.");
        return "redirect:/bandas";
    }

    model.addAttribute("banda", banda.get());
    model.addAttribute("modo", "editar");
    return "form-banda";
}

@PostMapping("/guardar")
public String guardar(@Valid Banda banda, BindingResult result, Model model, RedirectAttributes flash) {

    if (result.hasErrors()) {
        model.addAttribute("modo", banda.getIdBanda() == null ? "registrar" : "editar");
        return "form-banda";
    }
```

INTERPRETACION

BandaController actuó como despachador: @RequestMapping("/bandas") fijó la ruta base y cada método atendió una ruta distinta.

Los métodos GET (listar, nuevo) solo prepararon datos con model.addAttribute y devolvieron el nombre de la vista, que Thymeleaf resolvió a templates/bandas.html o form-banda.html.

El método POST guardar recibió el formulario como un objeto Banda, lo validó con @Valid y, si había errores, volvió al formulario con los mensajes; si no, delegó en service.save y respondió con redirect:/bandas para que recargar la página no repitiera el envío.

El controlador no contiene código de persistencia: todo acceso a datos pasó por IBandaService.

Figura 48 :Vista bandas.html

```
<article class="banda-catalogo" th:each="b : ${bandas}">
    <div class="banda-lista-img">
        <img th:if="${b.imagenUrl != null && !b.imagenUrl.isEmpty()}"
             th:src="${b.imagenUrl}"
             th:alt="${b.nombre}">

        <div class="banda-img-fallback"
             th:if="${b.imagenUrl == null || b.imagenUrl.isEmpty()}">
            <i class="bi bi-music-note-beamed"></i>
        </div>
    </div>

    <div class="banda-contenido">
        <span class="badge-genero" th:text="${b.genero}">Género</span>

        <h2 th:text="${b.nombre}">Nombre banda</h2>

        <p class="pais">
            <span th:text="${b.pais}">País</span>
            · Formada en
            <span th:text="${b.anioFormacion}">1980</span>
        </p>

        <p class="descripcion-card" th:text="${b.descripcion}">
            Descripción de la banda
        </p>

        <div class="acciones-card">
            <a th:href="@{/bandas/detalle/{id}(id=${b.idBanda})}" class="btn-secundario">
                Ver detalle
            </a>
         <th:block sec:authorize="hasRole('ADMIN')">
            <a th:href="@{/bandas/editar/{id}(id=${b.idBanda})}" class="btn-editar">
                Editar
            </a>

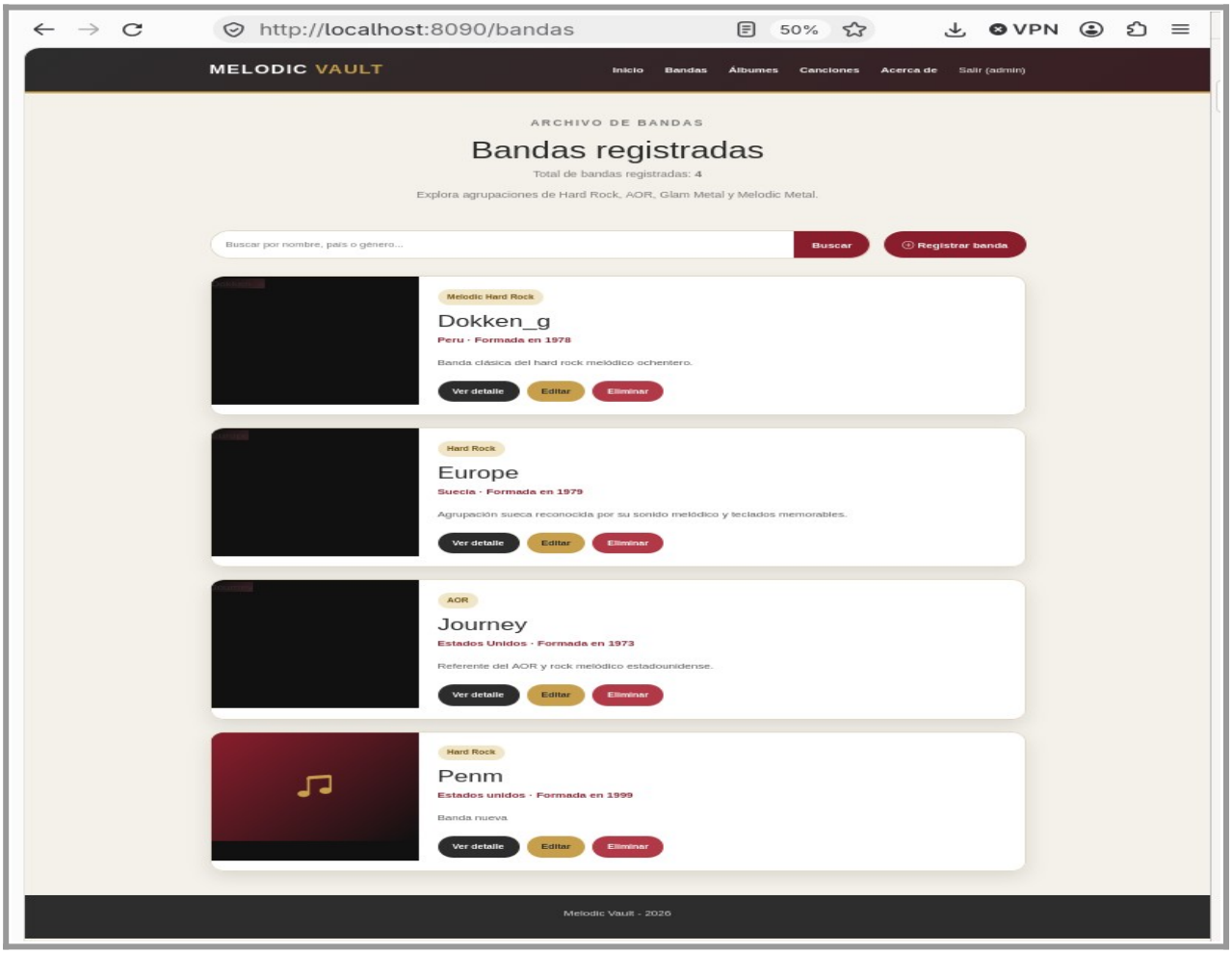
            <a th:href="@{/bandas/eliminar/{id}(id=${b.idBanda})}"
               class="btn-eliminar"
               onclick="return confirm('¿Seguro que deseas eliminar esta banda? También se eliminarán sus álbumes asociados.');">
                Eliminar
            </a>
         </th:block>
        </div>
    </div>
</article>
```

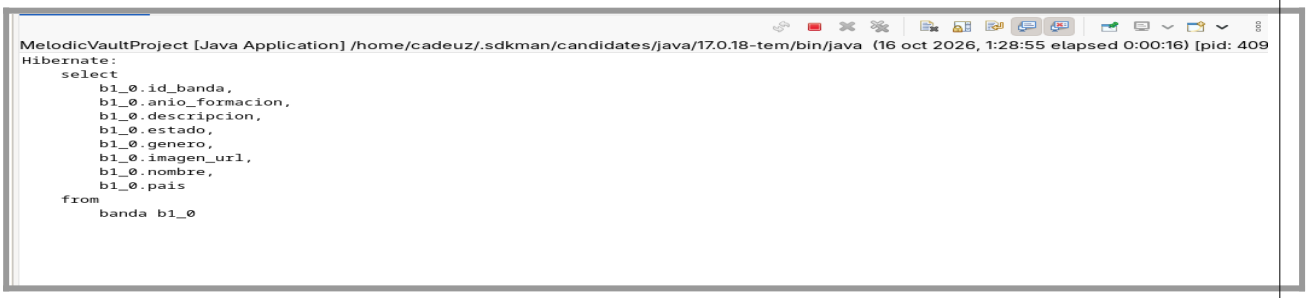
INTERPRETACION

th:each recorrió la lista bandas que el controlador puso en el Model, y th:text mostró los atributos de cada entidad mediante sus getters; así los datos de la tabla banda llegaron hasta la pantalla. th:href construyó las rutas que vuelven a BandaController (/bandas/detalle/{id}, /bandas/editar/{id}), con lo que se cerró el ciclo petición-respuesta.

Sec:authorize ocultó los botones de edición a quien no es Administrador, gracias a la dependencia thymeleaf-extras-springsecurity6 ; el bloqueo real lo hace SecurityConfig en el servidor.

Figura 49 :Listado de bandas en el navegador y consulta SQL generada por Hibernate





INTERPRETACION

La figura muestra el ciclo completo para /bandas: la página con las bandas leídas de MySQL y, en la consola, la consulta select sobre la tabla banda que Hibernate generó a partir de findAll, visible gracias a show-sql=true (Application properties).

Es la evidencia de que el controlador, el servicio, el repositorio y la vista estaban conectados con la base de datos.

7.6.2. **Gestión de sesiones y roles**

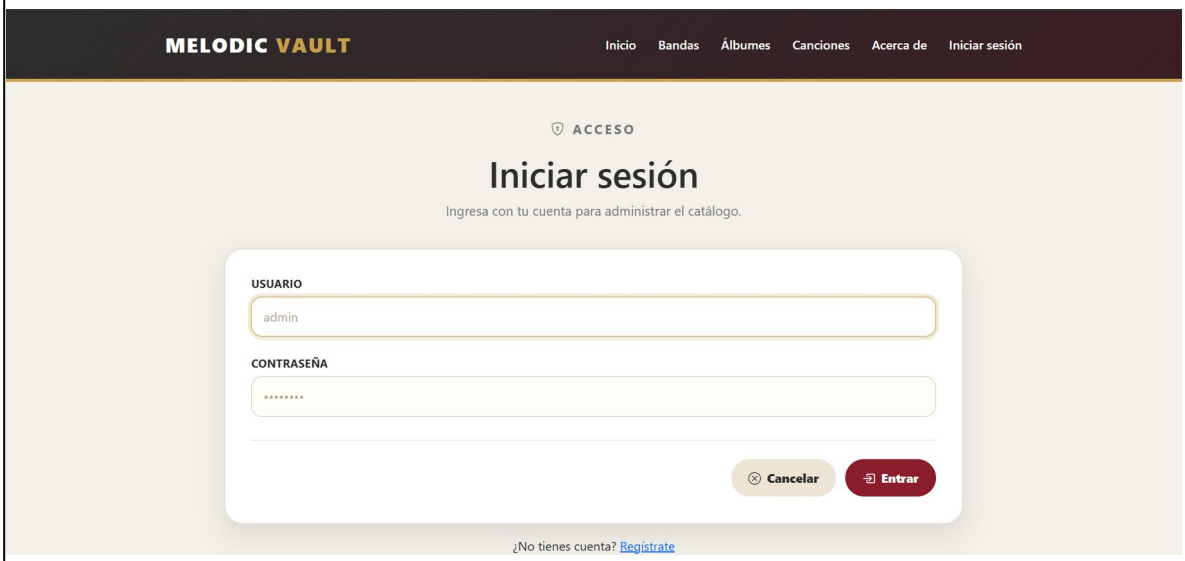
El sistema MelodicVault controló el acceso de los usuarios mediante Spring Security, integrado con Spring Boot y Spring MVC. La autenticación se implementó con una clase personalizada, UsuarioDetailsService, que implementó la interfaz UserDetailsService. Cuando un usuario intentaba iniciar sesión, esta clase consultaba la base de datos a través del repositorio IUsuario, buscaba al usuario por su nombre y, si existía, entregaba a Spring Security su username, su contraseña y su rol. Si el usuario no existía, se lanzaba una UsernameNotFoundException y el acceso era rechazado.

Las contraseñas nunca se almacenaron en texto plano: al registrarse, RegistroController las cifró con un PasswordEncoder antes de guardarlas, y Spring Security comparó de forma segura la contraseña ingresada con el hash almacenado.

La gestión de sesiones la realizó Spring Security: una vez autenticado, el usuario mantuvo una sesión activa en el servidor, identificada por una cookie, que conservó su identidad y sus permisos entre peticiones hasta que cerraba sesión o la sesión expiraba.

A continuación se presentaron las imágenes del proyecto.

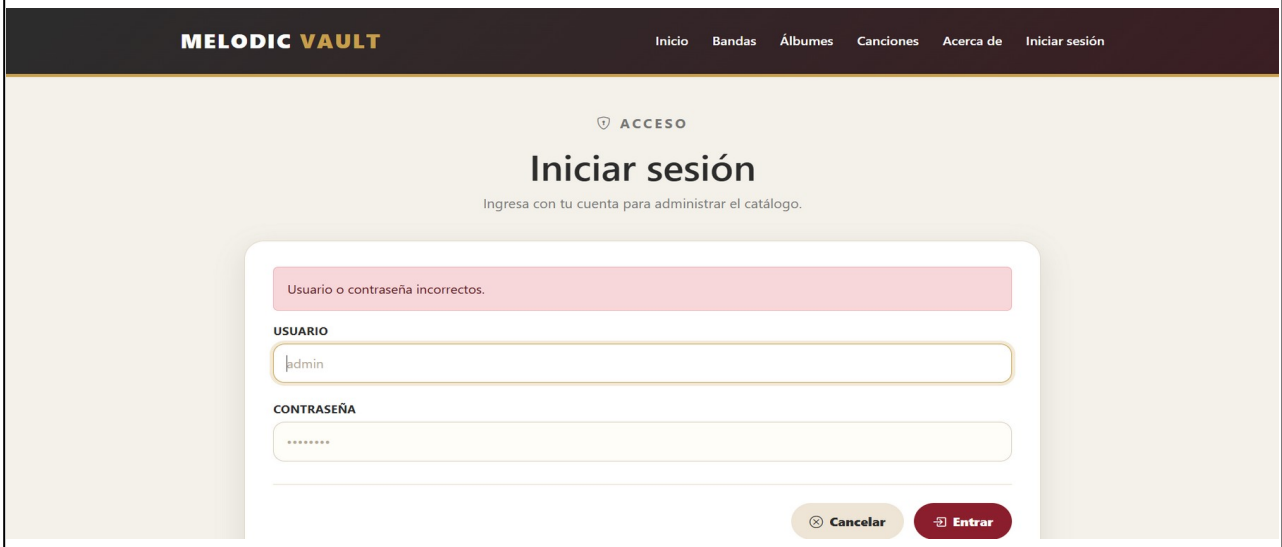
Figura 50: Formulario de Inicio de sesión



INTERPRETACION

En este formulario el usuario pudo iniciar sesión en el sistema mediante sus credenciales.

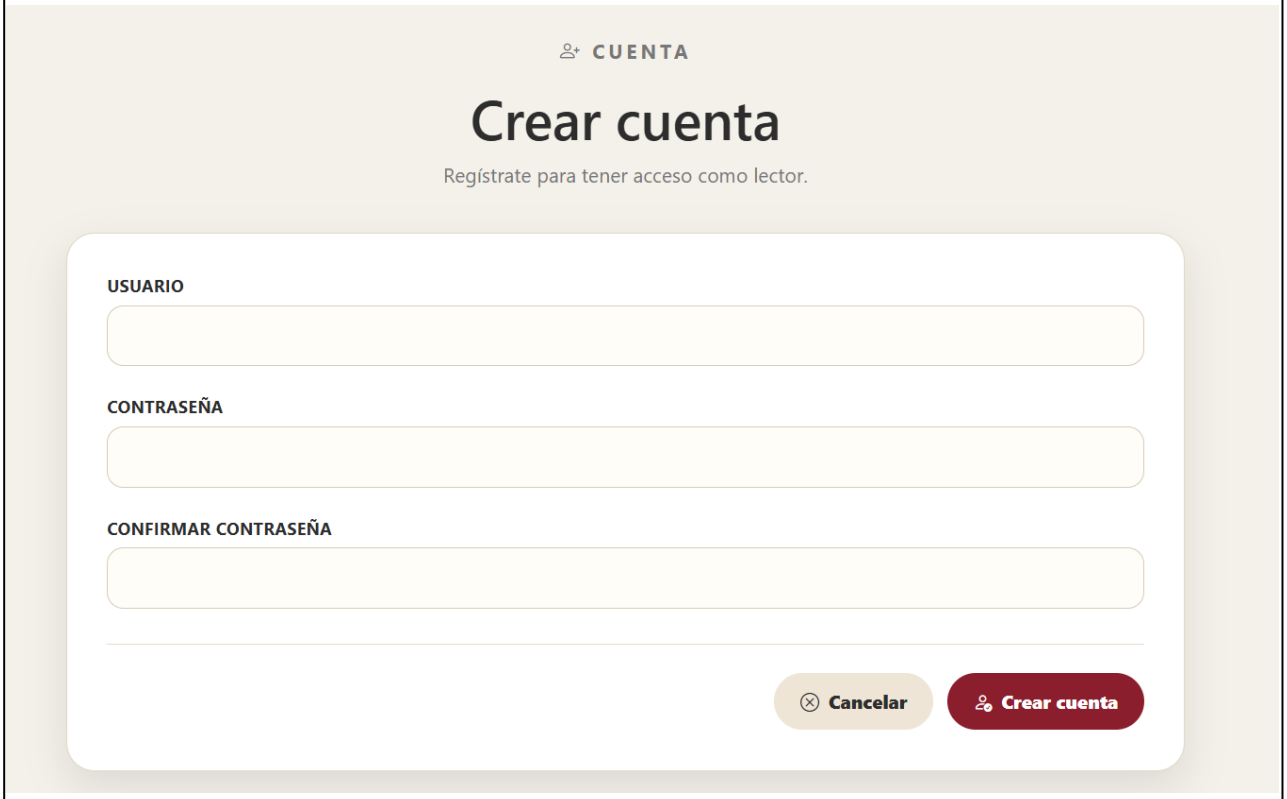
Figura 51: Error de datos al intentar Iniciar sesión



INTERPRETACION

En esta vista se evidenció cómo el sistema validó los datos enviados mediante el formulario para permitir el acceso y, al no coincidir con un usuario de la base de datos, no permitió el ingreso.

Figura 52: Formulario de registro



INTERPRETACION

Fue el formulario de registro de usuarios: se solicitó un nombre de usuario único y se pidió ingresar la contraseña dos veces para disminuir el riesgo de registrar una contraseña equivocada.

Figura 53: Error al registrar usuario Por usuario ya registrado



INTERPRETACION

En esta vista se evidenció cómo el sistema validó que el usuario no existiera en la base de datos.

Figura 54: Error al registrar usuario formato inválido



INTERPRETACION

En esta vista se evidenció cómo el sistema validó que el formato ingresado en los campos fuera correcto y devolvió la advertencia.

Figura 55: Dto de Registro

```
package pe.cibertec.melodicvault.dto;

import ...

public class RegistroForm {  3 usages  Cesar Burgos Villon
    @NotBlank(message = "El usuario es obligatorio")  2 usages
    @Size(min = 4, max = 50, message = "Entre 4 y 50 caracteres")
    @Pattern(regexp = "^[A-Za-z0-9_.-]*$", message = "Solo letras, números, punto, guion y guion bajo")
    private String username;

    @NotBlank(message = "La contraseña es obligatoria")  2 usages
    @Size(min = 6, max = 72, message = "Entre 6 y 72 caracteres")
    private String password;

    @NotBlank(message = "Confirma la contraseña")  2 usages
    private String confirmar;

    public String getUsername() { return username; }

    public void setUsername(String username) { this.username = username; }

    public String getPassword() { return password; }

    public void setPassword(String password) { this.password = password; }

    public String getConfirmar() { return confirmar; }

    public void setConfirmar(String confirmar) { this.confirmar = confirmar; }

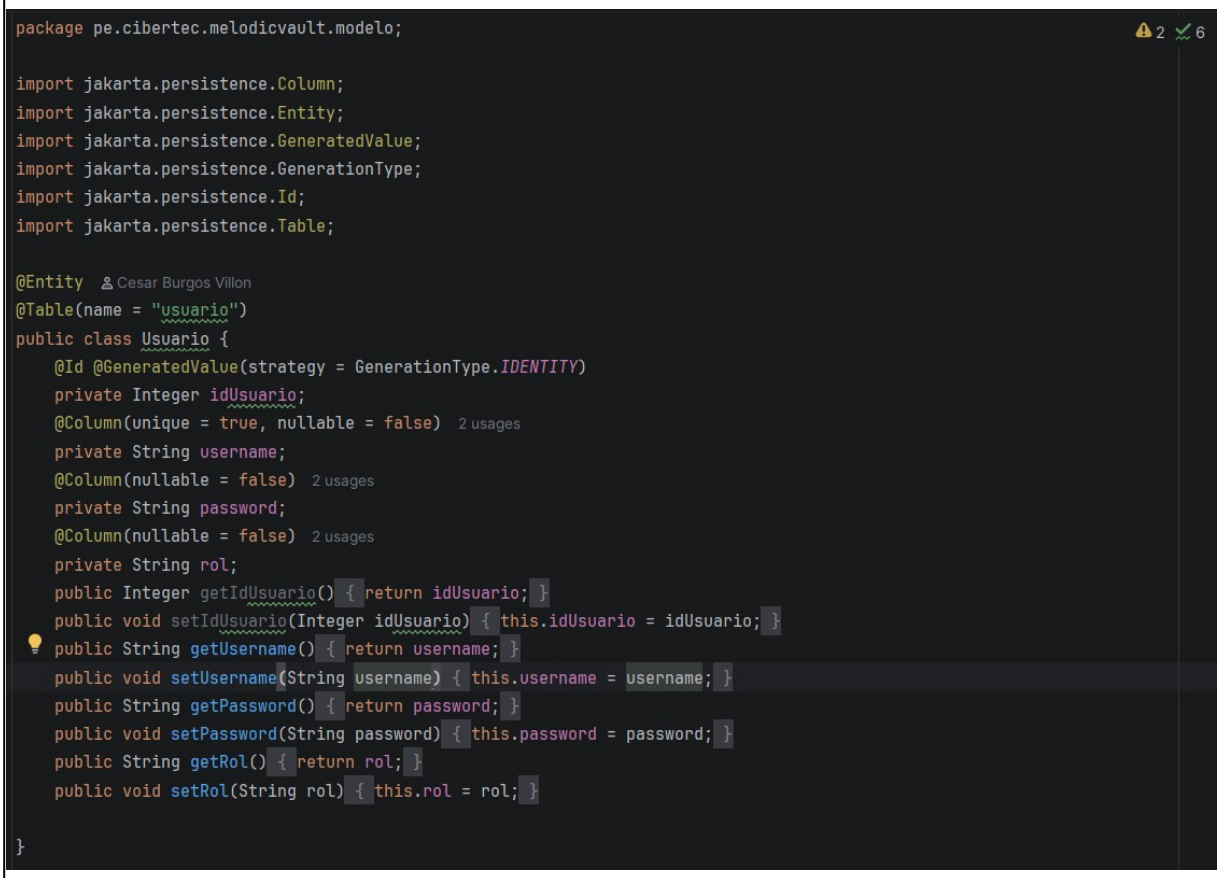

}
```

INTERPRETACION

RegistroForm fue un DTO (objeto de transferencia de datos) que representó los datos que el usuario ingresó en el formulario de registro. Su función fue recibir esa información desde la vista y validarla antes de que llegara a la lógica del sistema, sin exponer directamente la entidad Usuario.

Contuvo tres campos: username, password y confirmar (la repetición de la contraseña). Cada uno tuvo reglas de validación definidas con Bean Validation. Cuando el controlador recibió el formulario con @Valid, Spring aplicó estas reglas y, si alguna fallaba, devolvió los mensajes de error personalizados a la vista de registro. De esta forma se garantizó que solo datos correctos y con el formato esperado llegaran a guardarse en la base de datos.

Figura 56: Clase Usuario



```java
package pe.cibertec.melodicvault.modelo;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity  Cesar Burgos Villon
@Table(name = "usuario")
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer idUsuario;
    @Column(unique = true, nullable = false)  2 usages
    private String username;
    @Column(nullable = false)  2 usages
    private String password;
    @Column(nullable = false)  2 usages
    private String rol;
    public Integer getIdUsuario() { return idUsuario; }
    public void setIdUsuario(Integer idUsuario) { this.idUsuario = idUsuario; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getRol() { return rol; }
    public void setRol(String rol) { this.rol = rol; }

}
```

INTERPRETACION

Usuario fue la entidad JPA que representó a un usuario del sistema y se mapeó directamente a la tabla usuario de la base de datos. Cada objeto de esta clase correspondió a una fila de esa tabla, y Spring Data JPA se encargó de guardarlo, consultarlo y actualizarlo sin necesidad de escribir SQL manual.

Esta entidad fue la pieza central del control de acceso: UsuarioDetailsService la usó para cargar los datos de autenticación durante el inicio de sesión, y RegistroController la creó y persistió cuando alguien se registraba.

Figura 57: UsuarioDetailsService

```java
package pe.cibertec.melodicvault.service;
import org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.stereotype.Service;
import org.springframework.security.core.userdetails.User;

import pe.cibertec.melodicvault.interfaces.IUsuario;
import pe.cibertec.melodicvault.modelo.Usuario;

@Service  Cesar Burgos Villon
public class UsuarioDetailsService implements UserDetailsService {
    @Autowired private IUsuario repo;
    @Override  Cesar Burgos Villon
    public UserDetails loadUserByUsername(String username) {
        Usuario u = repo.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("No existe"));
        return User.withUsername(u.getUsername())
            .password(u.getPassword()).roles(u.getRol()).build();
    }
}
```

INTERPRETACION

Fue la clase que conectó la base de datos con Spring Security durante el inicio de sesión. Implementó la interfaz UserDetailsService, cuyo único método, loadUserByUsername, era invocado automáticamente por Spring Security cada vez que alguien intentaba autenticarse con un nombre de usuario.

Con ese objeto, Spring Security comparó por sí mismo la contraseña ingresada con el hash guardado, usando el PasswordEncoder configurado, y decidió si la autenticación era válida. Si lo era, el usuario quedaba con una sesión activa y con los permisos de su rol, que luego se usaban para restringir el acceso a rutas y funcionalidades.

Figura 58: IUsuario



INTERPRETACION

IUsuario fue el repositorio de acceso a datos de la entidad Usuario. Fue una interfaz que extendió CrudRepository<Usuario, Integer>, de Spring Data, donde Usuario era la entidad que gestionaba.

findByUsername buscó un usuario por su nombre de acceso y devolvió un Optional<Usuario>, de modo que el resultado podía estar vacío si no existía; UsuarioDetailsService lo usó durante el inicio de sesión para cargar los datos de autenticación. existsByUsername devolvió un valor booleano que indicó si ya existía un usuario con ese nombre.

Figura 59: RegistroController

```java
package pe.cibertec.melodicvault.controller;

import ...

@Controller  Cesar Burgos Villon
public class RegistroController {
    @Autowired private IUsuario repo;
    @Autowired private PasswordEncoder encoder;

    @GetMapping("/registro")  Cesar Burgos Villon
    public String form(Model model) {
        model.addAttribute( attributeName: "registroForm", new RegistroForm());
        return "registro";
    }
```

```java
    @PostMapping("/registro")  Cesar Burgos Villon
    public String registrar(@Valid @ModelAttribute("registroForm") RegistroForm form,
                            BindingResult result, RedirectAttributes flash) {

        if (!result.hasErrors() && !form.getPassword().equals(form.getConfirmar()))
            result.rejectValue( field: "confirmar",  errorCode: "nocoincide",  defaultMessage: "Las contraseñas no coinciden");

        if (!result.hasErrors() && repo.existsByUsername(form.getUsername()))
            result.rejectValue( field: "username",  errorCode: "duplicado",  defaultMessage: "Ese usuario ya existe");

        if (result.hasErrors()) return "registro";

        Usuario u = new Usuario();
        u.setUsername(form.getUsername());
        u.setPassword(encoder.encode(form.getPassword()));
        u.setRol("LECTOR");

        try {
            repo.save(u);
        } catch (DataIntegrityViolationException e) {
            result.rejectValue( field: "username",  errorCode: "duplicado",  defaultMessage: "Ese usuario ya existe");
            return "registro";
        }

        flash.addFlashAttribute( attributeName: "exito",  attributeValue: "Cuenta creada. Ya puedes iniciar sesión.");
        return "redirect:/login";
    }
}
```

INTERPRETACION

RegistroController fue el controlador de Spring MVC que gestionó el registro de nuevos usuarios. Atendió la ruta /registro con dos métodos: uno para mostrar el formulario y otro para procesarlo.

El método registrar, asociado a una petición POST, recibió los datos enviados. La anotación @Valid activó las validaciones definidas en el DTO, y BindingResult recogió los errores encontrados. Luego, el controlador aplicó dos validaciones adicionales de lógica de negocio: verificó que la contraseña y su confirmación coincidieran, y que el nombre de usuario no existiera ya en la base de datos.

Estas comprobaciones solo se ejecutaron si no hubo errores previos, y si alguna fallaba, se volvió a mostrar el formulario con el mensaje correspondiente.

Si todo era correcto, se creó una nueva entidad Usuario con el nombre ingresado, la contraseña cifrada con encoder.encode(...) (nunca en texto plano) y el rol fijo LECTOR, de modo que toda cuenta nueva nació con los permisos mínimos y nadie pudo asignarse un rol superior desde el formulario.

Finalmente, si el registro fue exitoso, se redirigió a /login con un mensaje flash que confirmó la creación de la cuenta, el cual se mostró una sola vez en la siguiente pantalla.

Figura 60: LoginController

```java
package pe.cibertec.melodicvault.controller;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller  Cesar Burgos Villon
public class LoginController {

    @GetMapping("/login")  Cesar Burgos Villon
    public String login() {
        return "login";
    }
}
```

INTERPRETACION

LoginController fue un controlador de Spring MVC muy sencillo cuya única función fue mostrar la página de inicio de sesión. Atendió las peticiones GET a la ruta /login y devolvió el nombre de la vista login, que el motor de plantillas resolvió y presentó al usuario con el formulario de acceso.

Figura 61: SecurityConfig

```java
@Configuration  Cesar Burgos Villon
@EnableWebSecurity
public class SecurityConfig {
    @Autowired private UsuarioDetailsService uds;

    @Bean  Cesar Burgos Villon
    public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean  Cesar Burgos Villon
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
                .requestMatchers( ...patterns: "/css/**", "/img/**", "/js/**").permitAll()
                .requestMatchers( ...patterns: "/", "/acerca", "/bandas", "/albumes", "/canciones", "/*/detalle/**", "/registro").permitAll()
                .requestMatchers( ...patterns: "/*/nuevo", "/*/editar/**", "/*/guardar", "/*/eliminar/**", "/albumes/completo/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .formLogin( FormLoginConfigurer<HttpSecurity> f -> f.loginPage("/login").permitAll())
            .logout( LogoutConfigurer<HttpSecurity> l -> l.logoutSuccessUrl("/"));
        return http.build();
    }
}
```

INTERPRETACION

SecurityConfig fue la clase central del control de acceso: definió qué rutas eran públicas, cuáles exigían iniciar sesión y cuáles estaban reservadas para un rol específico. Estuvo anotada con @Configuration y @EnableWebSecurity, lo que activó la configuración de seguridad web de Spring Security y la registró dentro del contexto de la aplicación.

Las rutas de recursos y de consultas fueron públicas; las rutas de manipulación y los reportes requirieron sesión iniciada (el Lector solo pudo acceder a los reportes).

Spring Security procesó el envío del formulario por su cuenta. logout definió que, al cerrar sesión, el usuario fuera redirigido a la página de inicio (/), invalidando la sesión activa.

Figura 62: Usuario de Prueba

9	$2b$10$aGsUNkvtNDwhTgm29zPs8eqWjD9I8rYghRx0p4ntzot/T.iCesOAq	ADMIN	admin1
10	$2b$10$DQuGrGBMFRwENahhD4sQ.uEe4hfvIGi/RHImkPataPw.MNhm6iFBm	LECTOR	lector

INTERPRETACION

En la siguiente tabla se muestran los permisos de cada rol en el sistema

Tabla *43*

Roles del sistema

Rol	Permisos principales	Usuario de prueba
Administrador	Gestionó álbumes, bandas y canciones.	admin1
Lector	Consultó y descargó información.	lector

7.6.3. Implementación de mantenimientos

En MelodicVault se implementaron tres mantenimientos: Bandas, Álbumes y Canciones. Cada uno fue un módulo CRUD completo que leyó y escribió en MySQL a través de la cadena controlador → servicio → repositorio , y los tres compartieron las mismas cinco rutas

El listado fue de acceso público; registrar, editar y eliminar solo estuvieron disponibles con la sesión del Administrador, por lo que todas las pruebas de esta sección se realizaron con esa sesión.

Tabla *44*

Rutas comunes de los tres mantenimientos

Operación	Ruta	Método	Acceso	Resultado
Listar	{base}	GET	Público	Vista con el listado y el total de registros.
Registrar (formulario)	{base}/nuevo	GET	ADMIN	Formulario vacío, en modo registrar.
Editar (formulario)	{base}/editar/{id}	GET	ADMIN	Formulario con los datos guardados, en modo editar; si el id no existe, vuelve al listado con un mensaje de error.
Guardar	{base}/guardar	POST	ADMIN	Valida y guarda el alta o la edición; redirige al listado con un mensaje de éxito.
Eliminar	{base}/	GET	ADMIN	Elimina el registro tras la confirmación del

	eliminar/ {id}			navegador; redirige al listado con un mensaje.

INTERPRETACION

En la tabla, {base} fue /bandas, /albumes o /canciones, de modo que los tres controladores siguieron la misma estructura. Una sola ruta POST atendió el alta y la edición: el formulario incluyó el identificador en un campo oculto y, si llegaba vacío, el registro se insertó; si traía un valor, se actualizó. Solo el listado quedó abierto a cualquier visitante.

7.6.3.1. Mantenimiento 1: Bandas

Figura 63 :Edición y eliminación en BandaController, y campo oculto del formulario

```
    @GetMapping("/editar/{id}")
    public String editar(@PathVariable("id") int id, Model model, RedirectAttributes flash) {
        Optional<Banda> banda = service.listarId(id);

        if (banda.isEmpty()) {
            flash.addFlashAttribute("error", "La banda solicitada no existe.");
            return "redirect:/bandas";
        }

        model.addAttribute("banda", banda.get());
        model.addAttribute("modo", "editar");
        return "form-banda";
    }
```

```
    @GetMapping("/eliminar/{id}")
    public String eliminar(@PathVariable("id") int id, RedirectAttributes flash) {
        if (service.listarId(id).isEmpty()) {
            flash.addFlashAttribute("error", "La banda no existe.");
            return "redirect:/bandas";
        }
        service.delete(id);
        flash.addFlashAttribute("mensaje", "Banda eliminada correctamente.");
        return "redirect:/bandas";
    }
```

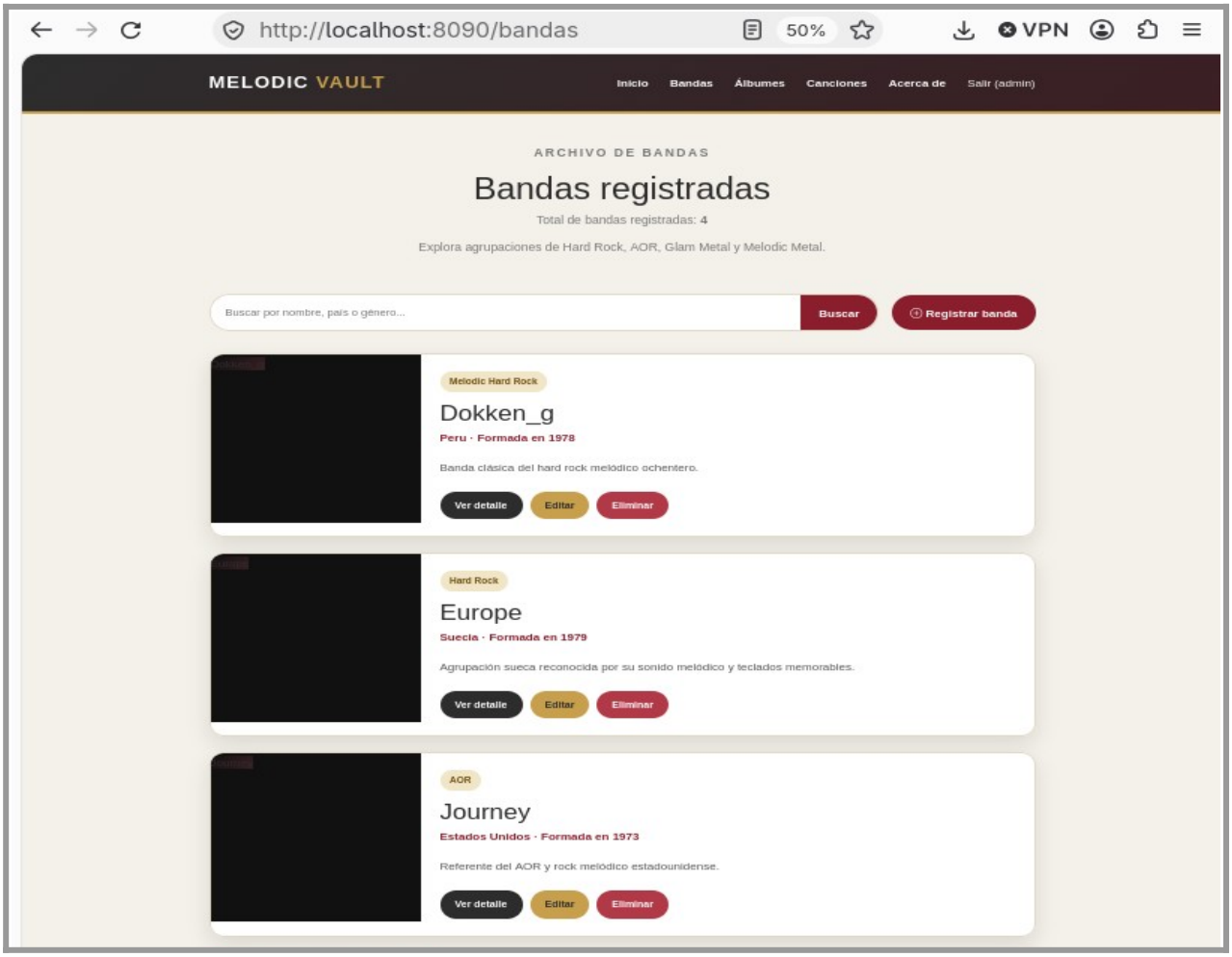
```
    <!-- bandas.html: el navegador pide confirmación antes de enviar la petición -->
    <a th:href="@{/bandas/eliminar/{id}(id=${b.idBanda})}"
       class="btn-eliminar"
       onclick="return confirm('¿Seguro que deseas eliminar esta banda? También se eliminarán sus álbumes asociados.');">
        Eliminar
    </a>
 </th:block>
</div>
```

INTERPRETACION

. El método editar buscó la banda con listarId y, si no existía, volvió al listado con un mensaje en lugar de abrir un formulario vacío; si existía, la entregó a form-banda en modo editar, que mostró "Editar banda" y "Guardar cambios".

El campo oculto idBanda permitió que guardar actualizara la fila existente en lugar de insertar otra. eliminar llamó a service.delete y, por cascade = ALL (Código 5), la eliminación alcanzó a los álbumes de la banda y a sus canciones.

Figura 64 :Pantalla de listado del mantenimiento de Bandas

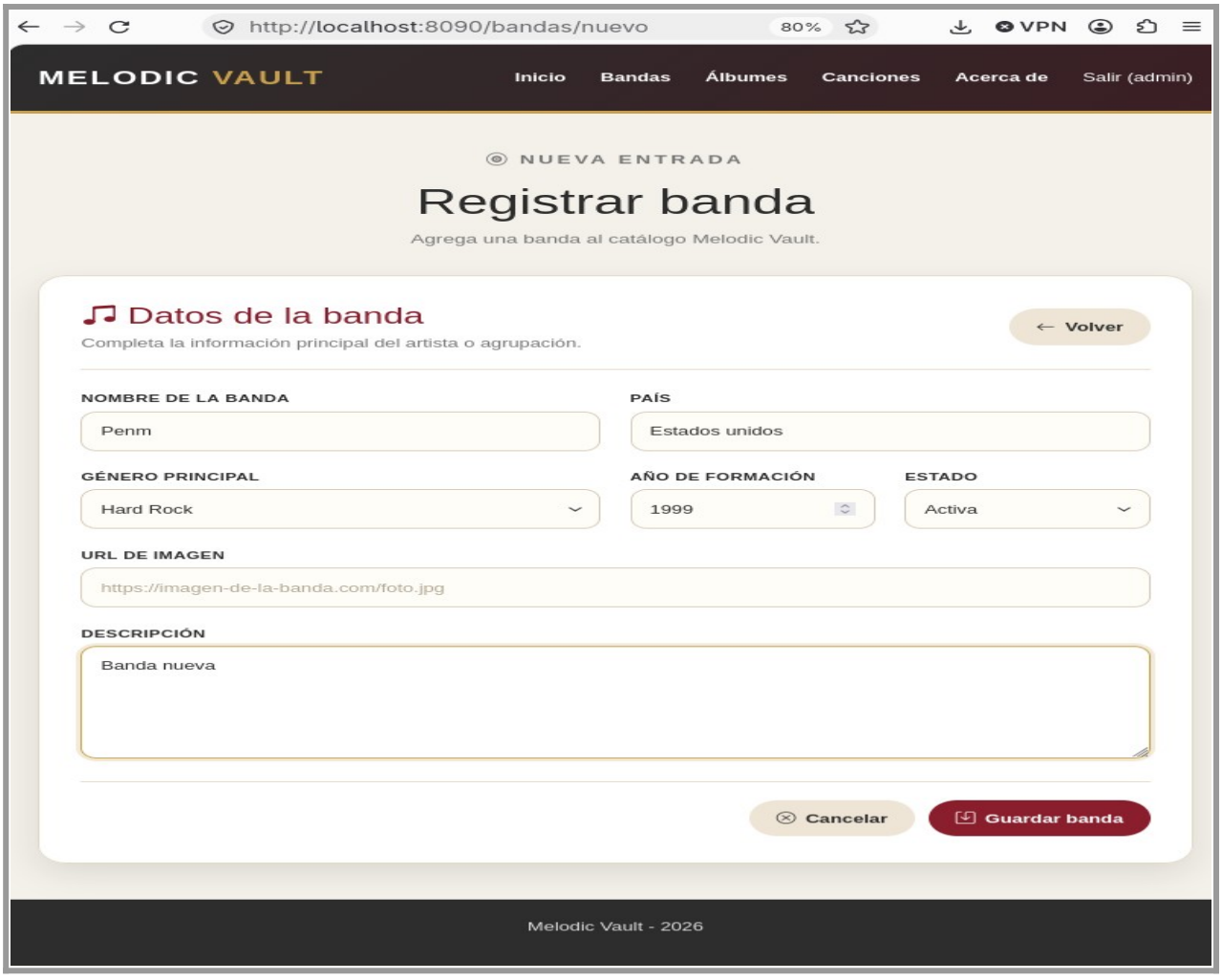


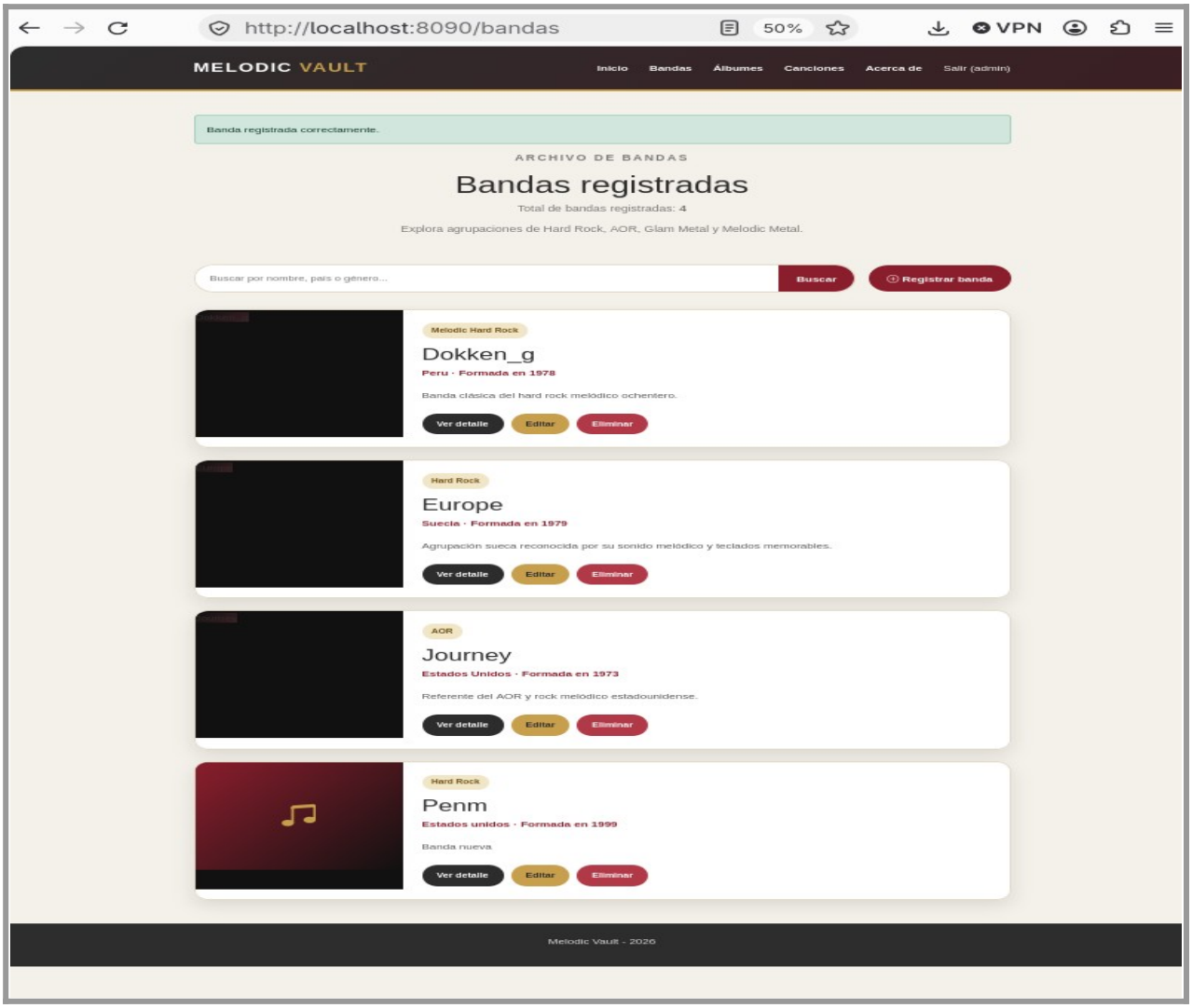
INTERPRETACION

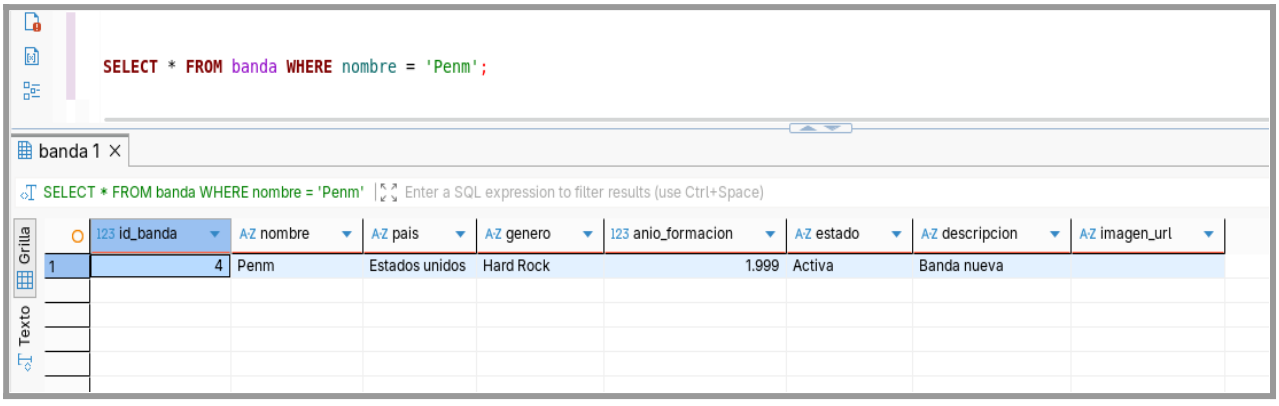
. La pantalla mostró las tres bandas cargadas por el script (Dokken, Europe y Journey) y el total de bandas registradas.

Los botones Registrar banda, Editar y Eliminar aparecieron porque la sesión fue de Administrador.

Figura 65 : Imagen – Escritura en la bd desde el lado del cliente





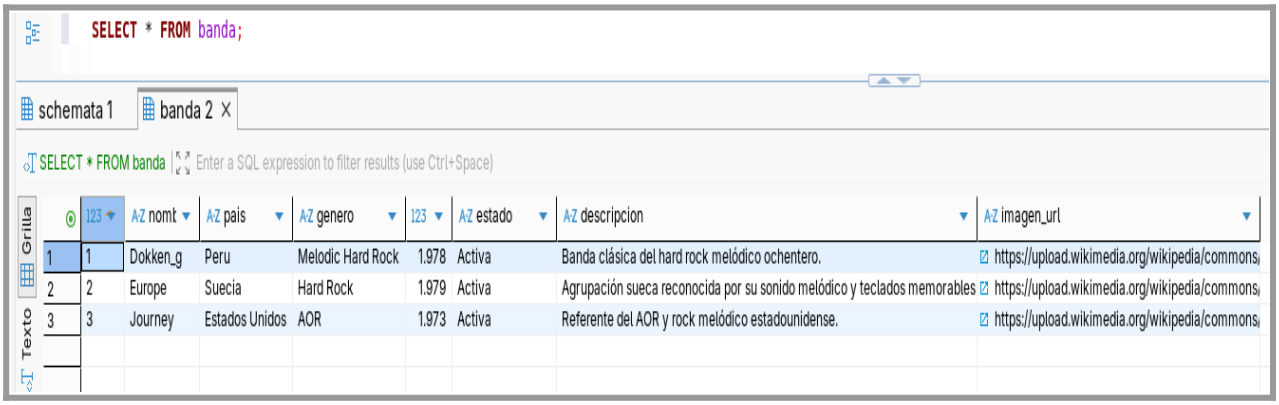


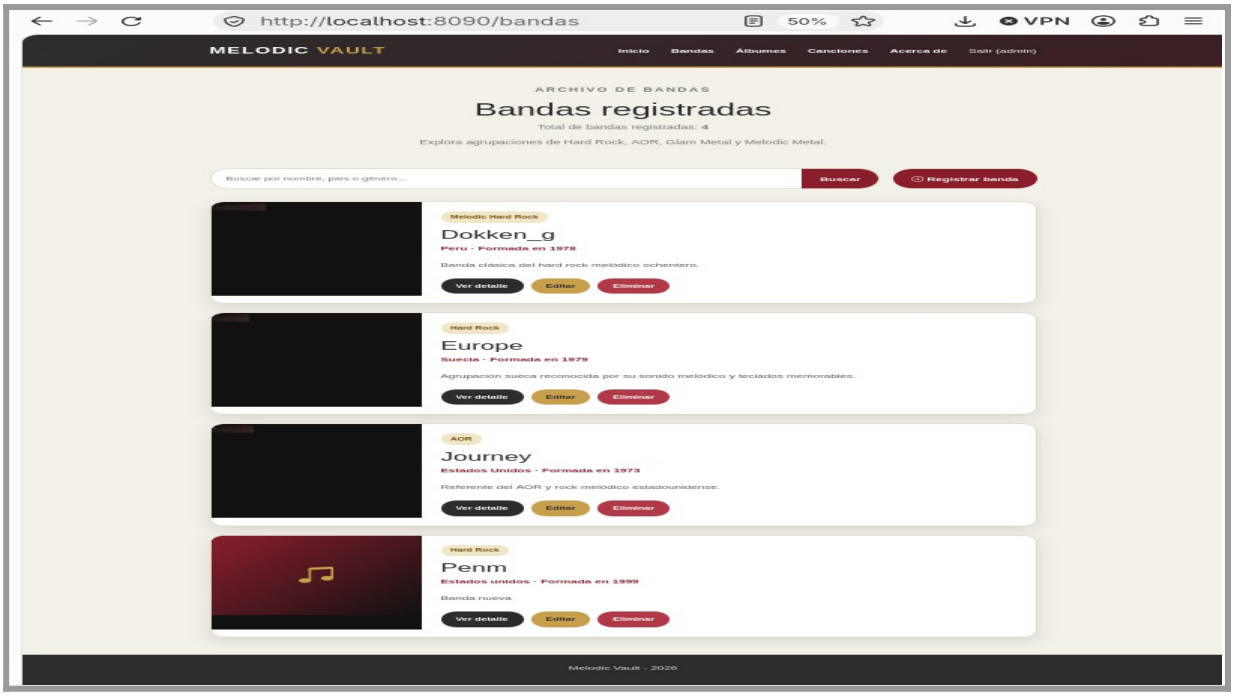
INTERPRETACION

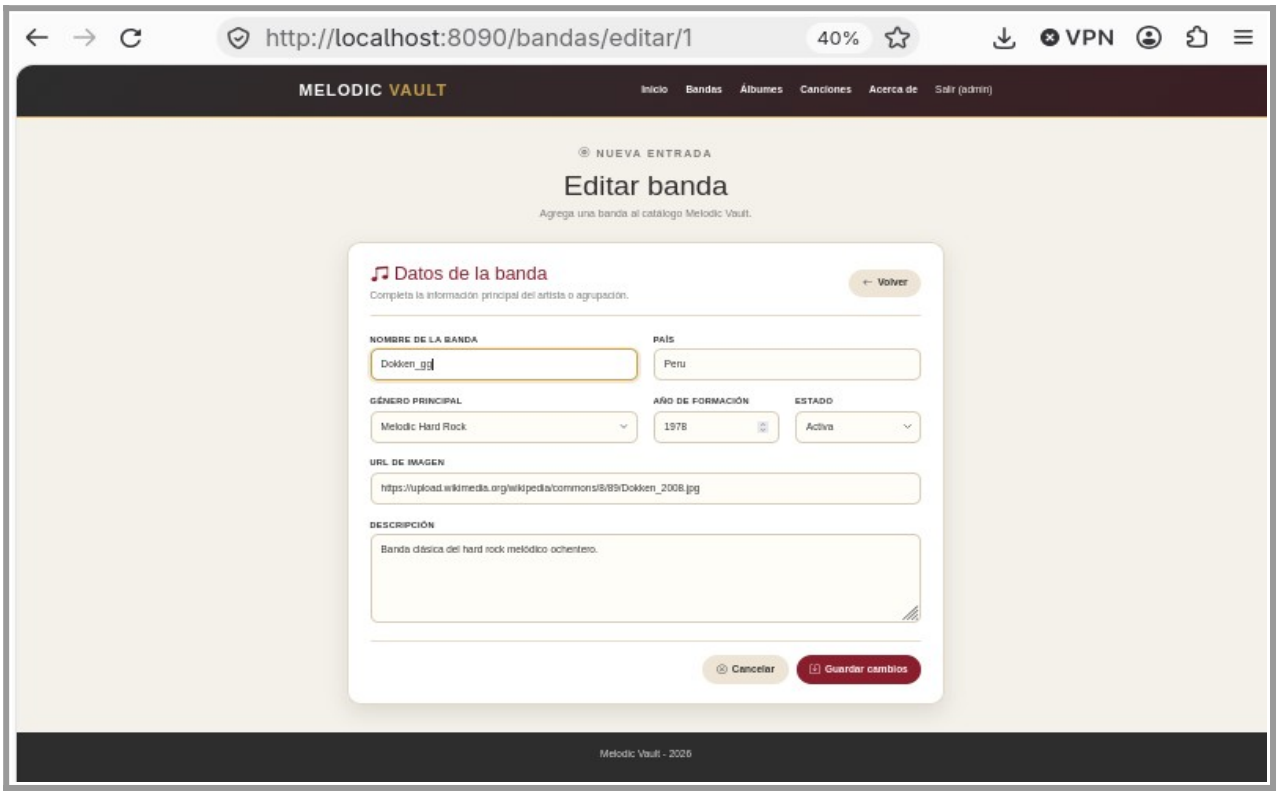
El formulario envió los siete campos de la banda a /bandas/guardar; tras la validación, la banda se insertó y el sistema redirigió al listado con el mensaje de éxito.

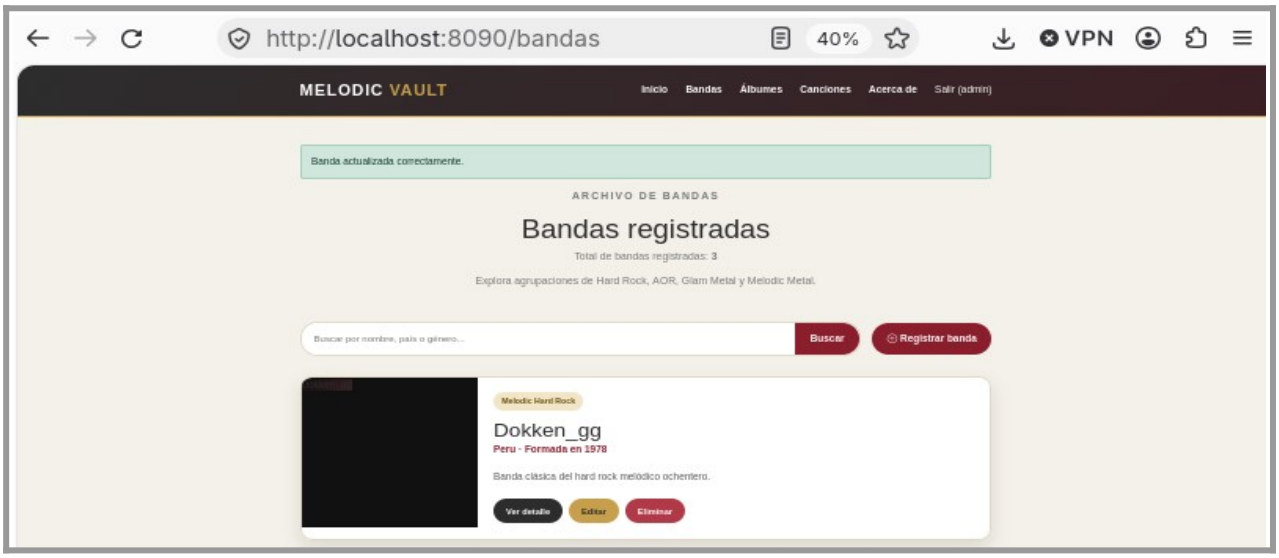
La consulta en MySQL Workbench confirmó que la fila existía en la tabla banda, con un id_banda generado automáticamente, es decir, que el registro no quedó solo en pantalla.

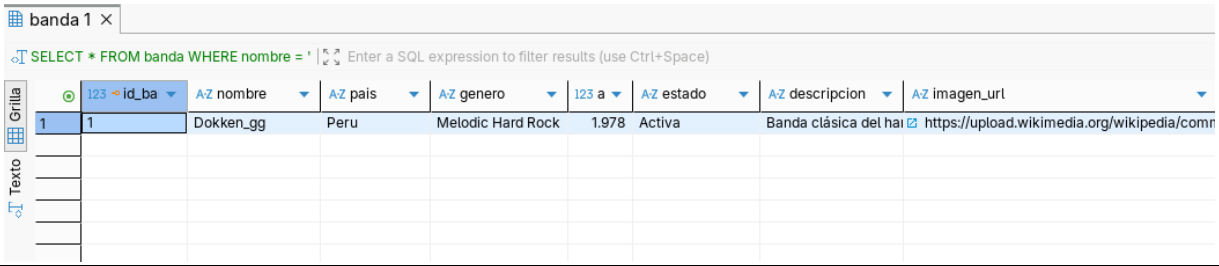
Figura 66 :Edición de una banda y evidencia en la base de datos






http://localhost:8090/bandas/editar/1
40%
VPN
MELODIC VAULT
Inicio
Bandas
Álbumes
Canciones
Acerca de
Salir (admin)
NUEVA ENTRADA
Editar banda
Agrega una banda al catálogo Melodic Vault.
Datos de la banda
Completa la información principal del artista o agrupación.
← Volver
NOMBRE DE LA BANDA
Dokken_gg
PAÍS
Peru
GÉNERO PRINCIPAL
Melodic Hard Rock
AÑO DE FORMACIÓN
1978
ESTADO
Activa
URL DE IMAGEN
https://upload.wikimedia.org/wikipedia/commons/8/89/Dokken_2008.jpg
DESCRIPCIÓN
Banda clásica del hard rock melódico ochentero.
Cancelar
Guardar cambios
Melodic Vault - 2026

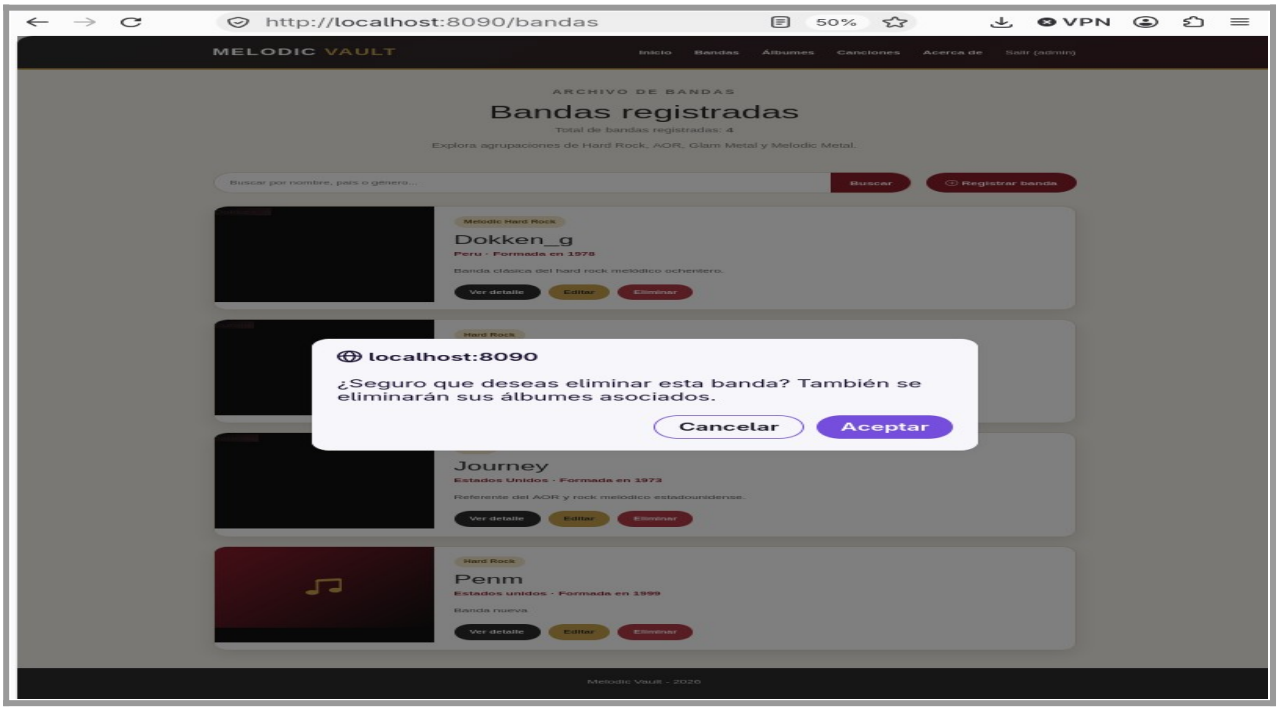

http://localhost:8090/bandas
40%
VPN
MELODIC VAULT
Inicio
Bandas
Álbumes
Canciones
Acerca de
Salir (admin)
Banda actualizada correctamente.
ARCHIVO DE BANDAS
Bandas registradas
Total de bandas registradas: 3
Explora agrupaciones de Hard Rock, AOR, Glam Metal y Melodic Metal.
Buscar por nombre, país o género...
Buscar
Registrar banda
Melodic Hard Rock
Dokken_gg
Peru · Formada en 1978
Banda clásica del hard rock melódico ochentero.
Ver detalle
Editar
Eliminar

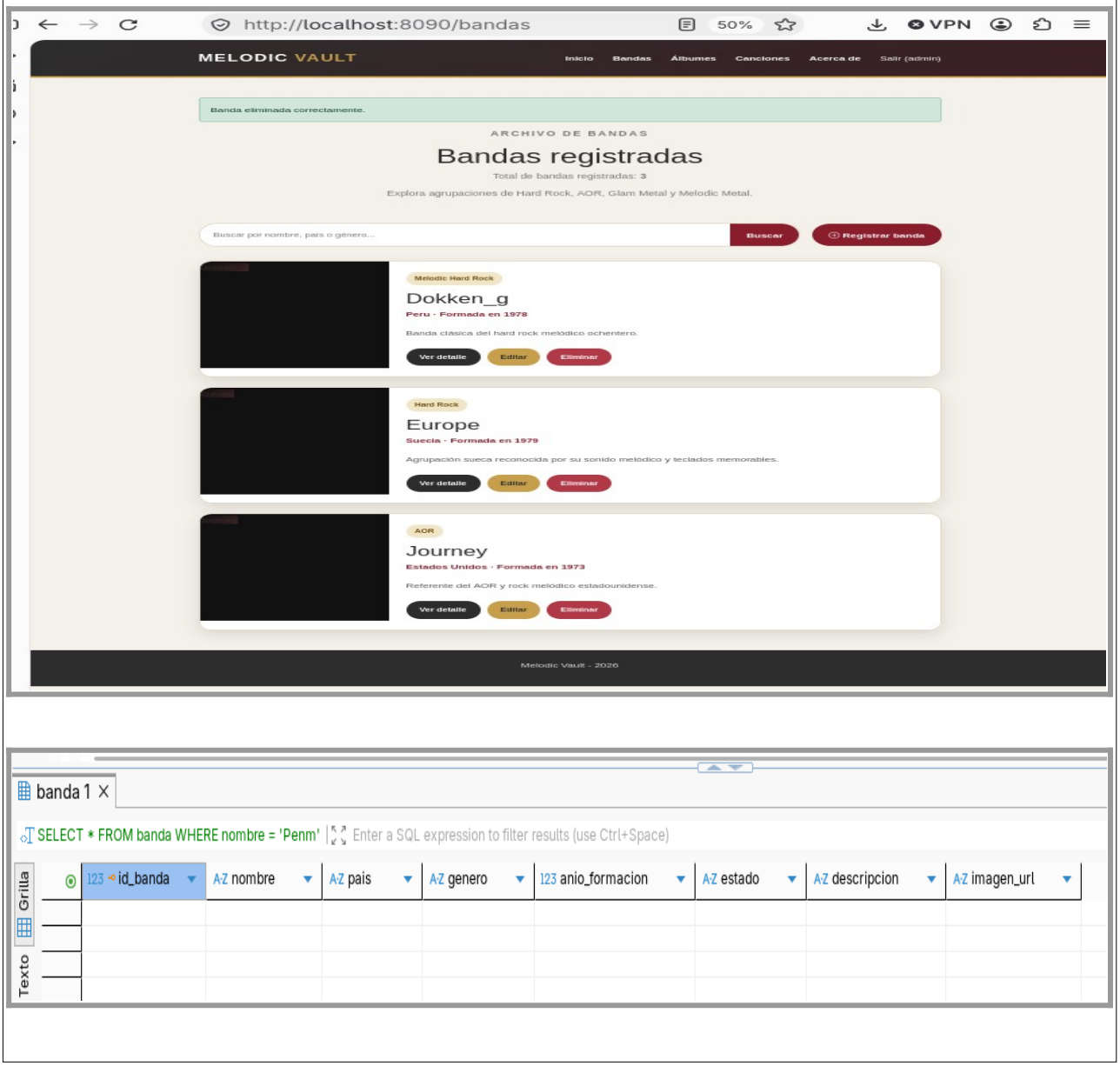

banda 1
SELECT * FROM banda WHERE nombre = '
Enter a SQL expression to filter results (use Ctrl+Space)
Grilla
Texto
id_ba
nombre
pais
genero
a
estado
descripcion
imagen_url
1
1
Dokken_gg
Peru
Melodic Hard Rock
1.978
Activa
Banda clásica del hai
https://upload.wikimedia.org/wikipedia/comn

INTERPRETACION

Tras cambiar el nombre de la banda de Dokken_g a Dokken_gg y guardar, la consulta devolvió la misma fila, con el mismo id_banda y el nombre modificado, lo que demostró que la edición actualizó el registro y no creó uno nuevo.

Figura 67 : Imagen – Eliminación de una banda y evidencia en la base de datos





INTERPRETACION

Tras aceptar la confirmación del navegador, el listado volvió a mostrar las tres bandas originales junto con el mensaje de eliminación.

La consulta sobre la tabla banda no devolvió filas, por lo que el borrado se aplicó en la base de datos.

7.6.3.2. Mantenimiento 2: Albumes

El mantenimiento de Álbumes trabajó sobre la entidad Album, que perteneció siempre a una banda (@ManyToOne, columna id_banda) y contuvo sus canciones (@OneToMany con CascadeType.REMOVE).

Por eso el formulario pidió elegir la banda desde una lista y la eliminación de un álbum arrastró sus canciones, tanto por la relación JPA como por la clave foránea ON DELETE CASCADE del script.

Antes de editar o eliminar, el controlador verificó que el álbum existiera y, si no, volvió al listado con un mensaje de error.

Figura 68 : Imagen – Selección de la banda en el formulario de álbum y recarga de la lista al fallar la validación

```
    <div class="col-md-6">
        <label class="form-label">Banda</label>
        <!-- form-album.html: la banda se eligió de una lista leída de la base de datos -->
        <select class="form-select input-vault" th:field="*{banda}">
            <option value="">Seleccionar banda</option>
            <option th:each="b : ${bandas}"
                    th:value="${b.idBanda}"
                    th:text="${b.nombre}">
            </option>
        </select>

        <span class="text-danger"
              th:if="${#fields.hasErrors('banda')}"
              th:errors="*{banda}">
        </span>


    </div>
```

```
@PostMapping("/guardar")
public String guardar(@Valid Album album, BindingResult result, Model model, RedirectAttributes flash) {

    if (result.hasErrors()) {
        model.addAttribute("bandas", bandaService.listar());
        model.addAttribute("modo", album.getIdAlbum() == null ? "registrar" : "editar");
        return "form-album";
    }

    boolean esNuevo = album.getIdAlbum() == null;

    service.save(album);

    flash.addFlashAttribute("mensaje",
            esNuevo ? "Álbum registrado correctamente." : "Álbum actualizado correctamente.");

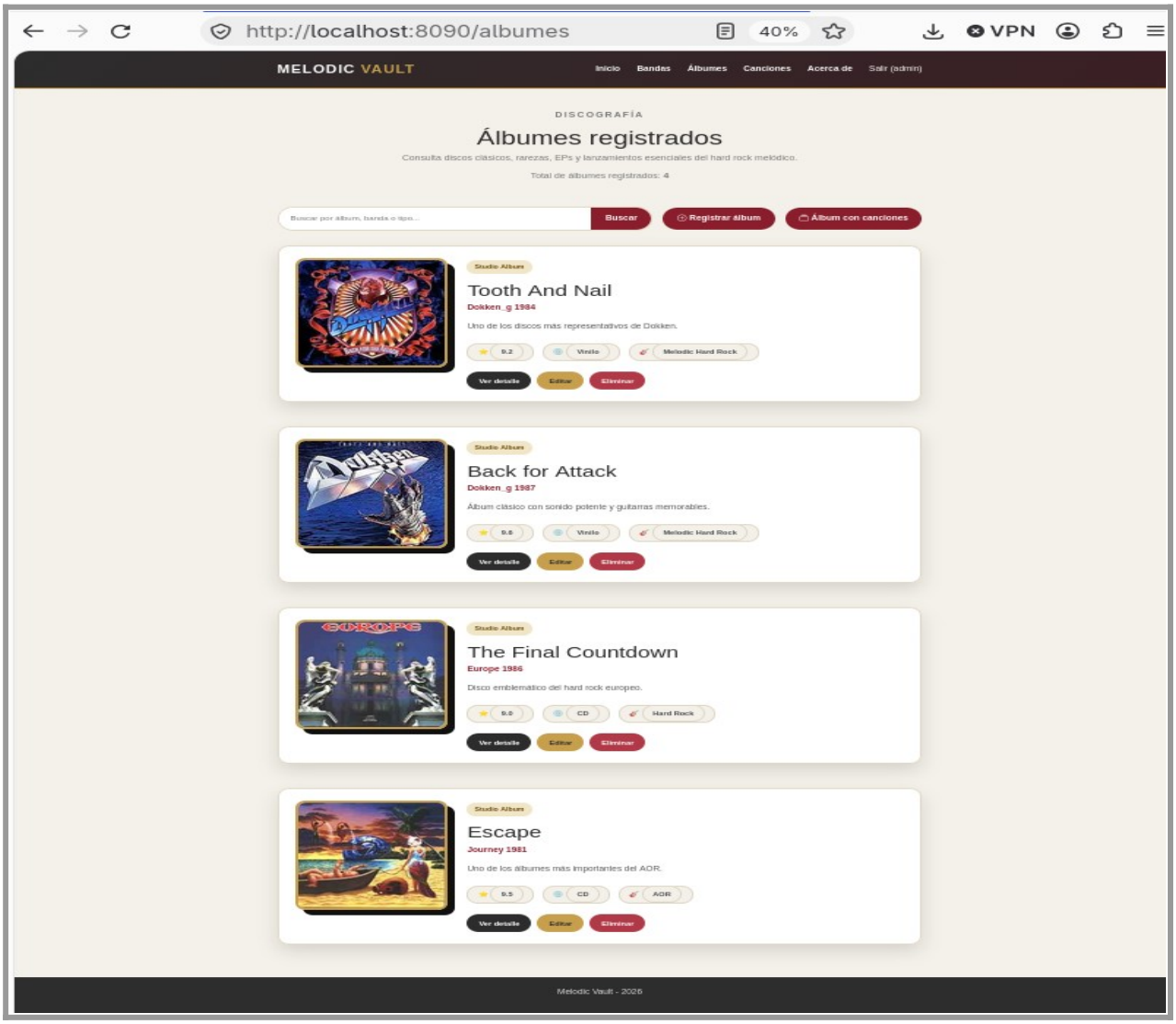
    return "redirect:/albumes";
}
```

INTERPRETACION

El select se llenó con las bandas que entregó bandaService.listar(), y el valor elegido quedó asociado a la propiedad banda del álbum, que se guardó como la clave foránea id_banda.

Si se dejaba "Seleccionar banda", la validación @NotNull mostraba "Debe seleccionar una banda". Al volver al formulario por un error, el controlador tuvo que cargar de nuevo la lista de bandas; de lo contrario, el select habría aparecido vacío.

Figura 69 : Imagen – Pantalla de listado del mantenimiento de Álbumes

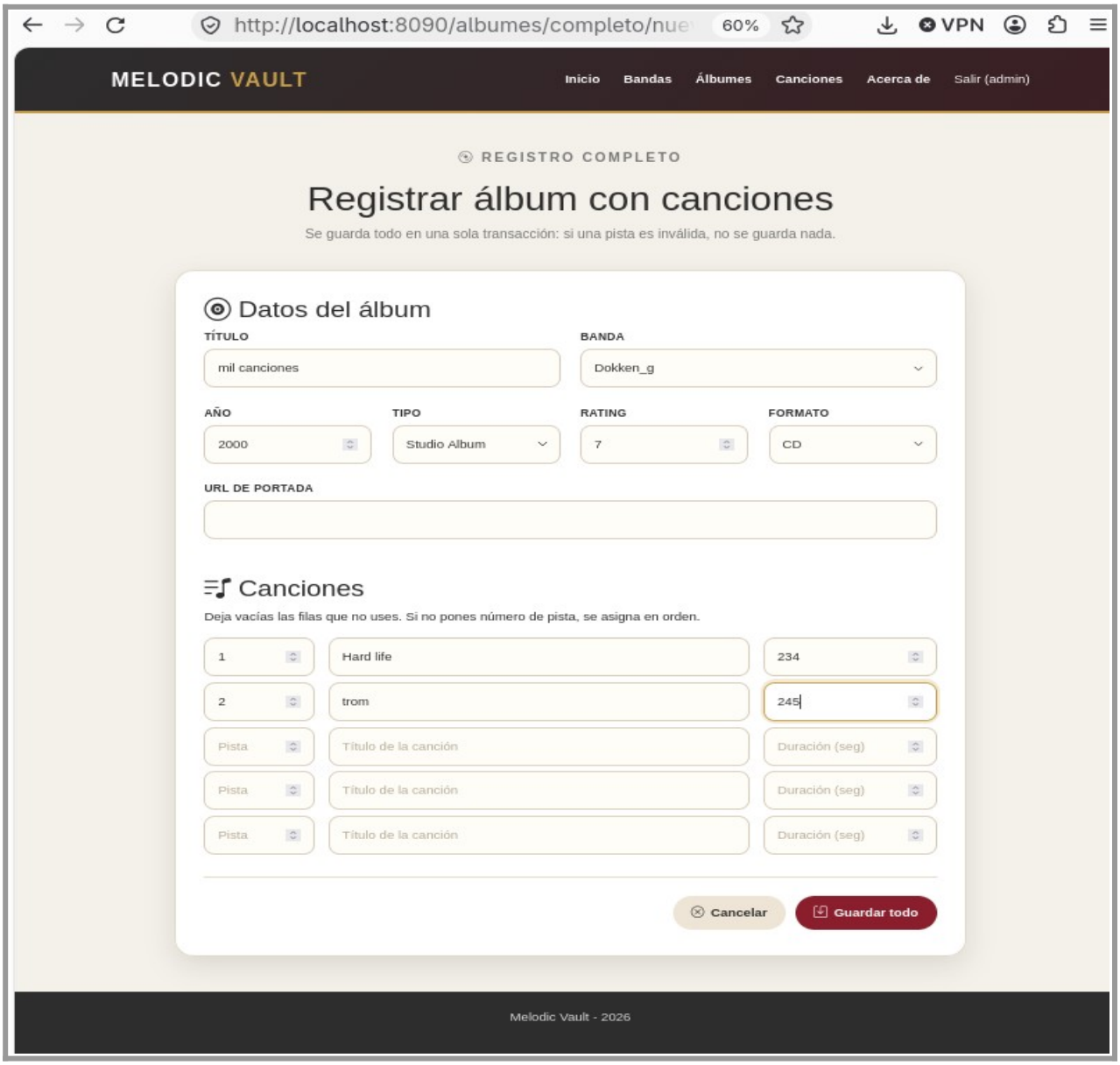


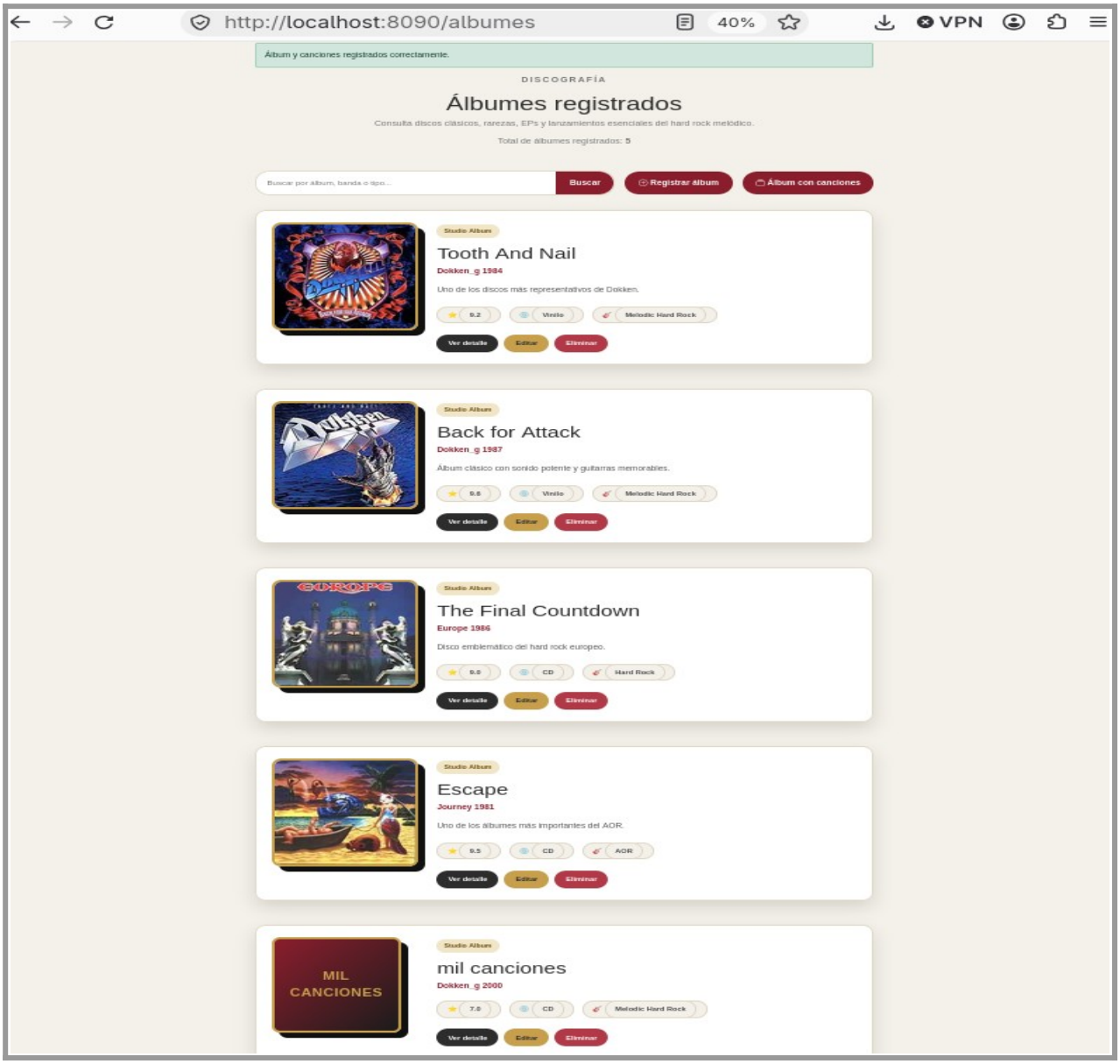
INTERPRETACION

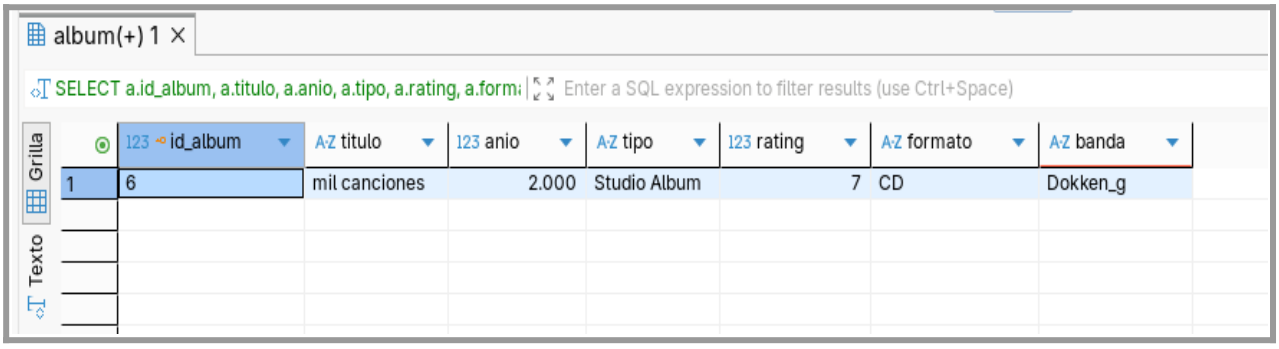
La pantalla mostró los cuatro álbumes cargados por el script, cada uno con su portada, tipo, banda, año, rating y formato, junto con el total de álbumes registrados.

Los botones Registrar álbum, Editar y Eliminar aparecieron por la sesión de Administrador.

Figura 70 : Imagen – Registro de un álbum y evidencia en la base de datos




http://localhost:8090/albumes
40%
VPN
Álbum y canciones registrados correctamente.
DISCOGRAFÍA
Álbumes registrados
Consulta discos clásicos, rarezas, EPs y lanzamientos esenciales del hard rock melódico.
Total de álbumes registrados: 5
Buscar por álbum, banda o tipo...
Buscar
Registrar álbum
Álbum con canciones
Studio Album
Tooth And Nail
Dokken_g 1984
Uno de los discos más representativos de Dokken.
9.2
Vinilo
Melodic Hard Rock
Ver detalle
Editar
Eliminar
Studio Album
Back for Attack
Dokken_g 1987
Álbum clásico con sonido potente y guitarras memorables.
9.6
Vinilo
Melodic Hard Rock
Ver detalle
Editar
Eliminar
Studio Album
The Final Countdown
Europe 1986
Disco emblemático del hard rock europeo.
9.0
CD
Hard Rock
Ver detalle
Editar
Eliminar
Studio Album
Escape
Journey 1981
Uno de los álbumes más importantes del AOR.
9.5
CD
AOR
Ver detalle
Editar
Eliminar
MIL CANCIONES
Studio Album
mil canciones
Dokken_g 2000
7.0
CD
Melodic Hard Rock
Ver detalle
Editar
Eliminar

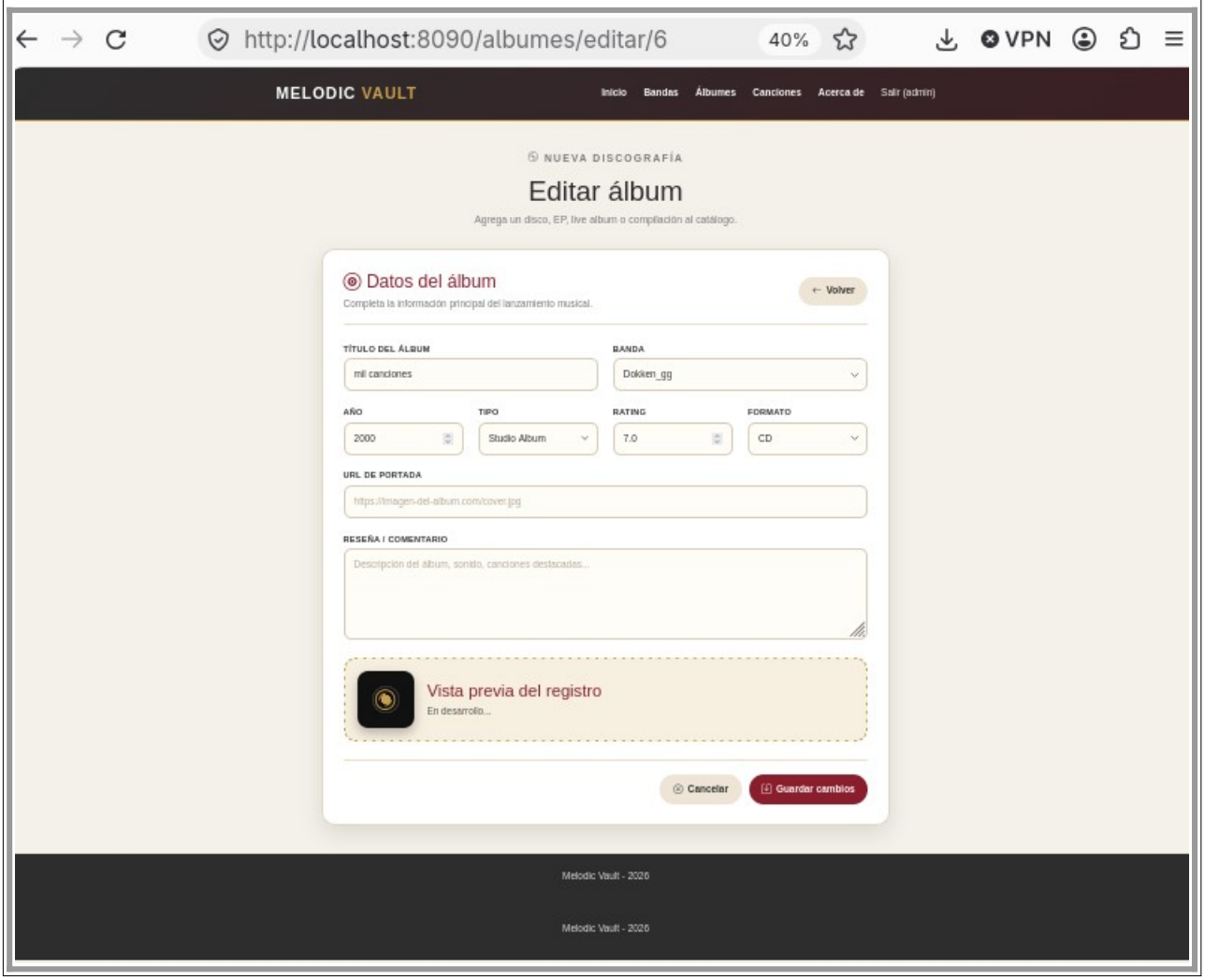

album(+) 1
SELECT a.id_album, a.titulo, a.anio, a.tipo, a.rating, a.formi
Enter a SQL expression to filter results (use Ctrl+Space)
Grilla
Texto
id_album
titulo
anio
tipo
rating
formato
banda
1
6
mil canciones
2.000
Studio Album
7
CD
Dokken_g

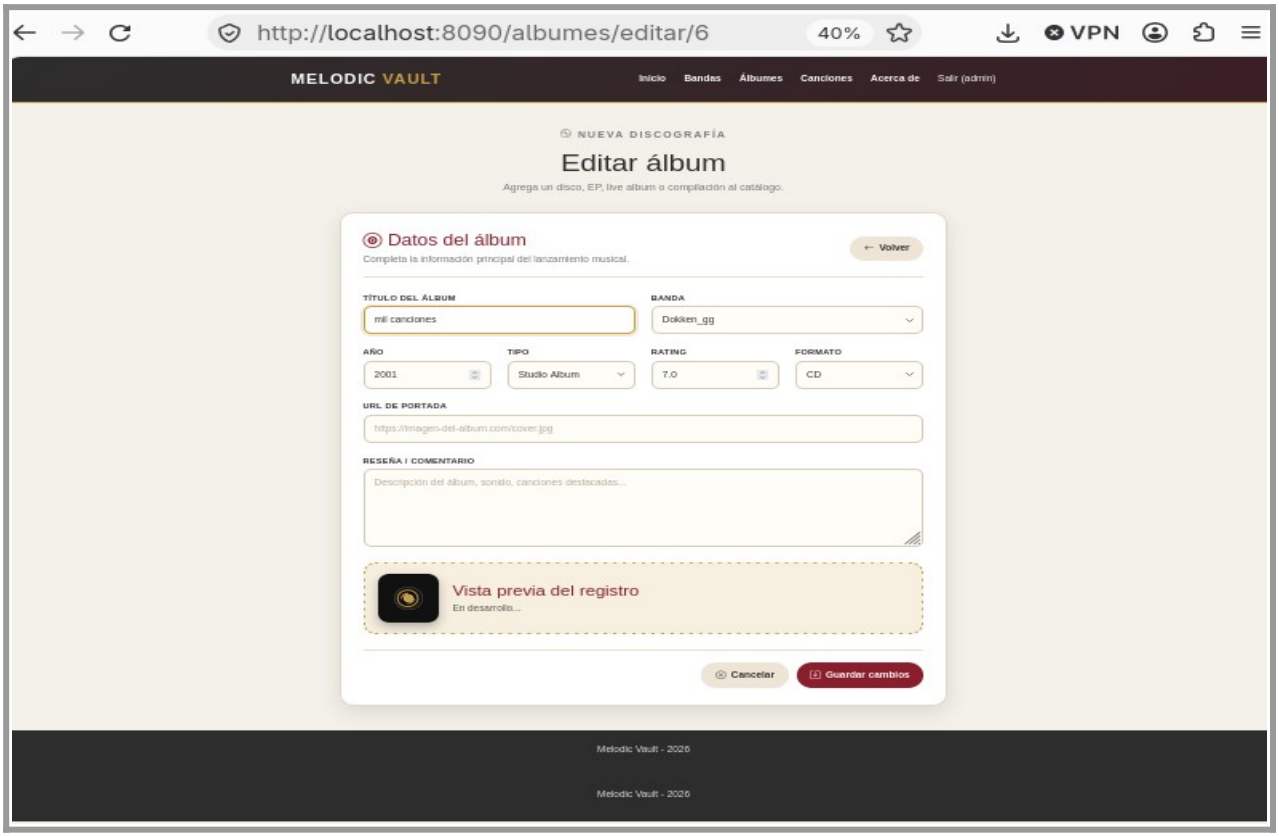
INTERPRETACION

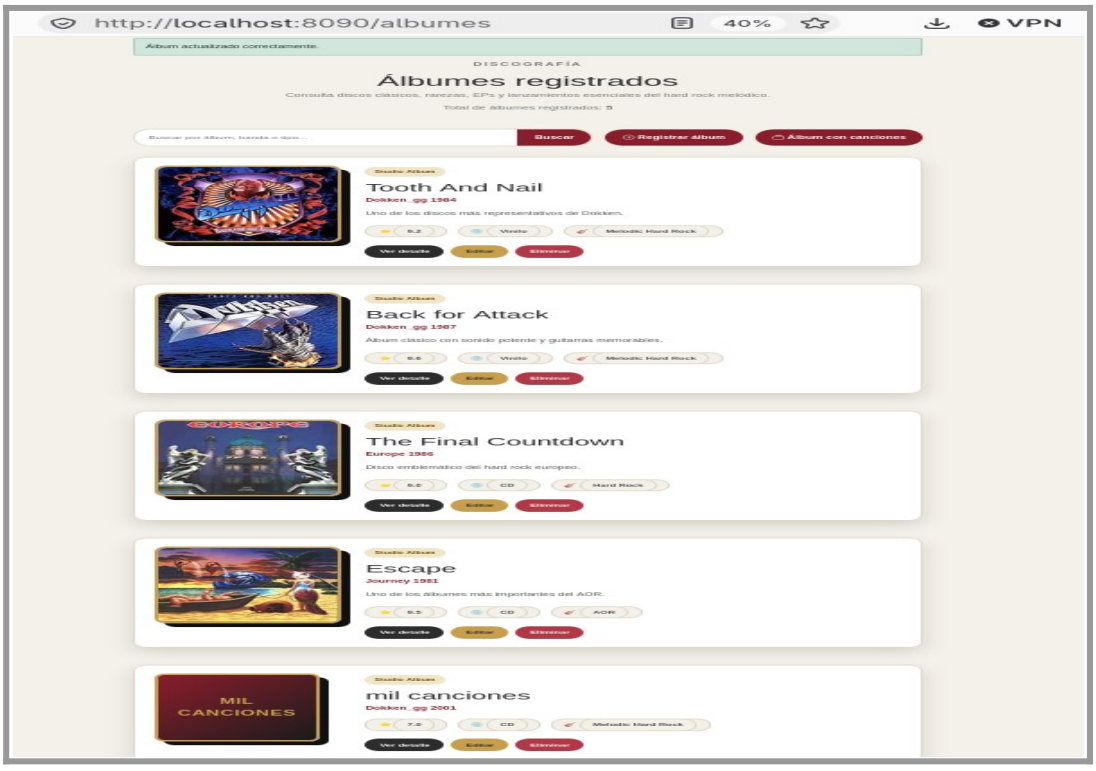
Se usó el formulario “Registrar álbum con canciones”: envió los datos del álbum (título, banda, año, tipo, rating, formato y portada) y dos filas de canciones, que AlbumTransaccionalService guardó en una sola transacción (@Transactional): si una pista era inválida, no se guardaba nada.

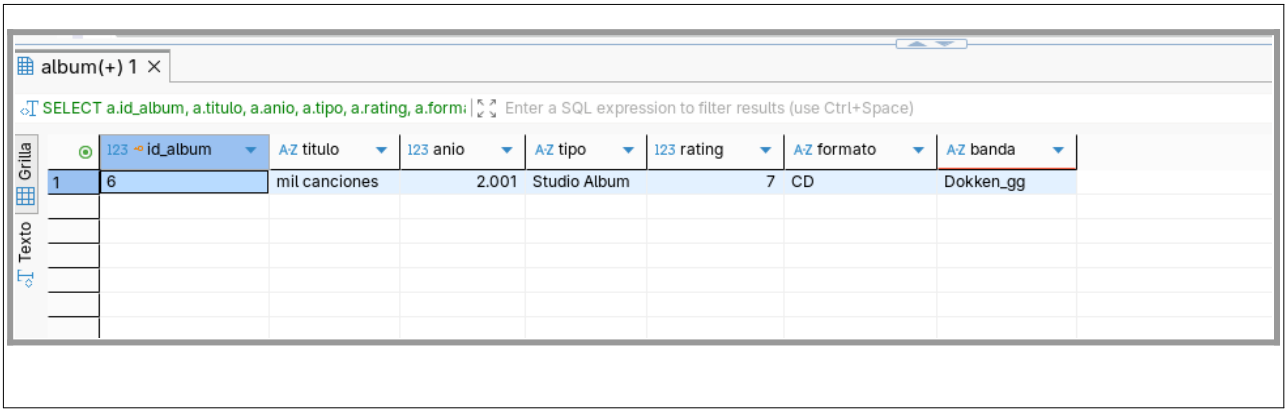
El listado mostró “Álbum y canciones registrados correctamente” y el total subió a cinco. La consulta con JOIN mostró el álbum asociado a su banda por id_banda y sus canciones asociadas por id_album.

Figura 71 : Edición de un álbum y evidencia en la base de datos




http://localhost:8090/albumes/editar/6
40%
VPN
MELODIC VAULT
Inicio
Bandas
Álbumes
Canciones
Acerca de
Salir (admin)
NUEVA DISCOGRAFÍA
Editar álbum
Agrega un disco, EP, live album o compilación al catálogo.
Datos del álbum
Completa la información principal del lanzamiento musical.
← Volver
TÍTULO DEL ÁLBUM
mil canciones
BANDA
Dokken_gg
AÑO
2001
TIPO
Studio Album
RATING
7.0
FORMATO
CD
URL DE PORTADA
https://imagen-del-album.com/cover.jpg
RESEÑA / COMENTARIO
Descripción del álbum, sonido, canciones destacadas...
Vista previa del registro
En desarrollo...
Cancelar
Guardar cambios
Melodic Vault - 2026
Melodic Vault - 2026


http://localhost:8090/albumes
40%
VPN
Álbum actualizado correctamente.
DISCOGRAFÍA
Álbumes registrados
Consulta discos clásicos, reseñas, EPs y lanzamientos esenciales del hard rock melódico.
Total de álbumes registrados: 5
Buscar por álbum, banda o tipo...
Buscar
Registrar álbum
Álbum con canciones
Tooth And Nail
Dokken_gg 1984
Back for Attack
Dokken_gg 1987
The Final Countdown
Europe 1986
Escape
Journey 1981
mil canciones
Dokken_gg 2001
MIL CANCIONES

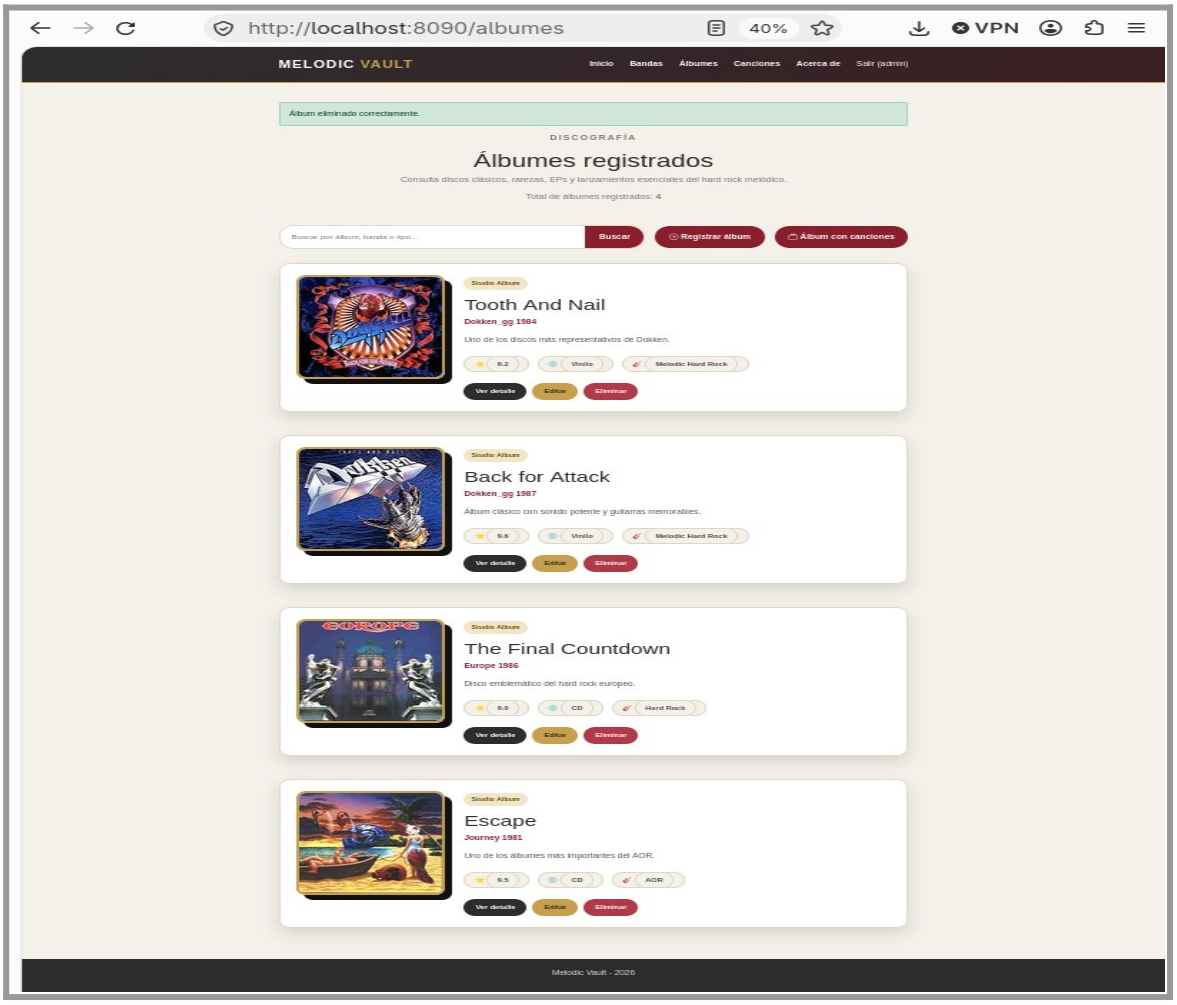


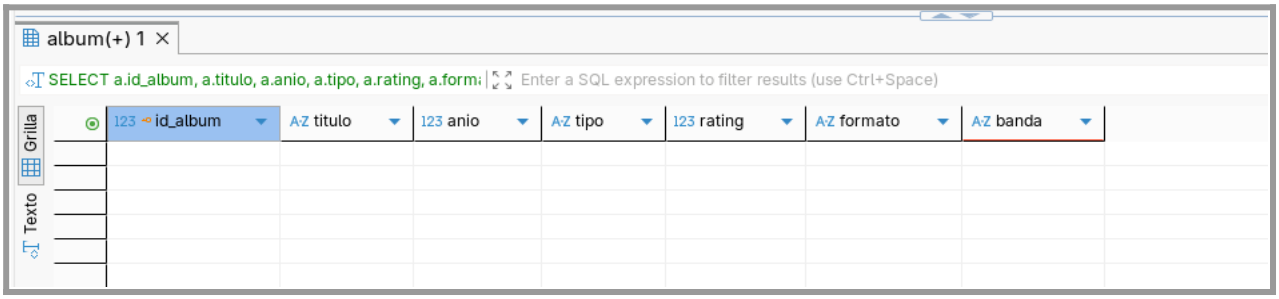
INTERPRETACION

El formulario se abrió con los datos guardados, el título “Editar álbum” y el año 2000. Tras cambiarlo a 2001 y guardar, el listado mostró “Álbum actualizado correctamente” y el año 2001 junto al nombre de la banda.

La consulta devolvió la misma fila (id_album 6) con el año actualizado, lo que confirma que se modificó el registro existente y no se creó otro.

Figura 72 : Imagen – Eliminación de un álbum y evidencia en la base de datos





INTERPRETACION

Tras aceptar la confirmación del navegador, el listado volvió a los cuatro álbumes originales y mostró el mensaje de eliminación.

La consulta sobre la tabla album no devolvió filas, por lo que el borrado se aplicó en la base de datos.

7.6.3.3. Mantenimiento 3: Canciones

Figura 73 : Imagen – Validación del álbum y control de pista repetida en CancionController

```
56    @PostMapping("/guardar")
57    public String guardar(@Valid Cancion cancion, BindingResult result, Model model, RedirectAttributes flash) {
58        if (cancion.getAlbum() == null || cancion.getAlbum().getIdAlbum() == null) {
59            result.rejectValue("album", "requerido", "Debe seleccionar un álbum");
60        }
61        if (result.hasErrors()) {
62            model.addAttribute("albumes", albumService.listar());
63            model.addAttribute("modo", cancion.getIdCancion() == null ? "registrar" : "editar");
64            return "form-cancion";
65        }
66        boolean esNueva = cancion.getIdCancion() == null;
67        try {
68            service.save(cancion);
69        } catch (DataAccessException e) {
70            result.rejectValue("numeroPista", "duplicado", "No se pudo guardar. Revisa que la pista no esté repetida en este álbum.");
71            model.addAttribute("albumes", albumService.listar());
72            model.addAttribute("modo", cancion.getIdCancion() == null ? "registrar" : "editar");
73            return "form-cancion";
74        }
75        flash.addFlashAttribute("mensaje",
76                esNueva ? "Canción registrada correctamente." : "Canción actualizada correctamente.");
77        return "redirect:/canciones";
78    }
79
```

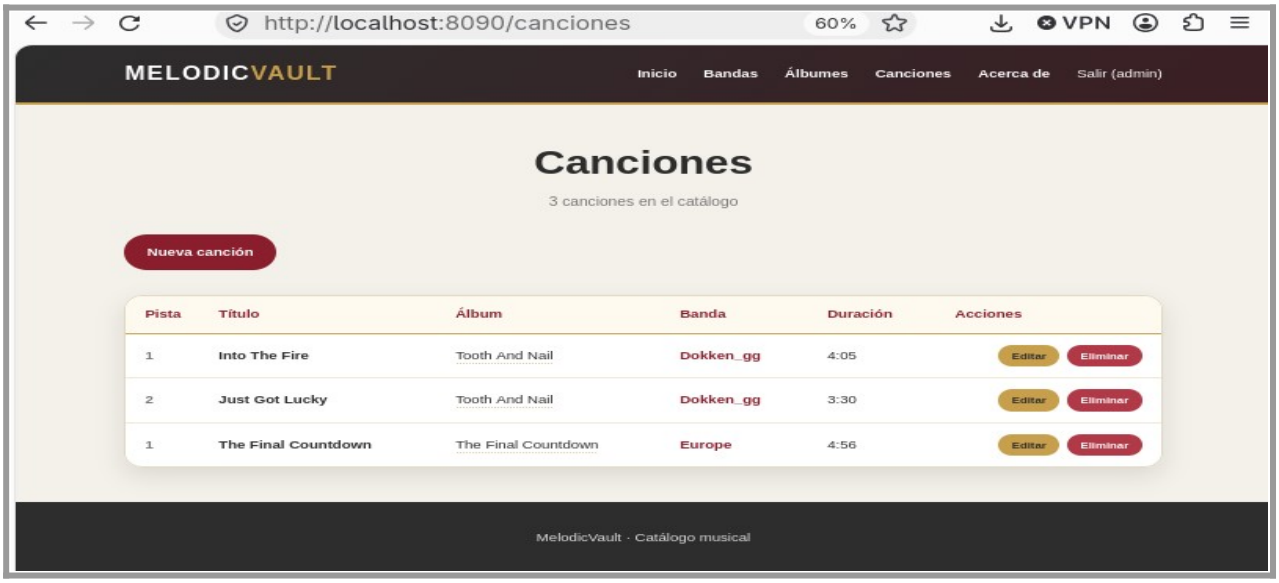
INTERPRETACION

Cuando el usuario dejaba "Selecciona un álbum", Spring igualmente creaba un Album vacío, por lo que @NotNull no lo detectaba y fue necesaria la comprobación manual del identificador.

La unicidad de la pista no la verificó el controlador: la garantizó la base de datos, y DataAccessException se tradujo en un mensaje bajo el campo "Número de pista" que conservó lo escrito.

Solo si ambas comprobaciones pasaban, service.save persistía la canción y el sistema redirigía al listado con el mensaje de éxito.

Figura 74 : Imagen – Pantalla de listado del mantenimiento de Canciones

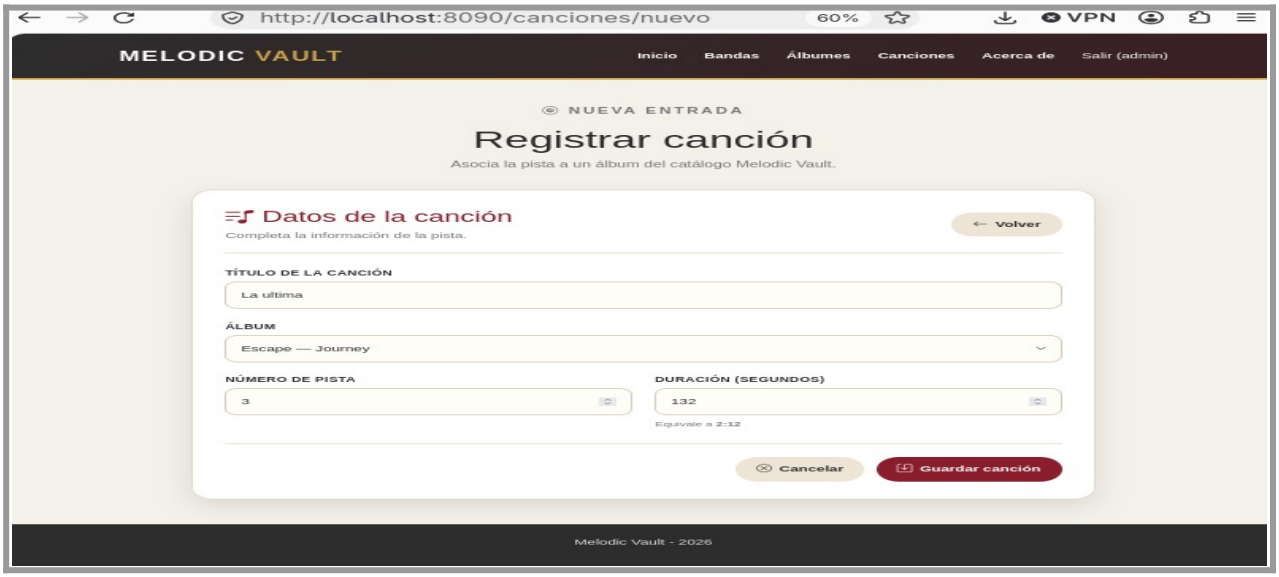


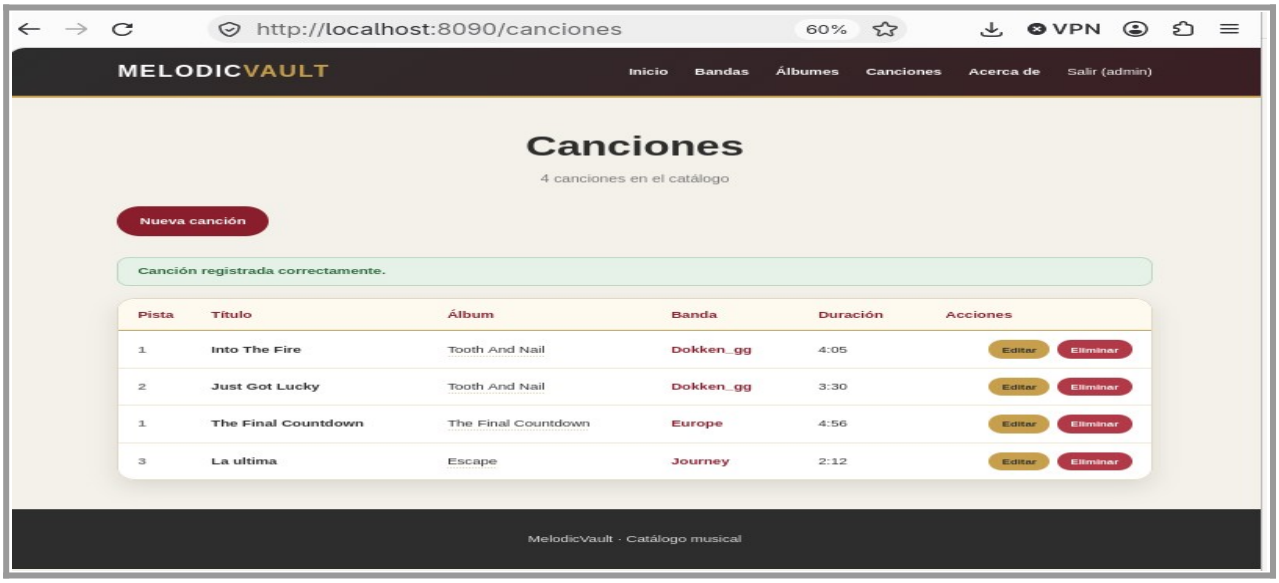
INTERPRETACION

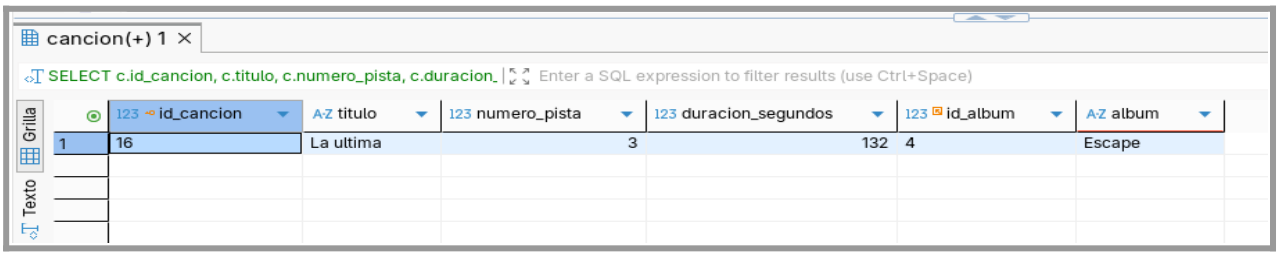
La tabla mostró las tres canciones cargadas por el script con su pista, título, álbum, banda y duración en formato m:ss, junto con el total del catálogo.

La columna Acciones, con Editar y Eliminar, apareció solo por la sesión de Administrador.

Figura 75 : Imagen – Registro de una canción y evidencia en la base de datos



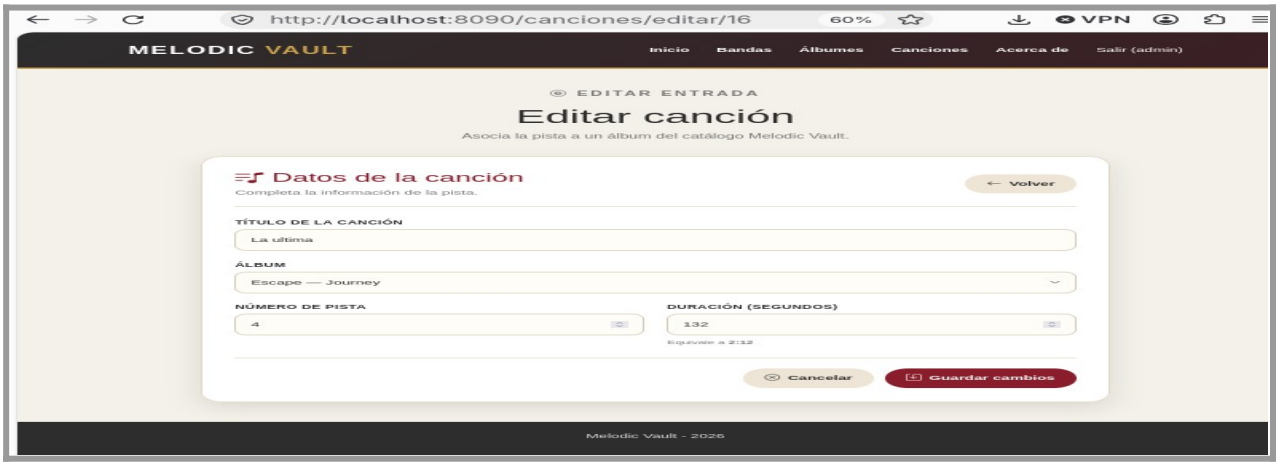


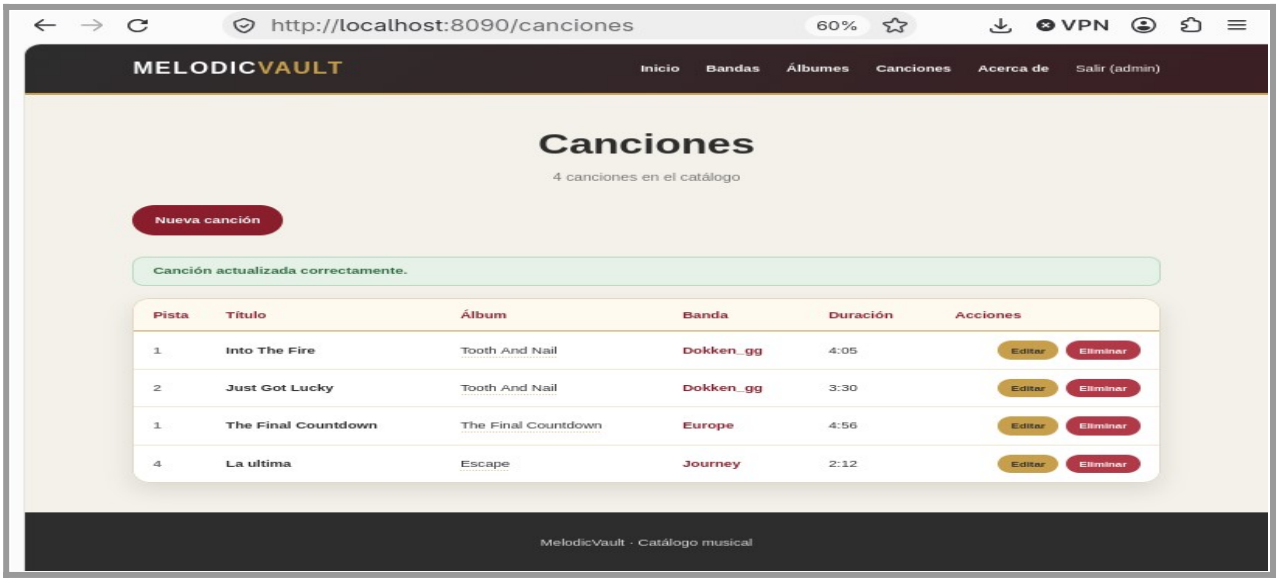


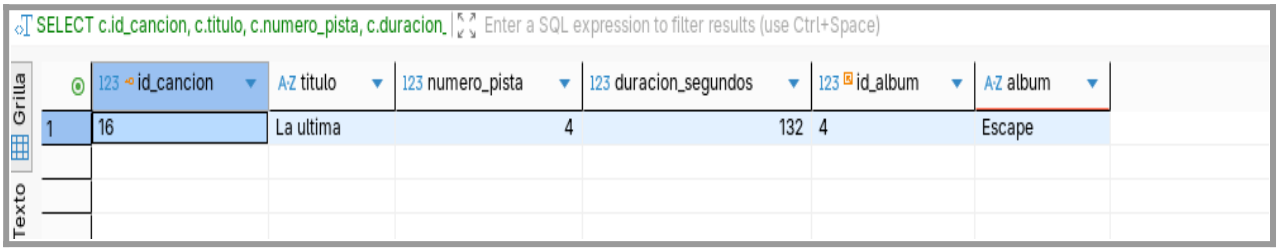
INTERPRETACION

El formulario envió el título, el álbum elegido, el número de pista y la duración en segundos, que el propio formulario convirtió a m:ss mientras se escribía; la canción se insertó y el listado la mostró con esa misma conversión. La consulta con JOIN confirmó la fila en la tabla cancion con el id_album del álbum elegido.

Figura 76 : Imagen – Edición de una canción y evidencia en la base de datos





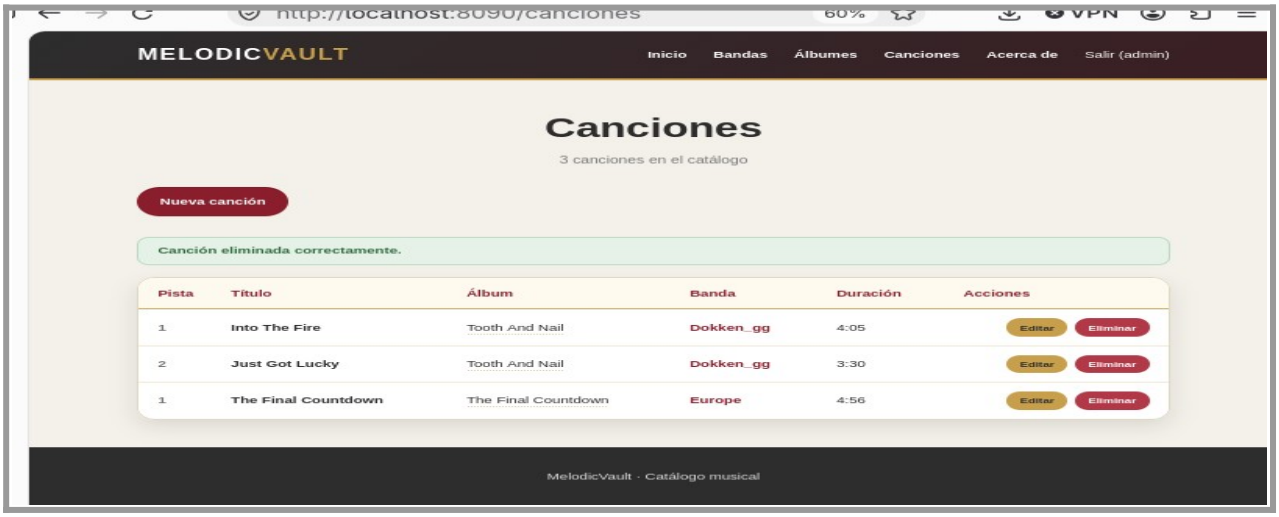


INTERPRETACION

El formulario se abrió con los datos guardados y el título “Editar canción”. Tras cambiar el número de pista de 3 a 4 y guardar, el listado mostró “Canción actualizada correctamente” con la pista nueva, y la consulta devolvió la misma fila, con el mismo id_cancion y el número de pista modificado.

La restricción única (id_album, numero_pista) no se violó porque la pista 4 estaba libre en ese álbum.

Figura 77 : Imagen – Eliminación de una canción y evidencia en la base de datos





INTERPRETACION

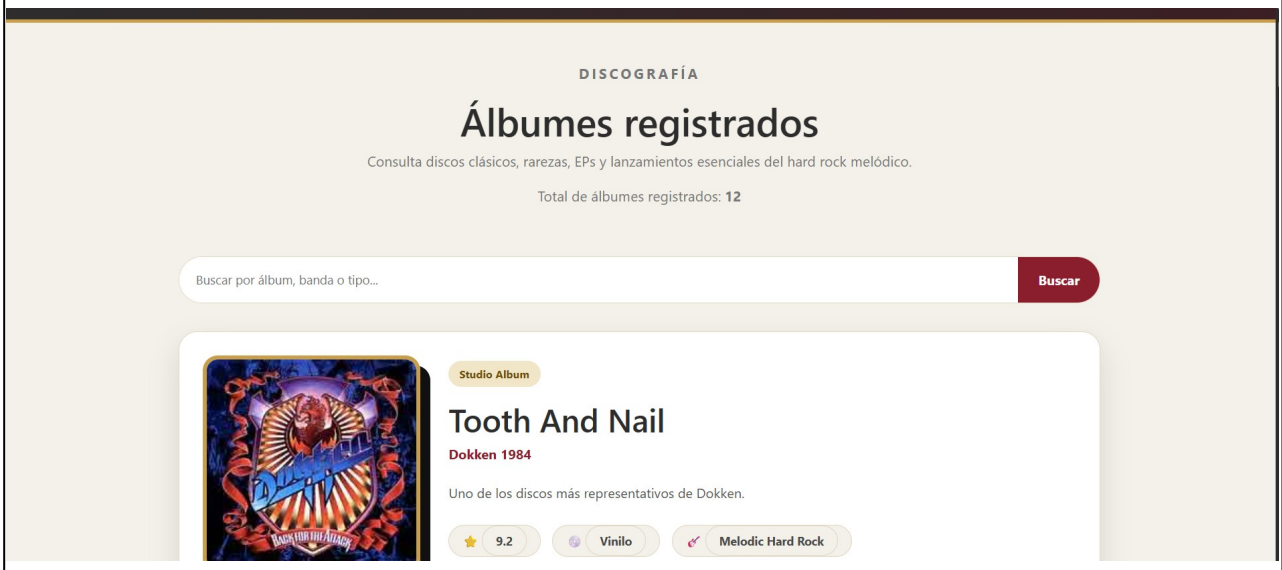
Tras aceptar la confirmación del navegador, el listado volvió a las tres canciones originales y mostró el mensaje de eliminación. La consulta sobre la tabla cancion no devolvió filas, por lo que el borrado se aplicó en la base de datos.

7.6.4. Implementación de consultas

En el sistema se implementaron dos consultas: una para álbumes y otra para bandas. Para su desarrollo se utilizó Spring Data JPA con Hibernate como proveedor de persistencia, lo que permitió reducir el código repetitivo necesario para consultar la base de datos, ya que las consultas se generan automáticamente a partir del nombre de los métodos del repositorio.

A continuación, se presentan las tablas que explican el funcionamiento de cada consulta y muestran la evidencia de su ejecución:

Figura 78 : Consulta de álbumes






```java
@Override  2 usages  Cesar Burgos Villon
public List<Album> buscar(String texto) {
    return data.findByTituloContainingIgnoreCaseOrTipoContainingIgnoreCaseOrBandaNombreContainingIgnoreCase(
            texto, texto, texto
    );
}

@Override  1 usage  Cesar Burgos Villon
public List<Cancion> listadoDeCanciones(int id) { return data.listadoDeCanciones(id); }
```

```java
    @GetMapping({"", "/"})  Cesar Burgos Villon
    public String listar(@RequestParam(value = "buscar", required = false) String buscar, Model model) {

        if (buscar != null && !buscar.trim().isEmpty()) {
            model.addAttribute( attributeName: "albumes", service.buscar(buscar));
            model.addAttribute( attributeName: "total", service.buscar(buscar).size());
            model.addAttribute( attributeName: "buscar", buscar);
        } else {
            model.addAttribute( attributeName: "albumes", service.listar());
            model.addAttribute( attributeName: "total", service.listar().size());
        }

        return "albumes";
    }
```

INTERPRETACION

La barra de búsqueda de álbumes funciona enviando el texto escrito en el campo “buscar” hacia la ruta /albumes, que es atendida por el método listar() del AlbumController.

Este método verifica si el parámetro contiene texto y si es así, invoca service.buscar(buscar); si está vacío, invoca service.listar() y muestra todos los álbumes.

El servicio AlbumService delega la búsqueda en el repositorio de Spring Data JPA mediante el método derivado implementado con SpringData, que genera automáticamente una consulta SQL .

La búsqueda se realiza sobre tres criterios: el título del álbum, su tipo y el nombre de la banda a la que pertenece.

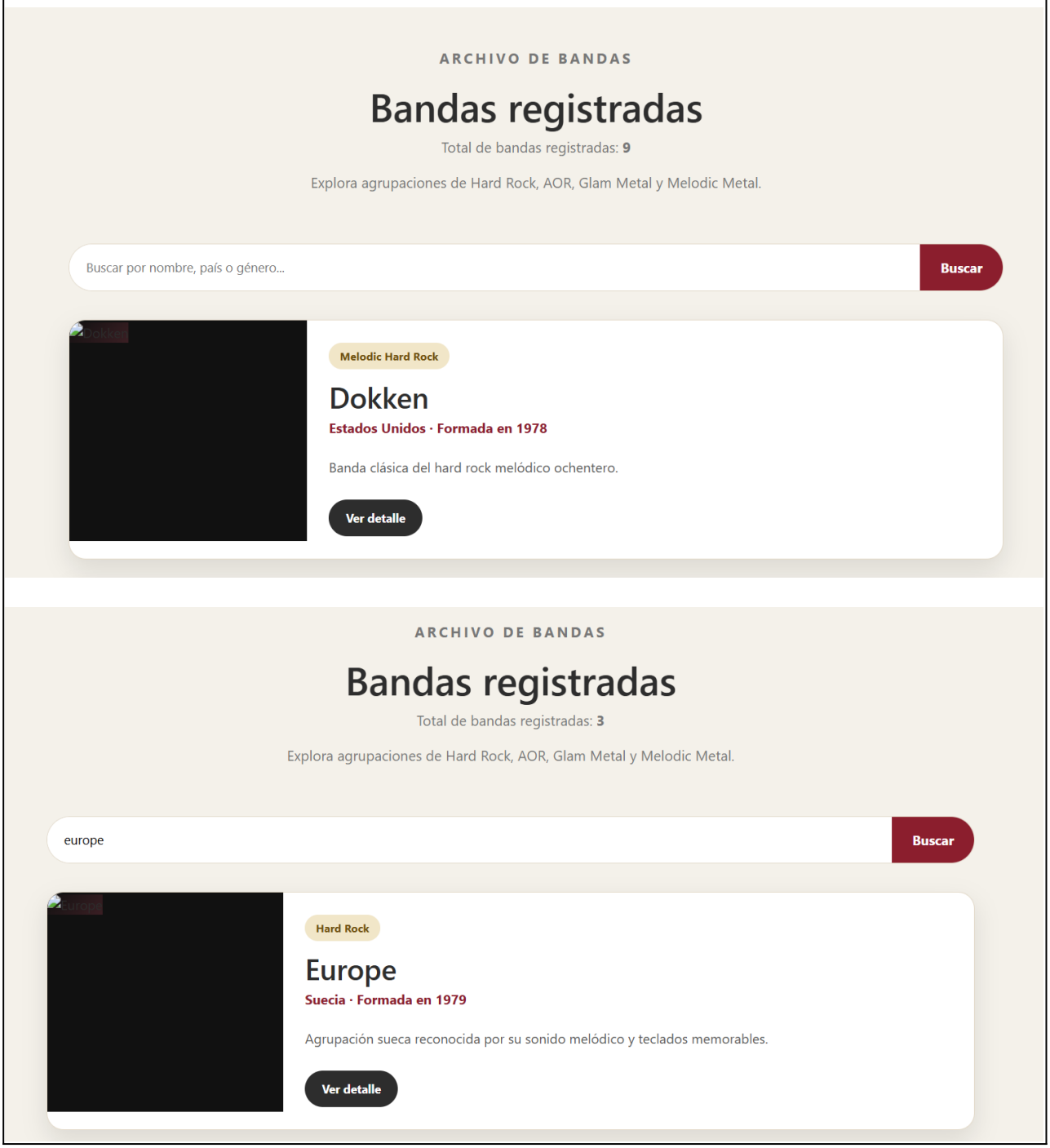
La base de datos devuelve únicamente los álbumes que coinciden en al menos uno de esos campos enviados en forma de texto como parametro, y el controlador los envía a la vista junto con el total de resultados y el texto buscado.

Todo el filtrado ocurre en el backend y la vista solo se encarga de mostrar el resultado.

La consulta SQL se realiza con SpringData y es el equivlente a preguntar el siguiente query en la base de datos:

```
SELECT a.* FROM album a LEFT JOIN banda b ON a.id_banda = b.id_banda WHERE LOWER(a.titulo) LIKE LOWER('%texto%') OR LOWER(a.tipo) LIKE LOWER('%texto%') OR LOWER(b.nombre) LIKE LOWER('%texto%');
```

Figura 79 : Consulta de Bandas



```java
    @Override  1 usage  Cesar Burgos Villon
    public List<Banda> buscar(String texto) {
        return data.findByNombreContainingIgnoreCaseOrPaisContainingIgnoreCaseOrGeneroContainingIgnoreCase(
                texto, texto, texto
        );
    }

    @Override  1 usage  Cesar Burgos Villon
    public Optional<Banda> listarConAlbumes(int id) { return data.findWithAlbumesByIdBanda(id); }
}
```

```java
    List<Banda> findByNombreContainingIgnoreCaseOrPaisContainingIgnoreCaseOrGeneroContainingIgnoreCase(  1 usage  Cesar Burgos Vill
            String nombre, String pais, String genero
    );
}
```

```java
    @GetMapping({"", "/"})  Cesar Burgos Villon
    public String listar(@RequestParam(value = "buscar", required = false) String buscar, Model model) {

        List<Banda> lista;

        if (buscar != null && !buscar.trim().isEmpty()) {
            lista = service.buscar(buscar);
            model.addAttribute( attributeName: "buscar", buscar);
        } else {
            lista = service.listar();
        }

        model.addAttribute( attributeName: "bandas", lista);
        model.addAttribute( attributeName: "total", lista.size());

        return "bandas";
    }
```

```html
<section class="acciones-superiores">
    <form th:action="@{/bandas}" method="get" class="filtro-simple">
        <input type="text"
               name="buscar"
               th:value="${buscar}"
               placeholder="Buscar por nombre, país o género...">
        <button type="submit">Buscar</button>

    </form>
```

```
<section class="acciones-superiores">
    <form th:action="@{/albumes}" method="get" class="filtro-simple">
        <input type="text"
               name="buscar"
               th:value="${buscar}"
               placeholder="Buscar por álbum, banda o tipo...">
        <button type="submit">Buscar</button>

    </form>
```

INTERPRETACION

La barra de búsqueda de bandas funciona mediante un formulario en la vista que envía por el texto escrito en el campo “buscar” hacia la ruta /bandas, atendida por el método listar() del BandaController. Este método verifica si el parámetro contiene texto.

Si es así, invoca service.buscar(buscar); si está vacío, invoca service.listar() y muestra todas las bandas. El servicio BandaService delega la búsqueda en el repositorio IBanda, de Spring Data JPA, mediante el método implementado con SpringData, que genera automáticamente una consulta SQL con coincidencia parcial (LIKE) sin distinguir mayúsculas de minúsculas.

La búsqueda se realiza sobre tres criterios: el nombre de la banda, su país y su género.

La base de datos devuelve únicamente las bandas que coinciden en al menos uno de esos campos, y el controlador las envía a la vista junto con el total de resultados y el texto buscado.

La consulta SQL se realiza con SpringData y es el equivlente a preguntar el siguiente query en la base de datos:

```
SELECT b.* FROM banda b WHERE LOWER(b.nombre) LIKE LOWER('%texto%') OR LOWER(b.pais) LIKE LOWER('%texto%') OR LOWER(b.genero) LIKE LOWER('%texto%');
```

En el sistema se implementaron dos consultas: una para álbumes y otra para bandas.

7.6.5. Implementación del caso transaccional

Estimados estudiantes, en este subpunto deben evidenciar la transacción principal del sistema. Una transacción agrupa varias operaciones que deben ejecutarse como una sola unidad lógica; si todo se realiza correctamente, se confirma la operación, y si ocurre un error, se debe evitar que la información quede incompleta. En el Tema 4 se revisa que una transacción permite agrupar operaciones elementales en una operación mayor mediante confirmación o reversión.

Tabla *45*

Caso transaccional implementado

Elemento	Descripción
Nombre de la transacción	Registrar álbum con canciones.
Actor involucrado	Administrador (único rol con acceso a /albumes/completo).
Tablas involucradas	album y cancion; banda solo se consultó para elegir la banda.
Operación principal	Registrar el álbum (cabecera) y sus canciones (detalle).
Operación complementaria	Vincular cada canción con el álbum recién guardado y validar su número de pista.
Resultado esperado	Álbum y canciones registrados juntos; si una canción falla, no se guarda nada.

INTERPRETACION

La tabla muestra que la transacción abarcó dos tablas con una relación de cabecera y detalle, y que la banda intervino solo como dato de consulta. No hubo actualización de filas

existentes: la operación complementaria fue enlazar y validar las canciones durante el registro, no modificar datos previos

7.6.5.1. Flujo de la transacción

Tabla *46*

Flujo de la transacción Registrar álbum con canciones

Paso	Componente	Acción	Si falla
1	Administrador	Abrió /albumes/completo/nuevo desde el botón Álbum con canciones; el sistema mostró el formulario con los datos del álbum y cinco filas de canciones.	—
2	Administrador	Completó el formulario y pulsó Guardar todo (POST /albumes/completo/guardar).	—
3	AlbumCompletoController	Validó los datos del álbum, ignoró las filas sin título y numeró en orden las canciones sin pista.	Vuelve al formulario con los errores o con "Agrega al menos una canción."; no se toca la base.
4	AlbumTransaccionalService	Inició la transacción e insertó el álbum en album.	Se lanza una excepción y no se guarda nada.
5	AlbumTransaccionalService	Por cada canción, verificó que la pista fuera mayor que 0, la vinculó al álbum e insertó la fila en	Pista inválida o repetida: se lanza una excepción y se revierte todo, incluidos el álbum y las canciones ya

		cancion.	insertadas.
6	Spring y MySQL	Sin excepciones, se confirmó la transacción (commit).	—
7	AlbumCompletoController	Redirigió a /albumes con el mensaje "Álbum y canciones registrados correctamente.".	Si hubo reversión, vuelve a form-album-completo con "No se guardó nada: …" y lo escrito.

INTERPRETACION

El flujo se dividió en dos zonas: los pasos 1 a 3 ocurrieron antes de la transacción y, si fallaban, no tocaron la base; los pasos 4 a 6 ocurrieron dentro de ella. La columna de la derecha muestra que cualquier error dentro de la transacción tuvo el mismo desenlace: la reversión de todo lo insertado hasta ese momento.

Figura 80 :Servicio transaccional AlbumTransaccionalService

```
@Service
public class AlbumTransaccionalService {
    @Autowired private IAlbum albumRepo;
    @Autowired private ICancion cancionRepo;

    @Transactional
    public void registrarAlbumConCanciones(Album album, List<Cancion> canciones) {
        Album guardado = albumRepo.save(album);

        for (Cancion c : canciones) {
            if (c.getDuracionSegundos() == null || c.getDuracionSegundos() < 0)
                throw new IllegalArgumentException("Falta la duración (o es negativa) en: " + c.getTitulo());
            if (c.getTitulo().length() > 100)
                throw new IllegalArgumentException("Hay un título de más de 100 caracteres");

            if (c.getNumeroPista() == null || c.getNumeroPista() <= 0)
                throw new IllegalArgumentException("Pista inválida: " + c.getTitulo());
            c.setAlbum(guardado);
            cancionRepo.save(c);
        }
    }
}
```

INTERPRETACION

@Transactional hizo que el álbum y todas las canciones se ejecutaran dentro de una misma transacción: se confirmaba al terminar el método y se revertía si este lanzaba una excepción.

El álbum se guardó primero porque cada canción necesitaba su id_album como clave foránea. Una pista inválida (IllegalArgumentException) o repetida (restricción UNIQUE (id_album, numero_pista)) interrumpía el ciclo y deshacía también el álbum y las canciones previas.

La anotación utilizada fue la de spring , que revierte ante excepciones no comprobadas como las dos anteriores

Figura 81 :Preparación de los datos y manejo de errores en AlbumCompletoController

```
@PostMapping("/guardar")
public String guardar(@Valid @ModelAttribute("form") AlbumCompletoForm form,
                      BindingResult result, Model model, RedirectAttributes flash) {

    // Ignora las filas vacias y numera automáticamente las que no traen pista
    List<Cancion> validas = new ArrayList<>();
    int pista = 1;
    for (Cancion c : form.getCanciones()) {
        if (c.getTitulo() == null || c.getTitulo().isBlank()) continue;
        if (c.getNumeroPista() == null) c.setNumeroPista(pista);
        validas.add(c);
        pista++;
    }

    if (result.hasErrors() || validas.isEmpty()) {
        if (validas.isEmpty()) model.addAttribute("error", "Agrega al menos una canción.");
        model.addAttribute("bandas", bandaService.listar());
        return "form-album-completo";
    }

    try {
        transaccionalService.registrarAlbumConCanciones(form.getAlbum(), validas);
    } catch (IllegalArgumentException e) {
        model.addAttribute("error", "No se guardó nada: " + e.getMessage());
        model.addAttribute("bandas", bandaService.listar());
        return "form-album-completo";
    } catch (DataAccessException e) {
        model.addAttribute("error", "No se guardó nada: revisa que no haya pistas repetidas en el ál
        model.addAttribute("bandas", bandaService.listar());
        return "form-album-completo";
    }

    flash.addFlashAttribute("mensaje", "Álbum y canciones registrados correctamente.");
    return "redirect:/albumes";
}
```

INTERPRETACION

El controlador preparó los datos antes de abrir la transacción: ignoró las filas vacías, numeró las canciones sin pista y comprobó que hubiera al menos una, de modo que los errores de formulario no llegaron a la base.

Las excepciones del servicio llegaban al controlador cuando la transacción ya se había revertido, por lo que el mensaje "No se guardó nada" era cierto

El mensaje por pista repetida fue fijo para no mostrar al usuario el texto técnico del error SQL.

7.6.5.2. Datos registrados y actualizados

Tabla *47*

Datos registrados y actualizados por la transacción

Tabla	Operación	Datos	Origen
album	Registro de 1 fila	Título, banda (id_banda), año, tipo, rating, formato y URL de portada.	Formulario; la banda se eligió de la lista cargada desde banda.
cancion	Registro de 1 a 5 filas	Título, número de pista, duración en segundos y álbum (id_album).	Filas del formulario con título; id_album fue el id del álbum recién insertado y la pista se numeró automáticamente si no se escribió.
banda	Solo lectura	—	Se consultó para llenar la lista desplegable; no se modificó.
Datos actualizados	Ninguno	—	La transacción solo insertó filas nuevas; no modificó ninguna fila existente.

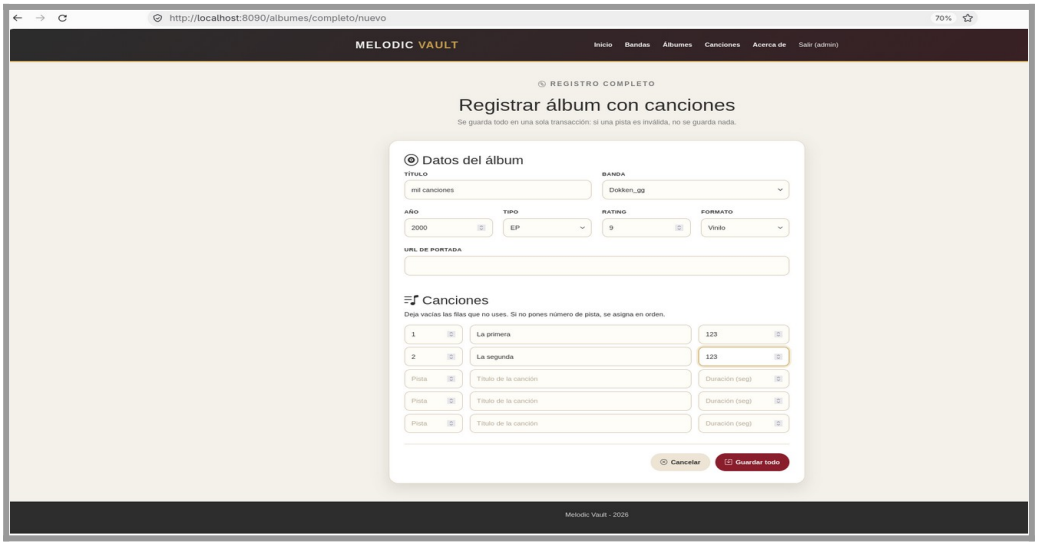
INTERPRETACION

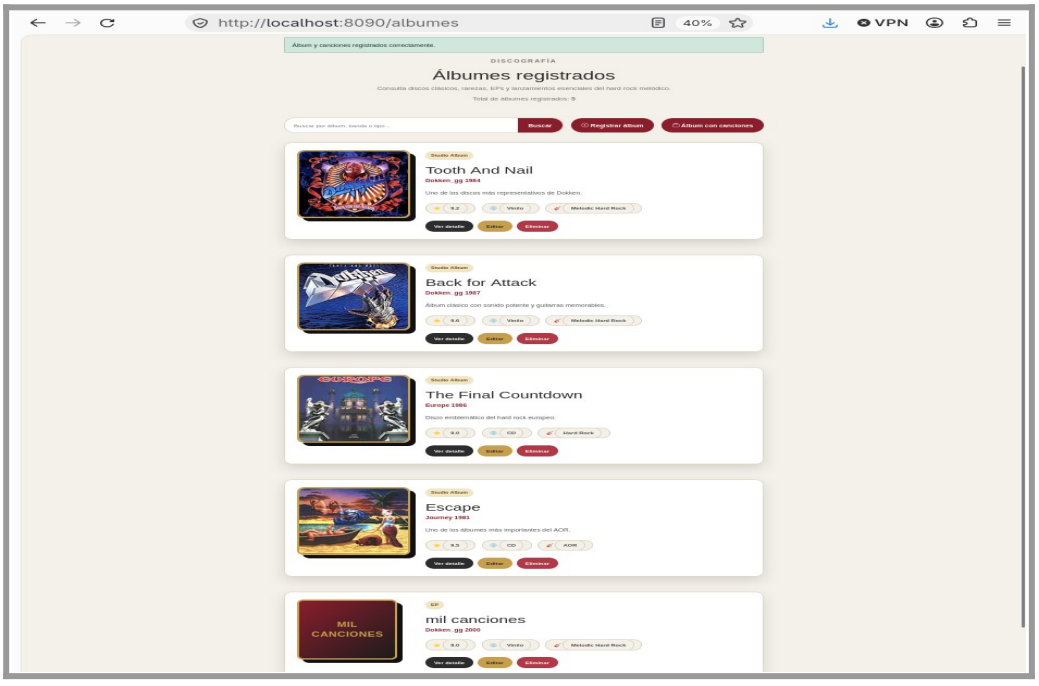
La transacción insertó una fila en album y de una a cinco en cancion, y el id_album de cada canción fue el identificador que la base generó al insertar el álbum.

No se actualizó ningún dato existente: el caso se justificó por la dependencia entre cabecera y detalle, no por una modificación posterior.

7.6.5.3. Evidencia de ejecución y en base de datos

Figura 82 :Registro de un álbum con canciones: formulario y resultado en el listado



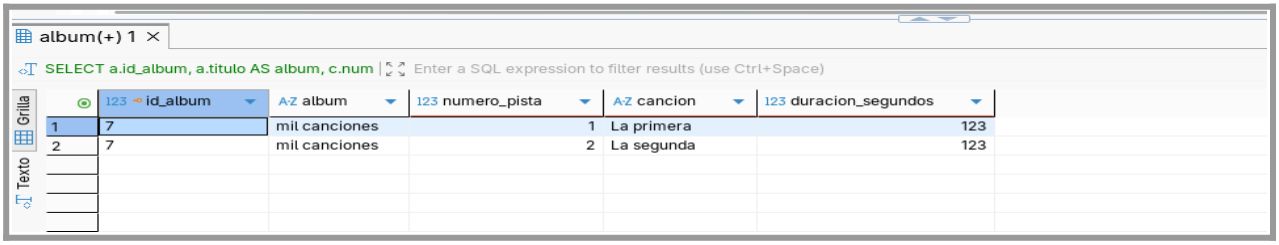


INTERPRETACION

El formulario reunió los datos del álbum y 2 de las cinco filas de canciones; las 3 filas vacías se ignoraron

Al pulsar Guardar todo, el sistema redirigió al listado con el mensaje de éxito y el nuevo álbum.

Figura 83 :Registro de un álbum con canciones: formulario y resultado en el listado

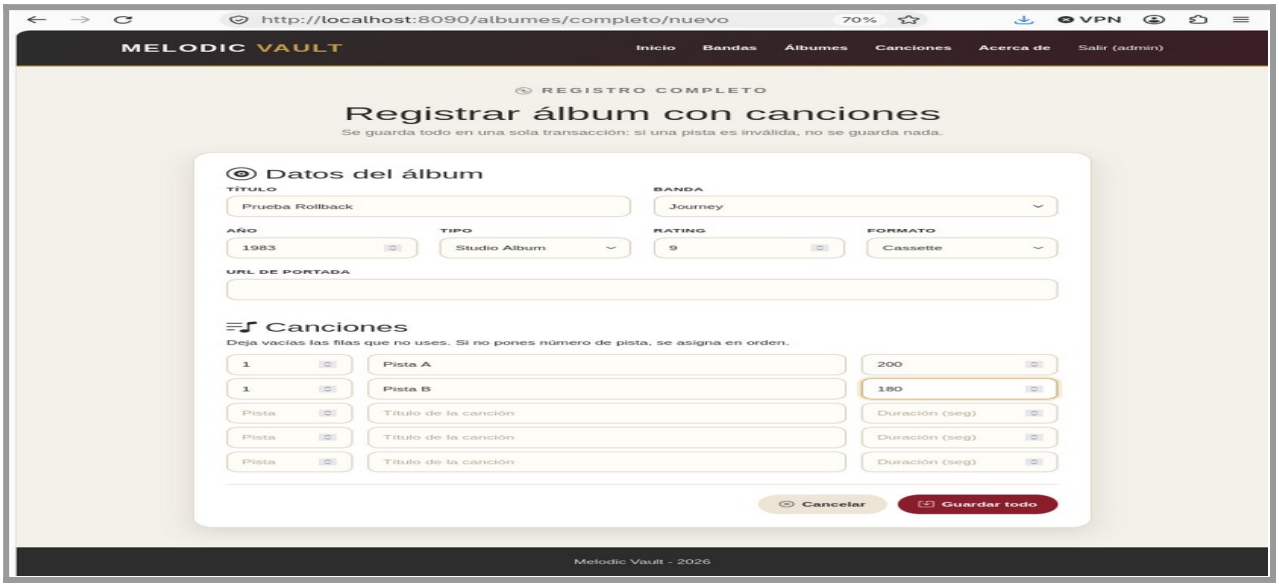


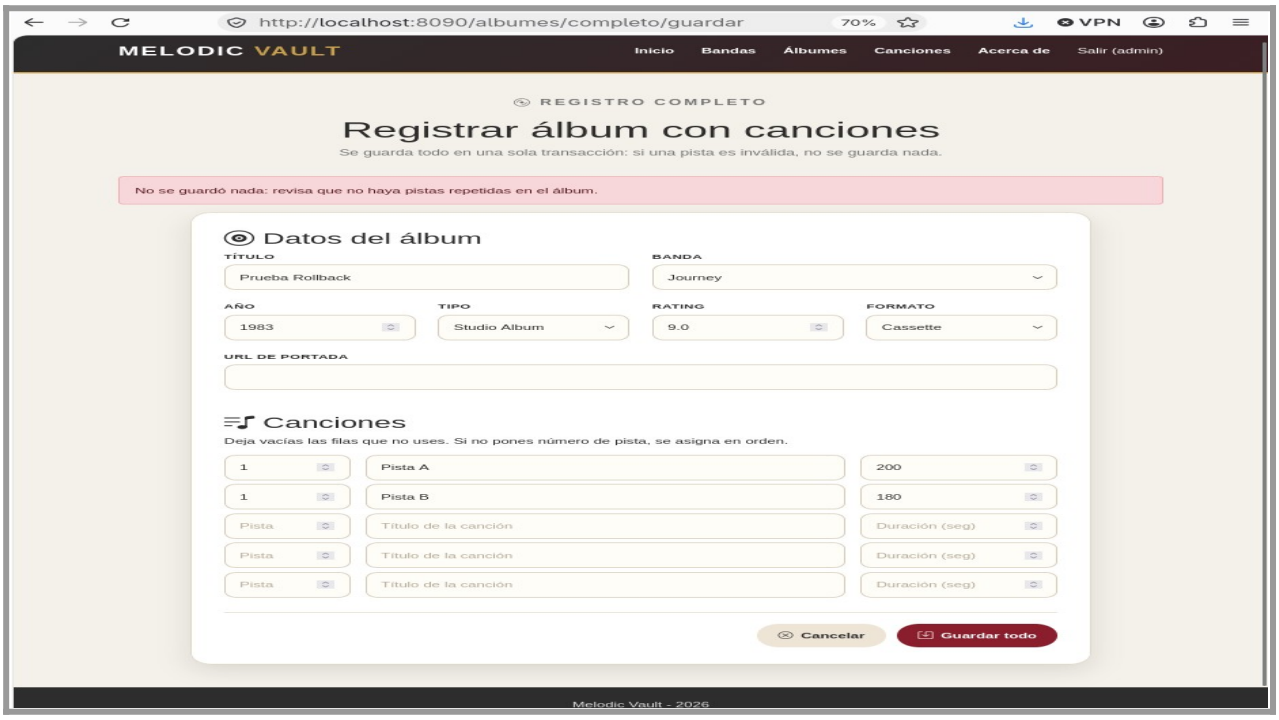
INTERPRETACION

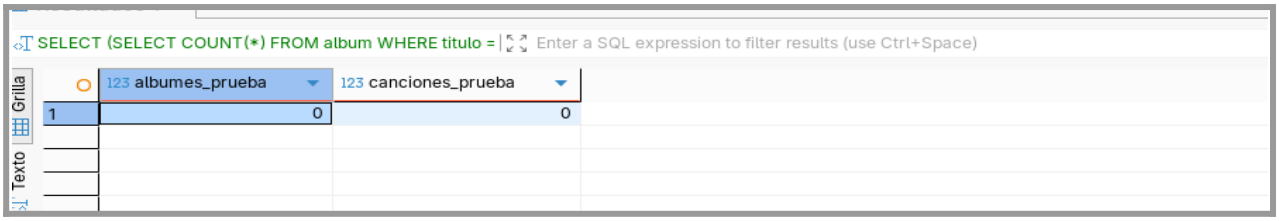
La consulta devolvió dos filas con el mismo nombre de el álbum que definimos y sus dos canciones quedaron guardados juntos.

Los números de pista aparecieron como 1 y 2

Figura 84 :Registro de un álbum con canciones: formulario y resultado en el listado


http://localhost:8090/albumes/completo/nuevo
MELODIC VAULT
Inicio
Bandas
Álbumes
Canciones
Acerca de
Salir (admin)
REGISTRO COMPLETO
Registrar álbum con canciones
Se guarda todo en una sola transacción: si una pista es inválida, no se guarda nada.
Datos del álbum
TÍTULO
Prueba Rollback
BANDA
Journey
AÑO
1983
TIPO
Studio Album
RATING
9
FORMATO
Cassette
URL DE PORTADA
Canciones
Deja vacías las filas que no uses. Si no pones número de pista, se asigna en orden.
1
Pista A
200
1
Pista B
180
Cancelar
Guardar todo
Melodic Vault - 2026


http://localhost:8090/albumes/completo/guardar
MELODIC VAULT
Inicio
Bandas
Álbumes
Canciones
Acerca de
Salir (admin)
REGISTRO COMPLETO
Registrar álbum con canciones
Se guarda todo en una sola transacción: si una pista es inválida, no se guarda nada.
No se guardó nada: revisa que no haya pistas repetidas en el álbum.
Datos del álbum
TÍTULO
Prueba Rollback
BANDA
Journey
AÑO
1983
TIPO
Studio Album
RATING
9.0
FORMATO
Cassette
URL DE PORTADA
Canciones
Deja vacías las filas que no uses. Si no pones número de pista, se asigna en orden.
1
Pista A
200
1
Pista B
180
Cancelar
Guardar todo
Melodic Vault - 2026


SELECT (SELECT COUNT(*) FROM album WHERE titulo =
Enter a SQL expression to filter results (use Ctrl+Space)
Grilla
Texto
123 albumes_prueba
123 canciones_prueba
1
0
0

INTERPRETACION

Se envió un álbum con dos canciones con la pista 1; la primera se insertó, pero la segunda violó la restricción única (id_album, numero_pista) y el sistema revirtió la transacción.

El formulario conservó lo escrito y mostró el mensaje de error, y la consulta confirmó que no quedó ni el álbum ni ninguna de sus canciones en la base de datos.

7.7. Implementación de reportes con JasperReports

En MelodicVault se implementaron dos reportes tabulares en formato PDF con JasperReports 6.20.6: uno con las canciones de un álbum y otro con los álbumes de una banda.

Ambos se alimentaron con datos leídos de MySQL en el momento de la petición, se definieron en plantillas .jrxml ubicadas en src/main/resources/reportes/ y se descargaron desde el botón

Descargar PDF de las pantallas de detalle, disponible para cualquier usuario con sesión iniciada (Lector o Administrador).

Tabla *48*

Reportes implementados en el sistema

Reporte	Tipo de reporte	Datos utilizados	Evidencia
Canciones de un álbum	Tabular / PDF	Título del álbum y nombre de su banda; canciones con número de pista, título y duración (album, banda, cancion).	Capturas del diseño y del PDF generado desde la aplicación
Álbumes de una banda	Tabular / PDF	Nombre, país, género y año de formación de la banda; álbumes con año, título, tipo y rating	Capturas del diseño y del PDF generado desde la

		(banda, album).	aplicación

INTERPRETACION

La tabla muestra que los dos reportes fueron tabulares y se exportaron a PDF, y que cada uno combinó datos de cabecera (el álbum o la banda) con las filas de una tabla relacionada.

La evidencia de cada reporte incluyó el diseño de la plantilla, el PDF ejecutado desde la aplicación y la consulta en MySQL que respalda los datos mostrados.

7.7.1. Implementación de reportes con JasperReports

Tabla *49*

Estructura de las plantillas .jrxml

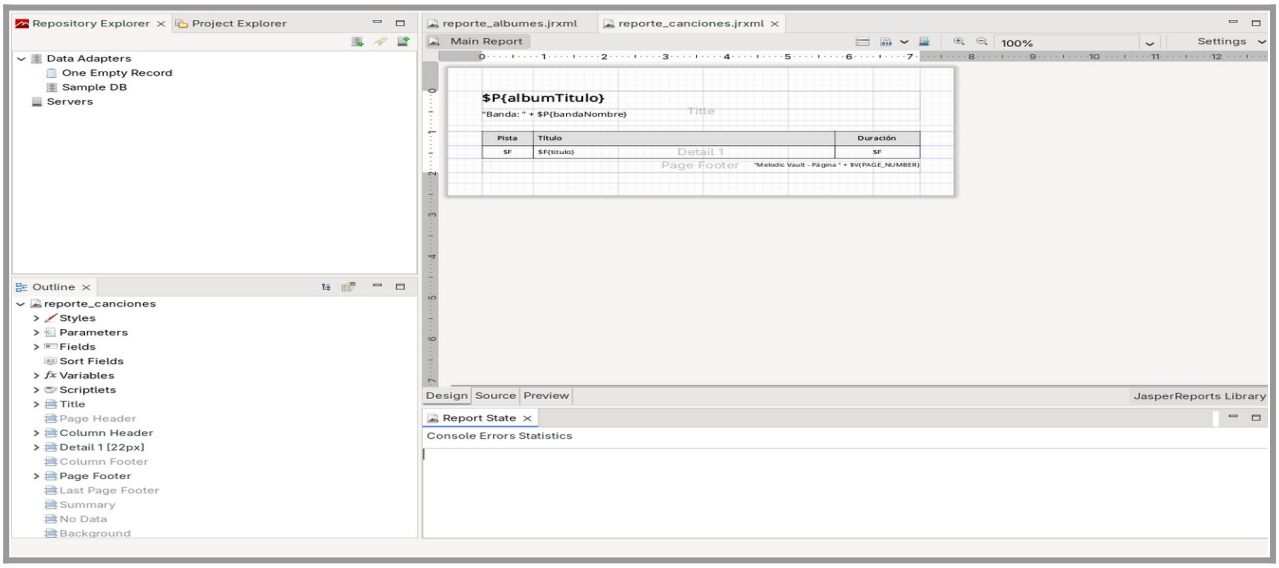
Elemento	reporte_canciones.jrxml	reporte_albumes.jrxml
Página	A4 (595 × 842), márgenes de 40.	A4 (595 × 842), márgenes de 40.
Parámetros	albumTitulo, bandaNombre.	bandaNombre, pais, genero, anioFormacion.
Campos	numeroPista, titulo, duracionSegundos.	anio, titulo, tipo, rating.
Title	Título del álbum y "Banda: nombre".	Nombre de la banda y "país · género · Formada en año".
Column Header	Pista, Título, Duración.	Año, Título, Tipo, Rating.
Detail	Una fila por canción; duración en m:ss, o - si no tiene.	Una fila por álbum; rating con un decimal, o - si no tiene.
Page Footer	"Melodic Vault - Página N".	"Melodic Vault - Página N".

INTERPRETACION

Las dos plantillas compartieron la misma estructura de bandas y el mismo estilo de celda; cambiaron los parámetros de la cabecera, las columnas y los campos de cada fila.

Los nombres de los campos coincidieron con los atributos de las entidades Cancion y Album, que es lo que permitió llenar la tabla directamente con listas de objetos.

Figura 85 : Diseño del reporte de canciones de un álbum en Jaspersoft Studio

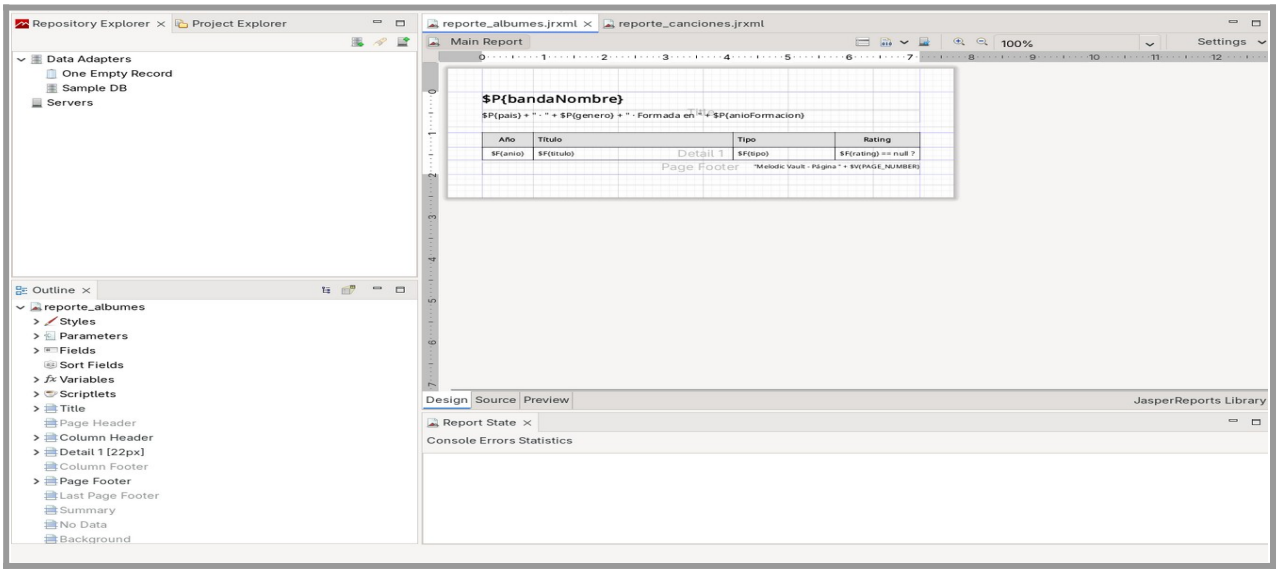


INTERPRETACION

La vista Design mostró las bandas Title, con el título del álbum y su banda, Column Header, con las columnas Pista, Título y Duración, Detail, con la fila que se repite por cada canción, y Page Footer, con la numeración de página.

Las expresiones $P{...} y $F{...} indican los datos que la aplicación entregó al llenar el reporte.

Figura 86 : Diseño del reporte de álbumes de una banda en Jaspersoft Studio



INTERPRETACION

La plantilla mostró la misma estructura de bandas, con una cabecera de tres líneas de datos de la banda y una tabla de cuatro columnas: Año, Título, Tipo y Rating.

Las expresiones de la cabecera usaron los cuatro parámetros de la banda y las de la tabla, los cuatro campos del álbum.

Figura 87 : Parámetros, campos y expresión de duración de reporte_canciones.jrxml

reporte_albumes.jrxml | reporte_canciones.jrxml ×

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!-- Created with Jaspersoft Studio version 6.21.5.final using JasperReports Library version 6.21.5-74d586df47b25dbd05bd
3 <jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports" xmlns:xsi="http://www.w3.org/2001/XMLSchema-ins
4     <style name="celda">
5         <box leftPadding="6" rightPadding="6">
6             <pen lineWidth="0.5" lineColor="#000000"/>
7         </box>
8     </style>
9     <parameter name="albumTitulo" class="java.lang.String"/>
10    <parameter name="bandaNombre" class="java.lang.String"/>
11    <field name="numeroPista" class="java.lang.Integer"/>
12    <field name="titulo" class="java.lang.String"/>
13    <field name="duracionSegundos" class="java.lang.Integer"/>
```

reporte_albumes.jrxml | reporte_canciones.jrxml ×

```
55        </band>
56    </columnHeader>
57    <detail>
58        <band height="22">
59            <textField>
60                <reportElement style="celda" x="0" y="0" width="60" height="22" uuid="e0810473-7ba3-44ac-8e72-b20f8e148050"/>
61                <textElement textAlignment="Center" verticalAlignment="Middle"/>
62                <textFieldExpression><![CDATA[$F{numeroPista}]]></textFieldExpression>
63            </textField>
64            <textField>
65                <reportElement style="celda" x="60" y="0" width="355" height="22" uuid="b25dcbcf-c00f-4c20-806c-e7a4a11bfbe5"/>
66                <textElement verticalAlignment="Middle"/>
67                <textFieldExpression><![CDATA[$F{titulo}]]></textFieldExpression>
68            </textField>
69            <textField>
70                <reportElement style="celda" x="415" y="0" width="100" height="22" uuid="49dbcae6-df10-46c4-ad57-2e63824304f8"/>
71                <textElement textAlignment="Center" verticalAlignment="Middle"/>
72                <textFieldExpression><![CDATA[$F{duracionSegundos} == null ? "-" :
73        ($F{duracionSegundos} / 60) + ":" + String.format("%02d", $F{duracionSegundos} % 60)]]></textFieldExpression>
74            </textField>
75        </band>
76    </detail>
```

INTERPRETACION

Los parámetros recibieron desde Java los datos de cabecera, y los campos tomaron cada valor de los atributos del mismo nombre de la clase Cancion; si se renombra un atributo, también debe renombrarse el campo.

La expresión convirtió los segundos guardados en la base a minutos y segundos, de modo que 245 se mostró como 4:05.

Figura 88 : Atributo de datos vacíos, parámetros, campos y expresión de rating de reporte_albumes.jrxml

reporte_albumes.jrxml × reporte_canciones.jrxml

```
et/xsd/jasperreport.xsd" name="reporte_albumes" pageWidth="595" pageHeight="842" whenNoDataType="AllSectionsNoDetail" columnWidth="515" leftMar
```

reporte_albumes.jrxml × reporte_canciones.jrxml

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Created with Jaspersoft Studio version 6.21.5.final using JasperReports Library version 6.21.5-74d586df47b25dbd05bd0957999819196e59
<jasperReport xmlns="http://jasperreports.sourceforge.net/jasperreports" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schem
    <style name="celda">
        <box leftPadding="6" rightPadding="6">
            <pen lineWidth="0.5" lineColor="#000000"/>
        </box>
    </style>
    <parameter name="bandaNombre" class="java.lang.String"/>
    <parameter name="pais" class="java.lang.String"/>
    <parameter name="genero" class="java.lang.String"/>
    <parameter name="anioFormacion" class="java.lang.Integer"/>
    <field name="anio" class="java.lang.Integer"/>
    <field name="titulo" class="java.lang.String"/>
    <field name="tipo" class="java.lang.String"/>
    <field name="rating" class="java.lang.Number"/>
```

reporte_albumes.jrxml × reporte_canciones.jrxml

```
                <textFieldExpression><![CDATA[$F{anio}]]></textFieldExpression>
            </textField>
            <textField>
                <reportElement style="celda" x="60" y="0" width="235" height="22" uuid="59fb73c7-6d2c-4be4-8a6a-2cfbed98ee73"/>
                <textElement verticalAlignment="Middle"/>
                <textFieldExpression><![CDATA[$F{titulo}]]></textFieldExpression>
            </textField>
            <textField>
                <reportElement style="celda" x="295" y="0" width="120" height="22" uuid="5c227cc4-5036-4232-b4bf-6086ee85cc62"/>
                <textElement verticalAlignment="Middle"/>
                <textFieldExpression><![CDATA[$F{tipo}]]></textFieldExpression>
            </textField>
            <textField>
                <reportElement style="celda" x="415" y="0" width="100" height="22" uuid="69ec2ee3-6e55-43b4-a1f9-4354bbc0aea6"/>
                <textElement textAlignment="Center" verticalAlignment="Middle"/>
                <textFieldExpression><![CDATA[$F{rating} == null ? "-" : String.format("%.1f", $F{rating})]]></textFieldExpression>
            </textField>
        </band>
    </detail>
    <pageFooter>
        <band height="25">
            <textField>
                <reportElement x="0" y="5" width="515" height="20" uuid="ca9164ff-db5b-4609-859b-3cab632dbbe4"/>
                <textElement textAlignment="Right">
                    <font size="9"/>
                </textElement>
                <textFieldExpression><![CDATA["Melodic Vault - Página " + $V{PAGE_NUMBER}]]></textFieldExpression>
            </textField>
        </band>
    </pageFooter>
</jasperReport>
```

INTERPRETACION

whenNoDataType="AllSectionsNoDetail" hizo que una banda sin álbumes igualmente generara un PDF con el título y la cabecera de la tabla, en lugar de un error.

Los cuatro parámetros alimentaron la cabecera con los datos de la banda, y los cuatro campos llenaron cada fila de la tabla con los datos del álbum.

7.7.2. Implementación de reportes con JasperReports

Figura 89 : Compilación y generación del reporte de canciones en ReporteService

```
51
52      public byte[] exportarCancionesAlbum(int idAlbum) throws JRException {
53          Album album = albumService.listarId(idAlbum)
54                  .orElseThrow(() -> new IllegalArgumentException("El álbum no existe"));
55
56          List<Cancion> canciones = albumService.listadoDeCanciones(idAlbum);
57
58          Map<String, Object> params = new HashMap<>();
59          params.put("albumTitulo", album.getTitulo());
60          params.put("bandaNombre", album.getBanda().getNombre());
61
62          JasperPrint print = JasperFillManager.fillReport(
63                  reporteCanciones, params, new JRBeanCollectionDataSource(canciones));
64
65          return JasperExportManager.exportReportToPdf(print);
66      }
```

INTERPRETACION

El método consultó el álbum y sus canciones en la base de datos y entregó a JasperFillManager.fillReport tres elementos: la plantilla ya compilada, un mapa de parámetros con los datos de cabecera (título del álbum y nombre de la banda) y un JRBeanCollectionDataSource con la lista de canciones de ese álbum, que alimentó las filas de la tabla.

El resultado fue un JasperPrint, que se exportó a PDF en bytes. exportarAlbumesBanda siguió el mismo flujo con los álbumes de la banda, que se cargaron con listarConAlbumes en una sola consulta.

Si el álbum no existía, se lanzaba una excepción en lugar de generar un reporte vacío.

Figura 90 : Ruta del reporte de canciones en ReporteController

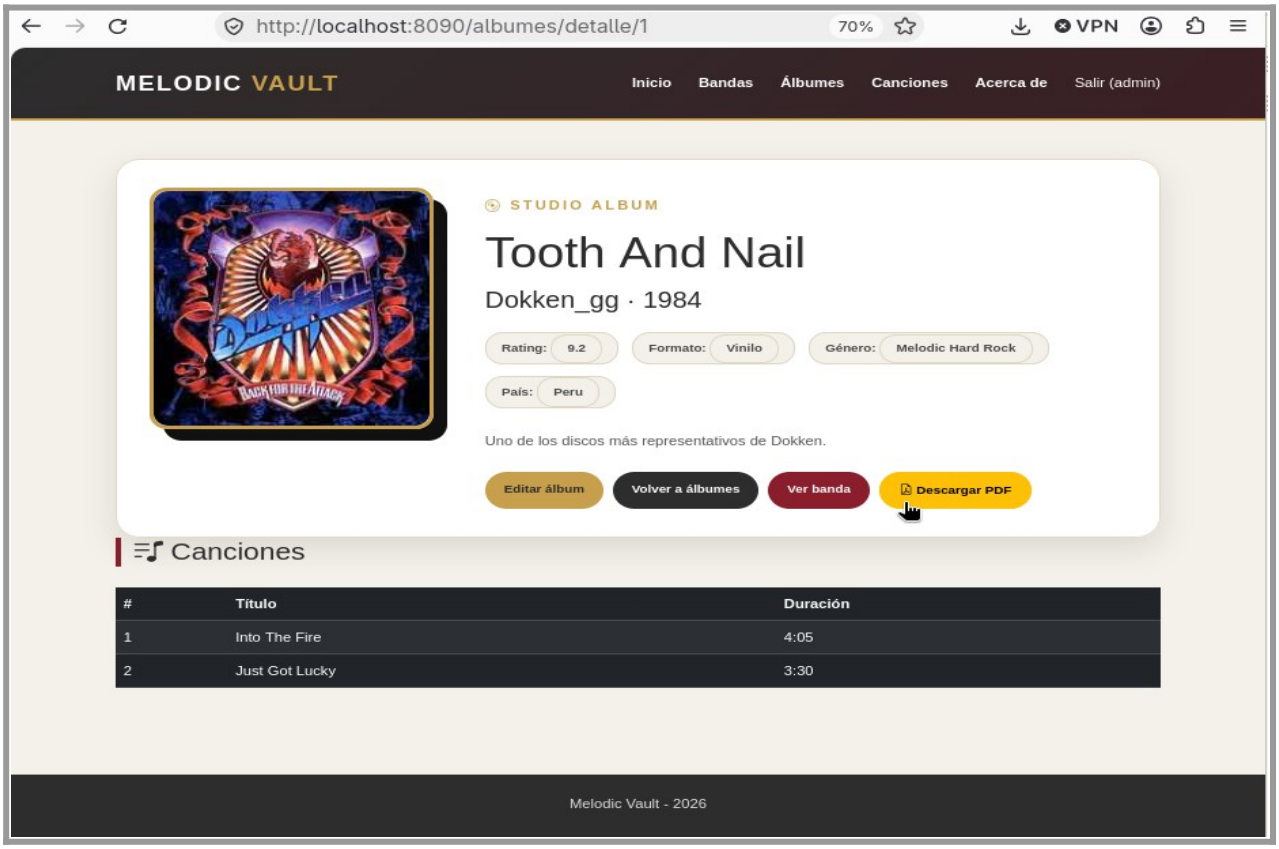
```
16 @Controller
17 @RequestMapping("/reportes")
18 public class ReporteController {
19
20     @Autowired private ReporteService reporteService;
21     @Autowired private IBandaService bandaService;
22
23     @GetMapping("/album/{id}")
24     public ResponseEntity<byte[]> reporteAlbum(@PathVariable int id) throws JRException {
25         byte[] pdf = reporteService.exportarCancionesAlbum(id);
26         return ResponseEntity.ok()
27                 .header(HttpHeaders.CONTENT_DISPOSITION, "inline; filename=canciones_album_" + id + ".pdf")
28                 .contentType(MediaType.APPLICATION_PDF)
29                 .body(pdf);
30     }
31
```

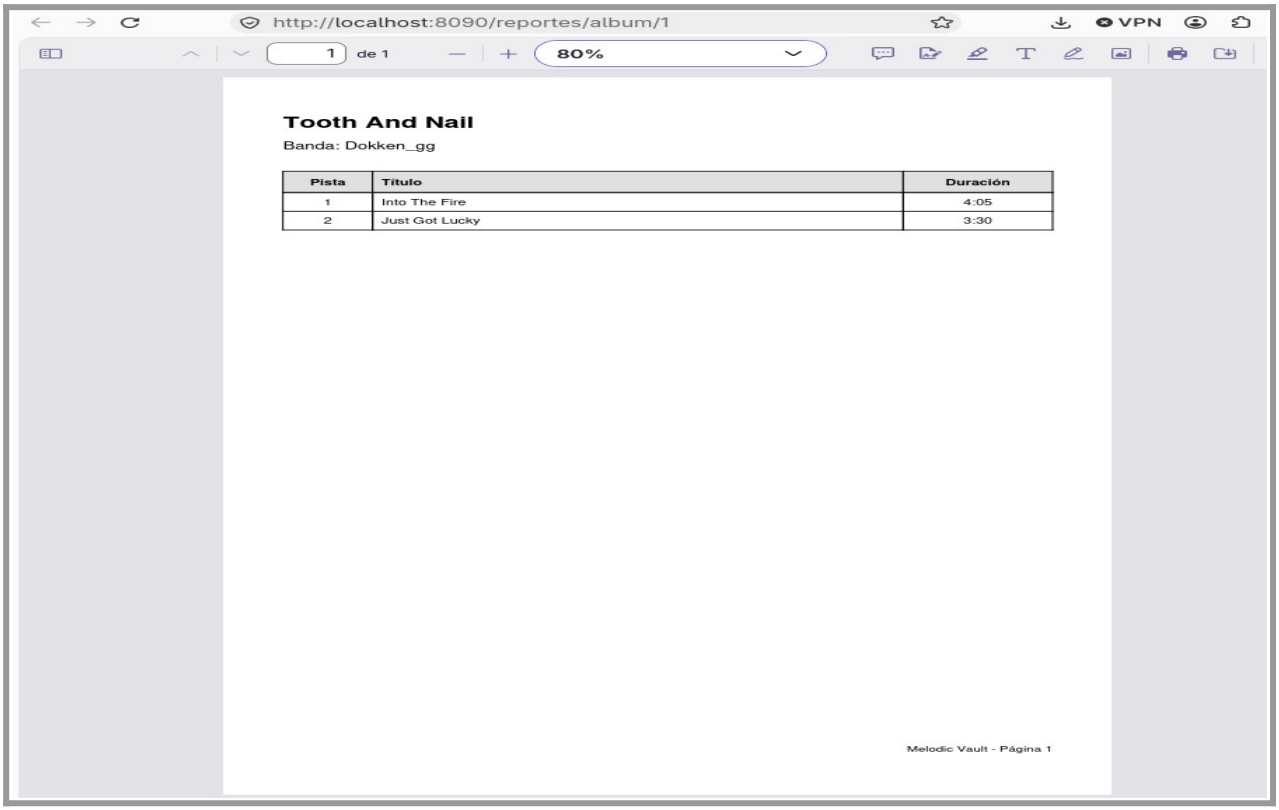
INTERPRETACION

El controlador solo recibió el identificador, delegó la generación en el servicio y devolvió los bytes del PDF. inline hizo que el navegador mostrara el documento en lugar de forzar la descarga, y desde esa vista el usuario pudo guardarlo o imprimirlo.

El reporte de la banda se sirvió de la misma forma en /reportes/banda/{id}.

Figura 91 : Reporte de canciones de un álbum ejecutado desde la aplicación



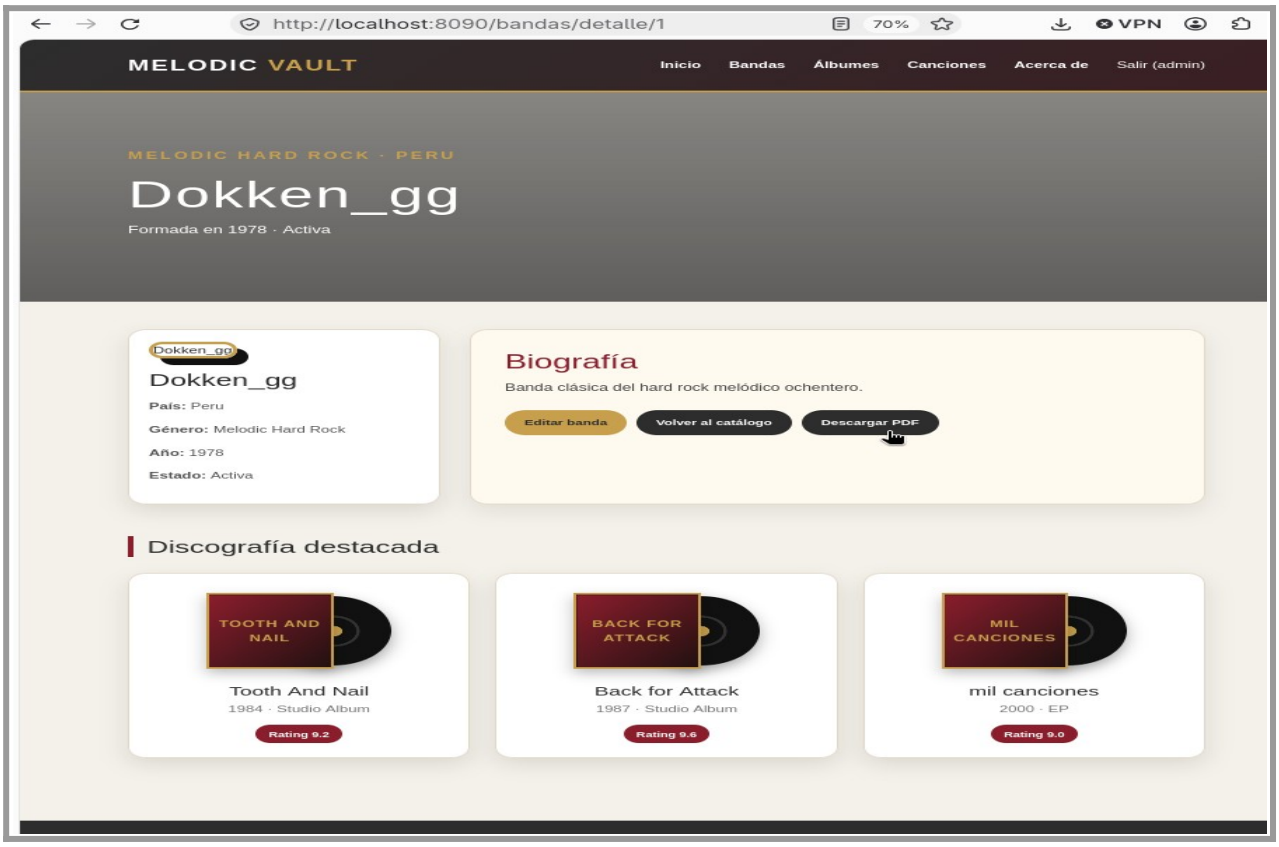


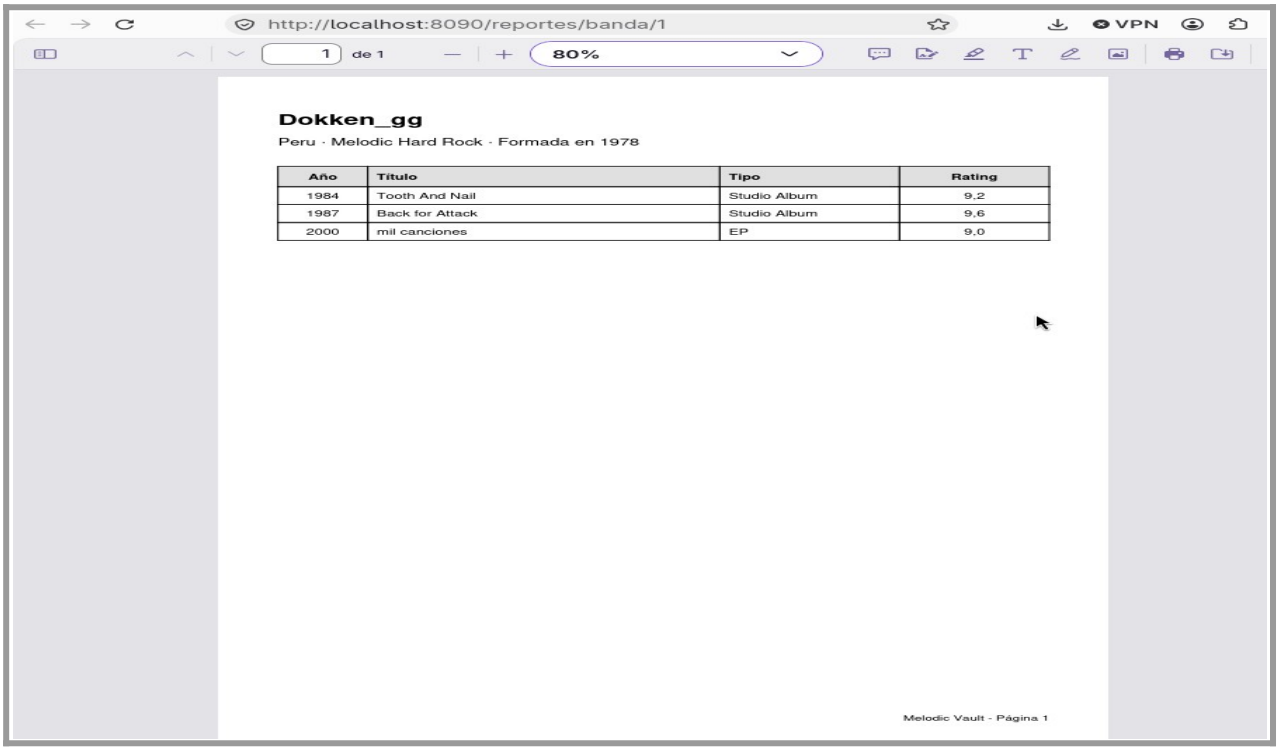
INTERPRETACION

. El botón Descargar PDF del detalle del álbum abrió en una pestaña nueva el reporte con el título del álbum, su banda y la tabla de canciones ordenadas por pista, con la duración en formato m:ss y el pie "Melodic Vault - Página 1".

La URL /reportes/album/1 confirmó que el documento se generó en el momento desde la aplicación.

Figura 92 : Reporte de álbumes de una banda ejecutado desde la aplicación





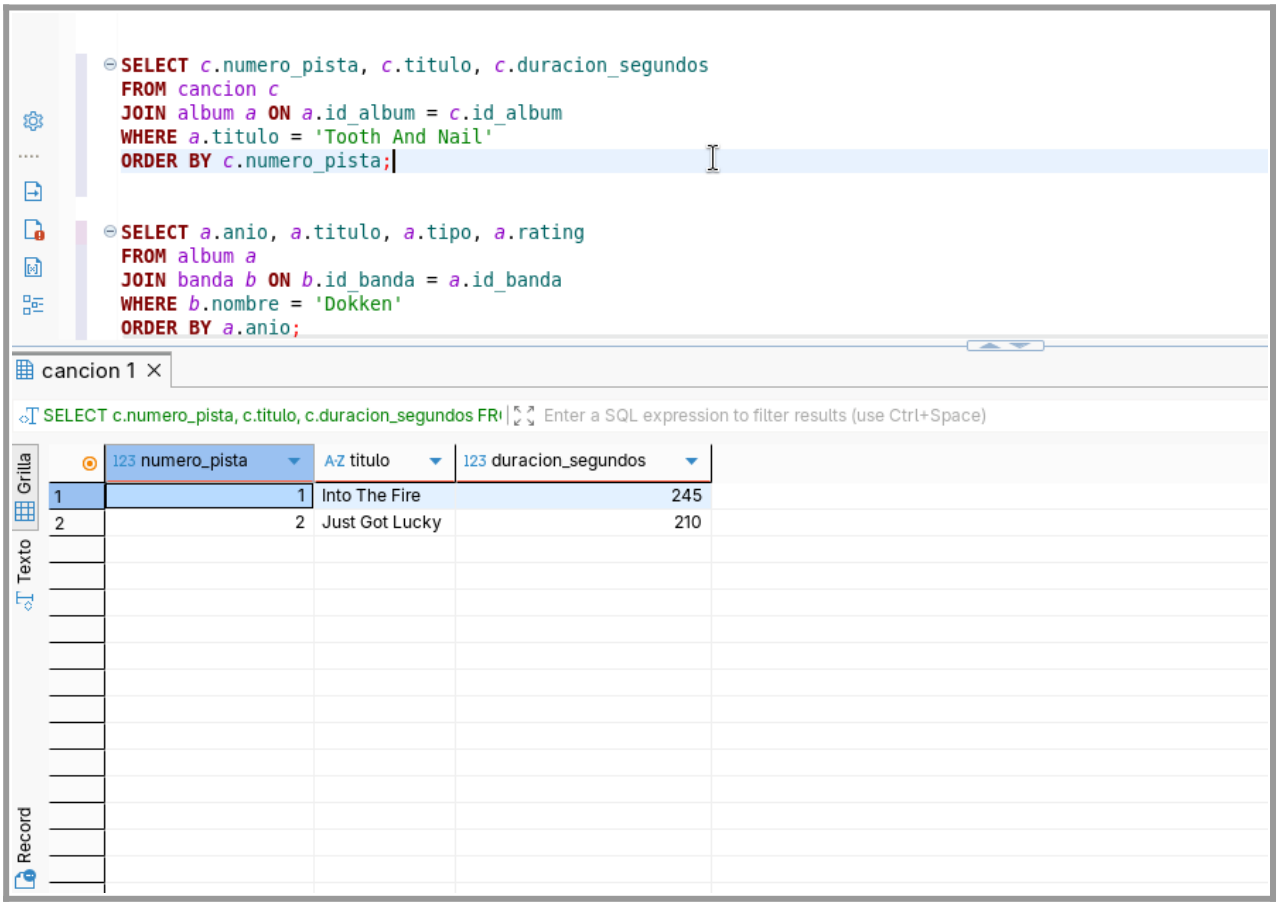
Año	Título	Tipo	Rating
1984	Tooth And Nail	Studio Album	9,2
1987	Back for Attack	Studio Album	9,6
2000	mil canciones	EP	9,0

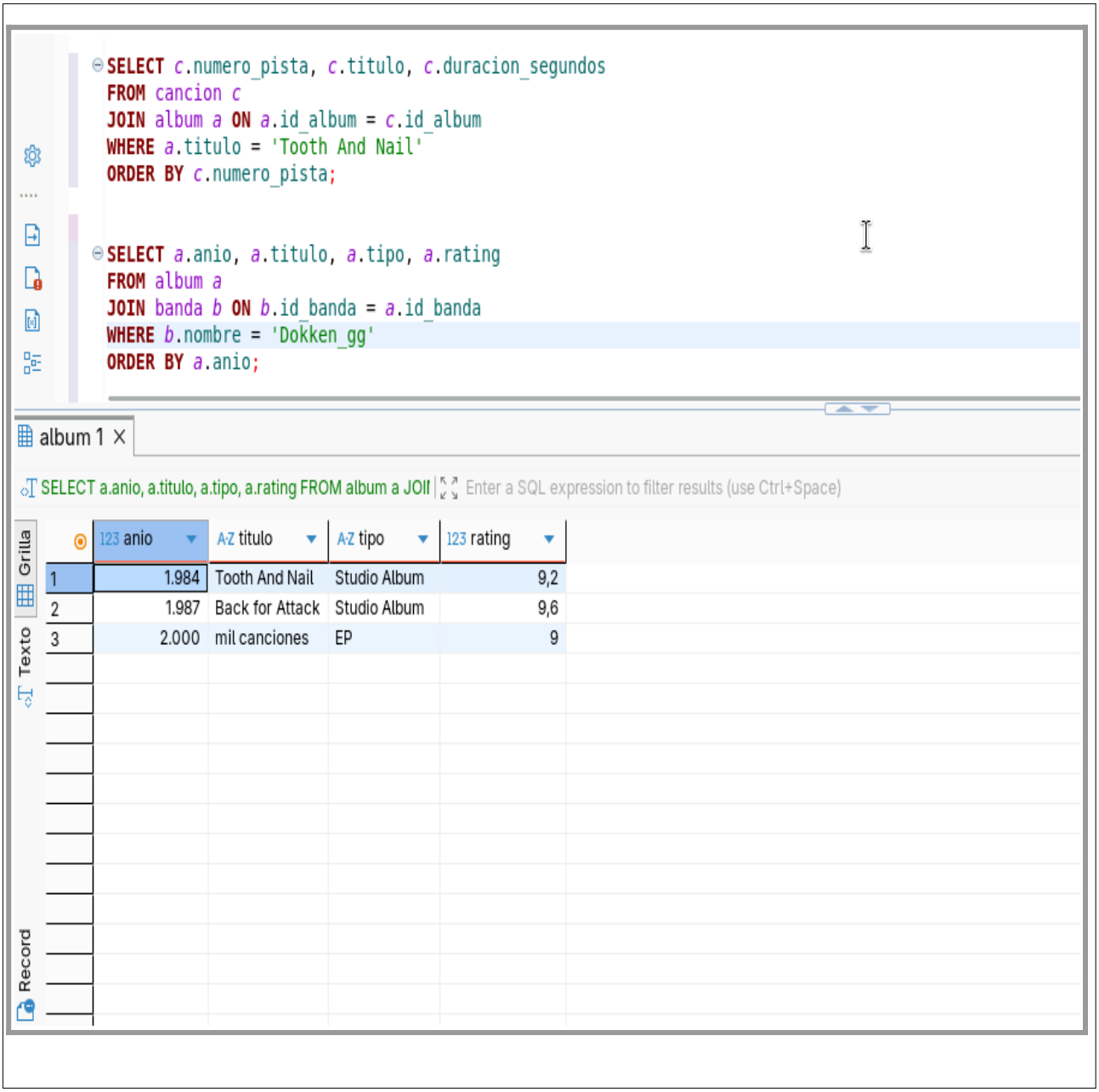
INTERPRETACION

.El Descargar PDF del detalle de banda abrió el reporte con el nombre, país, género y año de formación de Dokken, y la tabla con sus álbumes (año, título, tipo y rating).

Los datos de cabecera y de la tabla coincidieron con los registrados en la base de datos.

Figura 93 : Datos de MySQL utilizados por los reportes





INTERPRETACION

Las filas devueltas por las consultas coincidieron con las que aparecieron en los PDF de las Figuras anteriores lo que comprobó que los reportes se generaron con información real de la base de datos y no con datos fijos en la plantilla.

8. Conclusiones

1. **Análisis de requerimientos**. Se levantaron 21 requerimientos funcionales, 14 no funcionales y 30 reglas de negocio, organizados en 21 casos de uso con sus especificaciones y en un backlog Scrum de tres sprints, todos cumplidos. Esto definió con precisión los tres actores (Visitante, Lector y Administrador) y qué podía hacer cada uno, y dejó el alcance del catálogo musical acotado y verificable.

2. **Diseño de la base de datos y la arquitect**ura. El modelo relacional quedó en cuatro tablas: banda, album y cancion en cadena 1:N, y usuario independiente. La integridad se aseguró desde MySQL con claves foráneas con eliminación en cascada y restricciones únicas (username y el par id_album, numero_pista). La arquitectura de tres capas con MVC separó vistas, controladores, servicios y repositorios, de modo que cada capa tuvo una sola responsabilidad.

3. **Implementación de los módulos conSpring Boot**. Se implementaron los tres mantenimientos CRUD (bandas, álbumes y canciones), las búsquedas por nombre, país o género y por título, tipo o banda, y el registro público de usuarios con rol Lector. El acceso se controló con Spring Security y BCrypt, validado en el servidor y no solo en la vista. El caso transaccional de registro de un álbum con hasta cinco canciones, con @Transactional, se guardó completo o no se guardó: la prueba con una pista repetida no dejó ni el álbum ni sus canciones en la base. Todo se comprobó con evidencia en la interfaz y en MySQL.

4. **Reportes con JasperReports.** Se generaron dos reportes PDF a partir de plantillas .jrxml compiladas una sola vez al iniciar la aplicación: canciones de un álbum y álbumes de una banda. Se descargaron desde los detalles solo con sesión iniciada, y sus datos coincidieron con los registros de MySQL, lo que comprobó que se alimentaron de información real.

5. **Control de versiones y despliegue.** Git y GitHub gestionaron el código y la documentación con las ramas main, desarrollo y hotfix, 54 commits de cuatro integrantes e integraciones por fast-forward, con trazabilidad del aporte de cada uno. Maven resolvió las dependencias y generó el MelodicVault-0.0.1-SNAPSHOT.jar ejecutable con validate, compile, package e install.

Cumplimiento del objetivo

Se desarrolló una aplicación web funcional que centralizó la información de bandas, álbumes y canciones en un catálogo estructurado, con consulta pública, mantenimiento restringido y reportes, lo que respondió a la dispersión de información que originó el proyecto. Se aplicaron las tecnologías del curso (Maven, JPA e Hibernate, Spring Boot, Spring Security, Thymeleaf, JasperReports y Git/GitHub) y se obtuvo una solución ordenada por capas y con capacidad de crecimiento.

9. Recomendaciones

Gestión de cuentas (conclusión 3). Incorporar recuperación y cambio de contraseña, y una pantalla de gestión de usuarios solo para el Administrador. Hoy el rol ADMIN se asigna con SQL y el registro no inicia sesión automáticamente.

Aportes de los Lectores (conclusión 3). Permitir que el Lector registre álbumes y edite solo los que él registró. Requeriría guardar el autor en el álbum (por ejemplo, una clave foránea id_usuario hacia usuario) y verificar en el servicio que el autor sea el usuario en sesión antes de editar. Esto ampliaría la regla de negocio que hoy reserva todo el mantenimiento al Administrador. Convendría que el Administrador moderara esos aportes antes de publicarlos.

Pruebas automatizadas (conclusión 3). El proyecto no tiene src/test. Se recomienda agregar pruebas con JUnit y Spring Boot Test sobre los servicios, sobre todo AlbumTransaccionalService y las reglas del registro.

Registro de canciones desde el álbum (conclusiones 2 y 3). Agregar el botón «Agregar canción» en el detalle del álbum y en cada tarjeta del listado, que abra el formulario con el álbum ya seleccionado. El formulario sugeriría como número de pista el mayor existente del álbum más uno, mediante una consulta MAX(numeroPista) en ICancion, y el administrador podría modificarlo. Así se evitaría buscar el álbum en una lista extensa y se reducirían los errores de pista repetida, que de todos modos seguiría controlando la restricción única de la base de datos. También convendría agregar búsqueda y detalle de canciones, paginar los listados y usar fragmentos de Thymeleaf para no repetir cabecera, menú y pie.

Robustez de la persistencia (conclusión 2). Cargar las relaciones LAZY de los detalles con @EntityGraph, como ya se hizo en el reporte de banda, para no depender de open-in-view. Versionar el esquema con Flyway, porque ddl-auto=update no es confiable con restricciones.

Reportes (conclusión 4). Ofrecer exportación a Excel o CSV y un reporte del catálogo completo.

Despliegue y evolución (conclusión 5). Desplegar en Azure con integración continua en GitHub Actions. Exponer los services mediante una API REST permitiría reutilizarlos desde una app móvil sin tocar la lógica de negocio. Alojar las imágenes propias, en lugar de enlazar portadas externas, evitaría problemas de derechos de autor y enlaces rotos

10. Glosario

Definición de términos técnicos usados

1. **Arquitectura MVC (Modelo–Vista–Controlador):** Patrón arquitectónico que organiza una aplicación en tres componentes: modelo, encargado de los datos y la lógica; vista, responsable de la presentación; y controlador, que procesa las solicitudes y coordina las respuestas.
2. **Base de datos relacional:** Conjunto organizado de información almacenada en tablas relacionadas mediante claves, que permite mantener estructuras de datos consistentes y realizar consultas.
3. **Bootstrap:** Framework de interfaz que proporciona estilos y componentes reutilizables para desarrollar páginas web adaptables a diferentes tamaños de pantalla.
4. **BPMN (Business Process Model and Notation):** Notación gráfica utilizada para representar procesos de negocio, identificando actividades, participantes, decisiones y secuencias de trabajo.
5. **Caso de uso:** Descripción de una interacción entre un actor y un sistema para alcanzar un objetivo específico. Permite identificar funcionalidades y establecer requisitos desde la perspectiva del usuario.
6. **Clave foránea (Foreign Key):** Campo que establece una relación con la clave primaria o única de otra tabla, contribuyendo a mantener la integridad referencial de los datos.
7. **Clave primaria (Primary Key):** Campo o conjunto de campos que identifica de manera única cada registro de una tabla, evitando la existencia de identificadores duplicados.
8. **Commit:** Confirmación que registra un estado del proyecto dentro del historial de Git. Cada commit dispone de un identificador y contiene información asociada, como autor, fecha y mensaje descriptivo.
9. **Controlador (Controller):** Componente encargado de recibir solicitudes del usuario, interactuar con los servicios correspondientes y determinar la respuesta o vista que debe presentarse.
10. **CSS (Cascading Style Sheets):** Lenguaje de estilos utilizado para definir la presentación visual de una página web, incluyendo colores, tamaños, márgenes y distribución de elementos.
11. **Fast-forward:** Tipo de actualización en Git que permite avanzar directamente una referencia hacia un commit descendiente, sin generar un commit adicional de fusión.
12. **Git:** Sistema de control de versiones distribuido que permite registrar modificaciones en archivos, consultar versiones anteriores y administrar diferentes líneas de desarrollo.

13. **GitHub:** Plataforma de alojamiento de repositorios Git que facilita la colaboración entre desarrolladores, la revisión de cambios y la administración del historial de un proyecto.

14. **Hibernate:** Framework de mapeo objeto-relacional utilizado habitualmente como implementación de JPA. Facilita la correspondencia entre las clases Java y las tablas de una base de datos.

15. **HTML (HyperText Markup Language):** Lenguaje de marcado utilizado para estructurar el contenido de las páginas web mediante elementos como encabezados, formularios, tablas y enlaces.

16. **Java:** Lenguaje de programación orientado a objetos utilizado para desarrollar la lógica del sistema web Melodic Vault. Permite implementar clases, entidades y funcionalidades del lado del servidor.

17. **JasperReports:** Biblioteca de generación de reportes para aplicaciones Java que permite diseñar y producir documentos a partir de datos estructurados, incluyendo reportes exportables a PDF.

18. **JPA (Jakarta Persistence API):** Especificación de Java que define mecanismos para representar y gestionar datos relacionales mediante objetos y entidades.

19. **Maven:** Herramienta de automatización utilizada en proyectos Java para gestionar dependencias, compilar código y ejecutar procesos de construcción mediante la configuración del archivo pom.xml.

20. **Merge:** Operación de Git utilizada para integrar los cambios de una línea de desarrollo en otra. Dependiendo de la relación entre sus historiales, puede realizarse mediante avance rápido o requerir un commit de fusión.

21. **Spring Boot:** Herramienta del ecosistema Spring que simplifica la configuración y ejecución de aplicaciones Java. Facilita el desarrollo de sistemas web mediante configuraciones automáticas y dependencias integradas.

22. **Spring Data JPA:** Módulo de Spring que facilita la implementación de operaciones de persistencia mediante repositorios, reduciendo la necesidad de escribir manualmente consultas para operaciones comunes.

23. **Thymeleaf:** Motor de plantillas utilizado en aplicaciones Java para generar páginas HTML dinámicas mediante la incorporación de datos provenientes del servidor.

11. Bibliografía

Chacon, S. &. (2014). *Apress.* Obtenido de Pro Git (2.ª ed.).: https://git-scm.com/book/es/v2

Consortium., W. W. (23 de septiembre de 2026). *Web sustainability guidelines (WSG)* . Obtenido de ttps://www.w3.org/TR/web-sustainability-guidelines/

Docs., G. (. (s.f.). *GitHub*. Obtenido de https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories

Informática, I. N. (2026). *Estadísticas de las tecnologías de información y comunicación en los hogares: IV trimestre 2025 (Informe técnico N.º 01, marzo de 2026).* . Obtenido de https://www.inei.gob.pe/media/MenuRecursivo/boletines/boletin-tic-oct_dic2025.pdf

Ministros., P. d. (2021). *Decreto Legislativo N.º 822: Ley sobre el Derecho de Autor. Plataforma Digital Única del Estado Peruano*. Obtenido de https://www.gob.pe/institucion/pcm/normas-legales/1670023-822

Ministros., P. d. (2023). *Decreto Supremo N.º 085-2023-PCM: Decreto Supremo que aprueba la Política Nacional de Transformación Digital al 2030.* . Obtenido de https://www.gob.pe/institucion/pcm/normas-legales/4471543-085-2023-pcm

Oracle. (s.f.). *MySQL 8.4 reference manual.* Obtenido de https://dev.mysql.com/doc/refman/8.4/en/

Organización de las Naciones Unidas para la Educación, l. C. (2022). *Nuevas oportunidades y retos para unas industrias culturales y creativas inclusivas en el entorno digital*. Obtenido de https://www.unesco.org/reports/reshaping-creativity/2022/es/nuevas-oportunidades-retos-para-industrias-culturales-creativas-inclusivas-entorno-digital

Spring. (s.f.). *Spring Boot reference documentation.* Obtenido de https://docs.spring.io/spring-boot/reference/

12. Anexos

Diagramas originales (Bizagi u otro si lo considera prudente).

Tablas, listas, actas de reuniones, capturas de sistemas, etc.

Otro que considere pertinente.